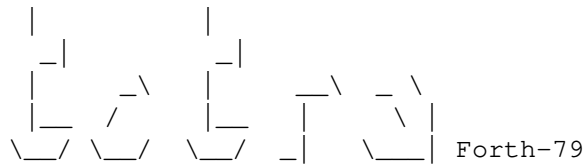


tetra Manual

Antonio Maschio
with the help of **Ian Jones** and **Bruce Axtens**

ABSTRACT



Introducing the **tetra** Forth-79 interpreter. **tetra** is a Forth-79 interpreter with Double-Number Standard Extensions, based on the Forth-79 Standard (1980).

This manual is not a Forth primer; I assume you have the most common knowledge about programming in Forth; this is more like a reference manual that helps in using correctly the various dictionary words and understand the memory state. You may also find **tetra** man page useful for the console execution options, other more general info and the installing instructions.

I here thank **Ian Jones** and **Bruce Axtens**, the ones who shared with me this creation. They discovered many bugs, suggested many things, cleared many concepts, tested everything, showing great patience throughout the whole process and supporting my discouragements. They helped me to interpret some features of the Forth-79 Standard. Despite never meeting in person, our friendship has grown strong through our shared work. Thank you, Ian and Bruce.

I don't belong to any Forth circle, no Forth group, no Forth organization; no university, no school, no structure. And I'm not a professional programmer, so please don't tell me *you should've done this that way or that this way!* because I know that my programming style is pretty naive (and yes, I use the go-tos!). I want to make my algorithms work: speeding them up and making them beautiful are tasks that go beyond my interests.

And since I make errors (as anybody), readers are encouraged to report all errors and inconsistencies (both in the program and in the manual) to:

ing dot antonio dot maschio at gmail dot com

Support for my efforts is appreciated. Donations can be made via PayPal to

tbin at libero dot it

Thanks.

19 June 2026

*Per correr miglior acque alza le vele
omai la navicella del mio ingegno...*
(Dante, Purgatorio, Canto I, 1472)

To run o'er better water hoists her sails
the little vessel of my genius now...
(Dante, Purgatory, Chant I,
transl. Courtney Langdon, 1920)

1. GENERAL NOTES

1.1. Introducing tetra

In late 1983 (or perhaps early 1984, I was seventeen), a good schoolmate and friend of mine, Alessandro (also known as Sguinz), excitedly shared with me during a classroom break: 'I've just read about a fantastic new programming language!' He quickly jotted down:

```
2 3 + DUP 4 SWAP
```

explaining me the stack, the basic stack manipulation words, and in general how Forth works. This got me hooked. I began insisting with my mother to buy me a Forth book (any) and finally she agreed. The book of Alan Winfield was mine. I read it dozens of times, did all the exercises, tried to get into the complexity of the defining words, but what I lacked was a real environment. No computer, only theory. Those were days of serene lack of electronic devices of any kind...

After years of curiosity, my journey with Forth took a new turn in 2011 (now adult with children) when I decided to design my own Forth compiler. The development soon stopped, because of my job and also because I had begun a carrier in resurrecting old forgotten programming languages by writing interpreters for them (BASIC and ALGOL interpreters, mainly). In 2017 I resurfaced the project, developed it a bit more and finally, in 2020, the Alpha version was out (at those times called *qartus*).

Why Forth? Perhaps because I enjoy programming, I find Forth appealing, and I love creating something by assembling smaller component. But principally because Forth is the most fascinating imperative language ever created, in my opinion.

Why Forth-79? I chose Forth-79 because it represents the first standardization of the Forth language, offering simplicity and historical significance, and it was the version I initially studied.

tetra is a **gcc** program for Linux or UNIX (compilable also by **clang**), it's completely free and it's distributed according to the GNU Public License version 3. It has all the most advanced structure control and cycling words (most coming from the modern ANSI version) but it retains the Forth-79 philosophy, and it's completely Forth-79 compliant. You are welcome to use it, but even if you choose not to, I ask that you report any errors or inconsistencies you may encounter: I will try to solve them as soon as possible.

The name *tetra* derives from the Ancient Greek word for *four*, suggesting a similitude with *fourth*. I hope you like this name, because it is the last of a long name series. The mascot is a fish, because *tetra* is also the name of a fish species.

Final note: I wrote this book using **vim/gvim** and **groff**. If you notice any mistakes or formatting issues, please let me know, so I can correct them.

I'm the only responsible for the things written in this manual. The directory `docs/src` of the package contains all the files needed to recompile the manual in case you lose it, provided you have **groff** and **ps2pdf** installed on

I understand this nickname might be difficult to pronounce outside of Veneto. It sounds similar to the English word 'swince,' but with a hard 'g' between the 's' and the 'w' (like the 'g' in 'get').

More here: <https://www.urbandictionary.com/define.php?term=Swince>.

your system. Simply enter this directory (in the project <home>) and type `make manual`:

```
<home>/ $ cd docs/src/  
<home>/docs/src $ make manual
```

and a fresh new copy of the pdf manual will be available to you. You can also download a free copy from the home page of **tetra**. You are free to choose the option you want.

1.2. The documents

tetra is an implementation of Forth-79, following the October 1980 document by the Forth Standard Team. Its aim is to give to the programmer the most complete number of words for programming, with the ambiguities of the 1979 Standard resolved. I don't know if I succeeded, really, in this huge task.

- The Forth-79 manual can be downloaded here:

<https://www.complang.tuwien.ac.at/forth/standards/Forth-79.pdf>

This document is included in the **tetra** package.

A textual document with the same text can be found here:

<https://www.complang.tuwien.ac.at/forth/fth79std/FORTH-79.TXT>

tetra's reference manual is modeled on this document.

- To create **tetra** I have read again (for the 100th time or so) the wonderful book of Alan Winfield "*The complete Forth*" (Italian edition, titled "*Forth - corso di programmazione per microcalcolatori*"), 1983, which belongs to my books collection since then, when I was a boy. The compiler used by Mr. Winfield was R-Forth, maybe a Forth-79 with some fig-Forth issues (for instance SIGN, DPL), or maybe a fig-Forth with some Forth-79 insertions... who knows...
- The fig-Forth manual too was a source of information. I've read the 1986 book "*Extended fig-Forth rev. 1*"; you can download it here (mistake included):

http://www.atarimania.com/documents/Extended_Fig_Forth_rev_1.pdf

The fig-Forth glossary (release 1) is available here:

<http://www.cs.drexel.edu/~bls96/6809sbc/doc/figdoc/glossary.txt>

The fig-Forth is the elder cousin of Forth-79.

- The two articles "*Upgrading Forth-79 Programs*" about how to translate code written for Forth-79 to Forth-83, available in "*Forth Dimensions*" Vol. VI, N. 3 September/October 1984 and downloadable at

<http://www.forth.org/fd/FD-V06N3.pdf>

and "*Forth-83 Program to Run Forth-79 Code*" in "*Forth Dimensions*" Vol. VI, N. 4 November/December 1984, downloadable at

<http://www.forth.org/fd/FD-V06N4.pdf>

were also useful to understand the real nature of Forth-79.

- An important resource file was the manual of Hartforth, a Forth-79 engine for the TRS80 computer, available at

<http://tim-mann.org/trs80/doc/forth.pdf>

- Of course I couldn't continue without mentioning the two imperative books by Leo Brodie, "*Starting Forth*" (1981), substantially written in fig-Forth in the book version (and ANSI-fied in the online version), and "*Thinking Forth*" (1984), written in Forth-83 (and ANSI-fied in the online version). The first is available online here:

Original scanned version (fig-forth side)

<http://www.forth.com/wp-content/uploads/2018/01/Starting-FORTH.pdf>

ANSI-fied version (ANSI 1994 side) <https://www.forth.com/starting-forth/0-starting-forth/>

while the second can be downloaded (only as the ANSI 1994 side, AFAIK) as a pdf file here:

<https://www.forth.com/forth-books/>

(scroll down to see the links); the pdf file is good if you have no latex system on your UNIX machine (or if you have no UNIX machine), otherwise you can download the latex files and compile.¹

Enough? No! I consulted many other specialized texts, here listed:

- Of course, the first and only Forth Handbook is the Conklin/Rather "*Forth Programmer's Handbook*"; I have a printed copy of the First Edition, October 2001. Of course it refers to the ANSI 1994 Forth, but it's a wonderful and clear source of information. You can find a more recent version on Internet (Searching for "Forth Programmer's Handbook").
- Similarly important (even if quite a rewriting of the previous) is the Elizabeth D. Rather book "*Forth Application Techniques*" (Sixth edition, 2019). This version is targeted for the 2012 ANSI, but in any case it is an illuminating textbook.
- Some ideas came from "*Forth on the BBC microcomputer*", by Richard De Grandis Harrison, January 17, 1983 (where I grabbed some ideas for the Screen Editor above all). This book was a great mine of information in all cases where the Standard rules were not clear. The book, published by Acornsoft, is available at

<http://stardot.org.uk/mirrors/www.bbcdocs.com/filebase/software/apps/forth-on-the-bbc-microcomputer.pdf>

- Some other ideas came from the manual "*microForth Primer*", second Edition, by the Forth, Inc., dating back to August 1978 (with Elizabeth Rather behind the book, I suppose), and that can be found (along with other documentation about microForth) here:

<https://www.forth.com/resources/archive-forth-systems-software/>

- The manual of the Max-Forth for the 68HC11 (a ROM-based operating system and language) was indeed very very useful; you can find it here:

<http://www.science-info.net/pages/GordonCouger/stuff/68hc11/1FORTH-programs/MASTERV3.ASC>

¹ I'm so lucky to have an original printed copy of it (Forth-83 side), which is invaluable, for me. A sort of secular bible I turn to from time to time.

A glossary of Max-Forth is also available here:

<http://www.newmicros.com/FORTH/HC11/V35/Alphabetical/Detailed/index.html>

- The manual of the tForth (Jun 1987), by Jef Raskin, is a wonderful conceived textual document, that can be found here²:

<http://www.canoncat.net/cat/Cat%20tForth%20Documentation.txt>

- How not to mention the manual of PolyForth, by Elizabeth D. Rather, Leo Brodie and the Technical Staff of FORTH, Inc.? PolyForth was somehow more than a Forth engine, and more like a Forth shell, the grandpa of SwiftForth. This manual gave also the base for the Forth Handbook (and later the sf manual). The pdf of the Fifth Edition is available here:

<http://www.greenarraychips.com/home/documents/greg/DB005-120825-PF-REF.pdf>

- Kim Harris's Forth Course, dating back to June 1980 and using fig-Forth, was a good lecture (it's not a book, rather a collection of slides), and can be found here:

<http://www.forth.org/KimHarris/KimHarris.html>

- The manual of the jforth, whose glossary is a huge effort (something I'll never be able to repeat...), can be found here:

<http://www.jforth.org/TableOfContents.html>

- Have you ever read "*Forth Dimensions*"? It was a wonderful review, dedicated to Forth only. It's a pity for me (and maybe for some of you) that I became aware of it only after it ceased publications. In the published reviews I found many programs (some of them were transcribed by me and included in tetra's package as block or files). This review can be found in many places, nowadays; I use the site www.forth.org. Another source can be found at

<https://www.complang.tuwien.ac.at/forth/forth-dimensions/>

- I cannot omit "*Forth for the complete Idiot*", by C.H. Ting, issued in 1983 and revised in 1984 (79-oriented, and targeted for the APPLE Forth). Here's an excerpt of the introduction: "*FORTH is a language of mystic quality. The learning process for many FORTH programmers can be described only in religious terms. However, what I consider to be the greatest myth about FORTH is that FORTH IS AN ELITIST'S LANGUAGE. This myth is perpetuated because most people think that FORTH is difficult to learn, FORTH programs are difficult to read and to comprehend, and that a FORTH computer is expensive, at least comparing to most other personal computers running BASIC. My opinion is that FORTH IS THE IDIOT'S LANGUAGE which is best suited for common people who are being shuffled into computer literacy, kicking and screaming. Why? Because FORTH is actually a very simple language to learn. It has the simplest grammar and the fewest syntax rules than any other computer languages. Above all, FORTH can be extended and modified so that you can use your own language or terms to converse with it. I was asked what is the qualification of a person to learn FORTH. My answer was that he must have at least one finger, to hit keys on a keyboard. Well, what else does he need to learn FORTH?*". You can find this essay here:

<http://www.forth.org/Ting/Forth-for-the-Complete-Idiot/Forth-for-the-Complete-Idiot.pdf>

²I must confess that the original name of my project was tforth, but I didn't know it was used something like 30 years before. So, when I finally stumbled into this site, I had to change the project's name to yforth, not without a tip of sorrow. 't' stood for Tony, of course! But this is not the entire story! Read on.

Enjoy this minimalistic manual!

- I must also cite this forth interpreter, called yforth, made by an Italian like me by the name of Luca Padovani in 1997 and developed until 2012. The name of yforth was the name that I'd been using for years, and it was only by chance that I discovered it was already in use³. You can find yforth at the following site:

<https://packages.debian.org/sid/interpreters/yforth>

- I must also cite Quartus⁴ Forth... To quote the site, Quartus "*...is a Forth Language compiler for PalmOs-based HandHeld computers. It allows development of applications on the device itself. Version 2.0 now supporting ARM based OS 5 and future OS 6 platforms.*

For more information, see

<http://quartus.net/products/forth/>

- Last (but not least), the marvellous guide to implement Forth-79 on a fig-Forth system, titled "*Forth-79 Standard Conversion*", by Robert L. Smith, dating back to 1981, and downloadable from Stackosaurus here:

http://www.stackosaurus.com/misc/Forth-79_Standard_Conversion.pdf

Behind the text lines you can see a huge brain work... The Stackosaurus site has also a page devoted to fig-Forth here:

<http://www.stackosaurus.com/figforth.html>

I have also consulted reference manuals of the further evolutions of Forth, specially Forth-83, whose manual is available here:

<http://forth.sourceforge.net/std/fst83/>

Forth-83 was very useful to detect the real nature of a Forth-79 compiler, by comparing their different features.

Many others are available over the Internet, I couldn't survey them all. Anyway, for any other technical Forth-related data you need to know, consult the site

<https://www.forth.com/resources/forth-programming-language/>

and read the Leo Brodie books! Better than anything else!

³ Again! after tforth, after yforth, I had to change the project name again, and chose quartus. Will maybe this be the last? No, unfortunately...

⁴ Oh my! After tforth and yforth, I had chosen qartus as the name of my Forth engine. But *qartus* is too much similar to *Quartus*, and they basically seem the same word. So I turned to *tetra*, the current and final name.

1.3. Definition of terms

These definitions clarify the **tetra**'s environment terminology used into this manual. They, when in lower case, are terms also used within the Standard, and present terms that are specifically used within Forth.

address, byte

An unsigned number that locates the first of two 8-bit bytes in the memory address space over the range {0..65535}. It is a virtual representation on a memory mapping over two bytes. Address arithmetic is modulo 65536 without overflow.

address, code field

The address of the first byte of memory in a word definition, where execution codes (in a colon-definition) are stored. This is the address returned by `FIND`.

address, compilation

The numerical value equivalent to a word definition, which is compiled for that definition. The address interpreter uses this value to locate the machine code corresponding to each definition.⁵

address, parameter field

The address of the first byte of memory associated with a word definition for the storage of compilation addresses (in a colon-definition), numeric data and text characters. This is the address returned by `'` (tick).

arithmetic

All integer arithmetic is performed with signed 16 or 32 bit two's complement results, unless noted.

block

The unit of data from mass storage, referenced by a block number. A block contains 1024 bytes. The translation from block number to device is done internally through UNIX calls, by mapping the storage block file content to a dedicated buffer in memory (see `'screen'`).

Block numbers range is {1..65535}. Block 0 is not available.

block buffer

A memory area where a mass storage block is maintained (see also `'screen'`).

byte

An assembly of 8 bits. With reference to memory, it is the storage capacity for 8 bits.

cell

A 16-bit memory location. The n-th cell contains the 2n-th and (2n+1)-th byte of **tetra**'s address space. When represented on the stack, the number is retrieved as a whole 16-bit number. When in memory, the lower 8-bits are at the lower address.⁶ For the definition of `'cell pair'`, see `'string'`.

⁵ The Standard specifies here: "*([It] may also be called the code field address.)*", but **tetra** uses a different method, compiling the Link Field Address (as a 32-bit number). The code field address can be derived from a fixed size distance from LFA (4 bytes + the name length). In any case, this is transparent to the user.

⁶ I must note that in the Standard the byte order is expressively ignored (page 3: "*The byte order is presently unspecified*"). When I designed this feature for **tetra**, I decided to use the same pattern generally used in previous Forth-79 compilers, probably designed to ease the usage of `C@`

character

An 8-bit number which represents a terminal character. The ASCII character set is used. When contained in a larger field, the higher order bits are zero. 7 bits are taken into account for the default range {1..127}. The correspondence with keyboard keys and ASCII control codes (in the range 1-31) are contained in the following table:⁷

Hex	ASCII	CTRL-Key	Hex	ASCII	CTRL-Key
0	NUL	(*)	10	DLE	CTRL-P
1	SCH	CTRL-A	11	DC1	(*)
2	STX	CTRL-B	12	DC2	CTRL-R
3	ETX	(*)	13	DC3	(*)
4	EOT	CTRL-D	14	DC4	CTRL-T
5	ENQ	CTRL-E	15	NAK	CTRL-U
6	ACK	CTRL-F	16	SYN	CTRL-V
7	BEL	CTRL-G	17	ETB	CTRL-W
8	BS	CTRL-H	18	CAN	CTRL-X
9	HT	CTRL-I	19	EM	CTRL-Y
A	LF	CTRL-J	1A	SUB	(*)
B	VT	CTRL-K	1B	ESC	(*)
C	FF	CTRL-L	1C	FS	(*)
D	CR	CTRL-M	1D	GS	CTRL-5
E	SO	CTRL-N	1E	RS	CTRL-6
F	SI	CTRL-O	1F	US	CTRL-7

(*) codes intercepted by the O.S. or by **tetra** and having a specific purpose.

The ASCII table for all normal printing characters in the range 32-127 are here reported:

Hex	ASCII	Hex	ASCII	Hex	ASCII	Hex	ASCII	Hex	ASCII	Hex	ASCII
20		30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	\$	34	4	44	D	54	T	64	d	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~

The range 128-255 depends on the Operating System and may vary from computer to computer, and it's not represented here. Any character sequence in the input stream is case-sensitive only for the strings, which are sequences of characters enclosed in double quotes; any other sequence corresponding to Forth words is case-insensitive, included created-words names (using e.g. VARIABLE, CREATE etc..).

and C! directly on the address number. I must thank **Ian Jones** for pointing me out the problem, that I had initially addressed in the opposite way.

⁷ Using KEY you can obtain these control ASCII codes, typing CTRL plus the relative key, except for those marked as (*).

compilation

The action of accepting text words from the input stream and placing corresponding compilation addresses/codes in a new dictionary entry to be executed later.

counted string

See 'string'.

defining word

A word that, when executed, creates a new dictionary entry through compilation. The new word name is taken from the input stream. If the input stream is exhausted before the new name is available, an error condition exists. The built-in defining words are:

```
2CONSTANT 2VARIABLE : CONSTANT CREATE VARIABLE VOCABULARY
```

Any word that contains a defining word, or equivalently that contains another word defined via a defining word, is classified as a defining word.

definition

See 'word definition'.

digit

(*definition by Robert L. Smith*) A digit is any one of a set of ASCII characters which represent numeric values in the range from 0 to base-1. For bases greater than decimal 10, the set of characters is {0..9 A B C..Z} where the ascending ASCII sequence is used for A and above. The digit sequence is always case-insensitive.

dictionary

The set of word definitions in a computer memory. The dictionary entries are organized in vocabularies to enable location by name. The dictionary is extensible, growing toward high memory.

division, symmetric

Integer division in which the remainder carries the sign of the dividend or is zero, and the quotient is rounded toward zero. Note that, except for error conditions, the sequence `n1 n2 SWAP OVER /MOD ROT * +` is identical to `n1`.

Examples:

dividend	divisor	remainder	quotient
10	7	3	1
-10	7	-3	-1
10	-7	3	-1
-10	-7	-3	1

equivalent execution For the execution of a standard program, a set of non-time dependent inputs will produce the same non-time dependent outputs on any FORTH Standard System with sufficient resources to execute the program. Only standard source code will be transportable.

error condition

An exceptional condition which requires action by **tetra**. The most general action is to display a message, returning control to the user.

The General Error Conditions (GCE) are:

1. Input stream exhausted before a required <name>.
2. Empty stack and full stack for the text interpreter for all explicit and implicit stacks.
3. Unknown word or an invalid number for the text interpreter.
4. Compilation of incorrectly nested conditionals and loops.
5. Interpretation of words restricted to compilation.
6. Compilation of words restricted to interpretation.
7. FORGETting a dictionary word which is beyond the imposed FENCE.
8. Insufficient remaining space in the dictionary or in the buffers.

In the built-in words Dictionary chapter, all situations where the error condition may manifest are reported; in all other unmentioned cases, the GCE previously defined takes place by emitting an error message and stopping the execution, returning control to the operator.

false

A zero number representing the false condition flag. All words returning a flag leave 0 as a 'false' flag.

flag

A number that has only two logical states, zero and non-zero. These are named 'true' = non-zero, and 'false' = zero. Words that test a condition leave 1 for true, 0 for false.

glossary

A set of word definitions given in a natural language describing the corresponding computer execution action.

immediate word

A word defined to automatically execute when encountered during compilation, which handles exception cases to the usual compilation. See IF, DO, ." etc.

input stream

A sequence of characters available to the system, for processing by the text interpreter. The input stream is conventionally taken from a terminal (via the terminal input buffer where keyboard characters are stored) and mass storage (via a block buffer or via a file). >IN and BLK respectively specify the position in the input stream and the block number used to load the input stream. Words using or altering >IN and BLK are responsible for maintaining and restoring control of the input stream.

interpreter, address

The (set of) word definitions which interprets (sequences of) FORTH compilation addresses by executing the word definition specified for each one.

interpreter, text

The (set of internal) word definitions that repeatedly accept a word name from the input stream, locate the corresponding dictionary entry, and start the address interpreter to execute it. Text in the input stream interpreted as a number leaves the corresponding value on the data stack. When in compile mode, the addresses of Forth words are compiled into the dictionary for later interpretation by the address interpreter. Numbers are compiled, and when later interpreted place the data on the stack. Numbers are accepted as unsigned or negatively signed, according to BASE.

KiB unit⁸

The kibibyte is the quantity of 1024 bytes. The unit symbol for the kibibyte is KiB.

load from a block buffer

The acceptance of text from a mass storage device and interpretation of the dictionary words, in the form of a block buffer. In the current implementation, a block is by default 1024 characters wide (1 Kib), and it is loaded in one single pass and copied to the Input Buffer for interpretation when LOAded. This is the second general method for compilation of new definitions into the dictionary.

load from a file

The acceptance of text from a mass storage device and interpretation of the dictionary words, in the form of a textual file, specifying its name as the console argument of **tetra**'s executable. In the current implementation, each line of the file, which must not be greater than 1024 characters (1 Kib), is loaded and copied to the Input Buffer for interpretation, until the end of file. This is the first general method for compilation of new definitions into the dictionary.⁹

mass storage (block buffer)

Data is read from mass storage in the form of 1024 byte blocks in case of block buffers. The editing, saving, copying, altering and formatting of these blocks is accomplished through the internal editors. When indicated as UP-DATED (modified), data will be ultimately written to mass storage in 1024 file blocks located by default in \$HOME/.tetra/blocks/, a directory created by the --config option (see also BLOCK). See the man page for the differences with the Windows© configuration.

mass storage (files)

Data is read from mass storage in the form of free-form textual files with preferred extension .ft. The editing, saving, copying, altering and formatting of these files is accomplished outside of **tetra**.

memory

The space where dictionaries, buffers and stacks are hosted, in a contiguous space in the range {0..65535}. The physical structure holding memory bytes is an array of 32-bits numbers, each called 'unit'. Each unit hosts the basic 8-bits byte capable of holding the higher or lower parts of a 16-bits number, the character in a buffer, the addresses and compilation codes (these last often in a full 32-bits format).

This structure is not perceivable by the normal user. Only through DUMP and of 32@, 32, and 32! the user can access the full 32-bits format (but this is done out of the Forth-79 Standard and is peculiar to **tetra** only).

See the paragraph "The memory model" in this manual.

⁸ from Wikipedia...

⁹ More from the inner help; type: **tetra --help** on the command line.

null

The value 0 (zero) marks the end of a string or the end of the input stream or the compiled code at the end of each definition in memory, to also mark the end of the execution tokens. Address 0 (zero) contains 0 (zero) and must not be changed, or the whole word-search engine will fail¹⁰.

number

(definition by Robert L. Smith, with integrations of mine) A number is represented in the input stream as a word composed of a sequence of one or more digits, with a leading ASCII minus (-), if the number is negative, or a leading optional ASCII plus (+), if the number is positive, and at least an ASCII dot (.), in any position (but following the sign), if the value is to be considered as a double precision number. A leading ASCII plus (+) is substantially ignored.

Commas in the number sequence (at any position, but after the sign both for numbers and double length numbers) are ignored, and can be used for the three-digit division: 2,000,000.00, which is far more comprehensible than, but equivalent to, the double length number 200000000.

Examples of illegal numbers:

- 345 the leading minus is detached
- 34.5 this is not a number (there's a dot)
- 345- the minus is in a wrong position

Examples of illegal double numbers:

- .-345 the leading minus is after the dot
- 345 this is not a double number (a dot is missing)
- , -345.0 the comma is before the sign

Numbers are classified according to the following table:

type	range	minimum bit field
bit	0..1	1
character	0..127	7
(unsigned) byte	0..255	8
number	-32,768..32,767	16
positive number	0..32,767	16
negative number	-32,768..-1	16
unsigned number	0..65,535	16
double number	-2,147,483,648..2,147,483,647	32
positive double number	0..2,147,483,647	32
negative double number	-2,147,483,648..-1	32
unsigned double number	0..4,294,967,295	32

Numbers input follows the following rules (expressed in DECIMAL):

- a number in the range {-32768..32767} is pushed as-is (included bit, character and byte formats);
- a number in the range {0..65535} is interpreted as an unsigned integer and pushed as-is;
- a number in the range {-2147483648..2147483647} with at least one dot in any position (but after the minus in case it's negative) is interpreted as a signed double integer and pushed as-is.

¹⁰ Indeed, writing in this cell is prohibited with the usual methods. But devious programmers can conceive many malicious subtle methods to override the default methods, so I have to give this advice: **don't change this value.**

- a number in the range {0..4294967295} with at least one dot is interpreted as an unsigned double integer and pushed as-is.
- The position and number of the dots in the double numbers are not important, as long as they follow the minus sign in case of negative numbers. The variable DPL holds the number of digits after the *first* dot in the number; if the number is a single number, DPL is set to -1.
- a single number outside previous proper ranges evaluates arithmetically modulo 65536.
- a double number outside previous proper ranges evaluates arithmetically modulo 4294967296.

These rules adhere, *mutatis mutandis*, to the rules of the program R-Forth (the Forth interpreter used by Alan Winfield for his "*The Complete Forth*", cit.)¹¹.

Warning: the dot and the comma are part of the double number and the comma is also part of the single number; they are parsed while reading a number, that may start with comma or dot. This is a deliberate design, so that the following commands are all valid inputs:

. . . will push a double zero
, , , will push a single zero
. . . 3 . . . will push the double number 3 This differs from the behaviour of, say gforth, that in each of the previous cases returns an error.

Warning: any bit configuration can be seen either as a signed or an unsigned number, both for single and double numbers, and the relative effects depend of course on the way the number is shown or used into calculations.

When represented in memory, numbers are contained in a cell, preceded by a code which specify the number type. The byte order within each 16-bit field is specified according to the rules defined in 'cell'.

When represented on the stack, the higher 16-bits (with sign) of a double number are most accessible. When in memory, the higher 16-bits are at the lower address. Storage extends over four bytes toward high memory.

output, pictured

The use of numeric output primitives, which convert numerical values into text characters sequences. The operators are used in a sequence which resembles a symbolic 'picture' of the desired text format. Conversion proceeds from lower to higher digit, from higher to lower memory.

packed string

See 'string'.

program

A complete specification of execution to achieve a specific function (application task) expressed in FORTH source code form.

return

The means of terminating text from the input stream. A null (ASCII 0) indicates end of text in the input stream. This character is left by the 'return' key actuation of the operator's terminal, to mark a stop for the text interpretation.

¹¹ The fig-Forth, as reported by C.H. Ting in his book "*System Guide to fig-Forth*" (1982), had the feature of recognizing hexadecimal numbers if directly followed by the letter H. This feature, in modern compilers, and starting from the Forth-79 Standard, interferes with handling numbers in bases from 18 to 36, because in these cases the letter H is a digit, and it would hardly be possible to detect the final H as a hex detector or as a normal digit.

screen

A memory space containing textual data (in Forth-79 usually called 'buffer'). By convention, a screen consists of 16 lines (numbered 0..15) of 64 characters (numbered 0..63), for a total of 1024 bytes (corresponding to 1 KiB or 1 block). Screens usually contain program source text. The first byte of a screen, once loaded, is the beginning point for text interpretation during a load. **tetra** has three independent screens with a dedicated memory space for each one.

search order

A specification of the order in which selected vocabularies in the dictionary are searched. Execution of a vocabulary makes it the first vocabulary in the search order. The dictionary is searched whenever a word is to be located by its name. This order applies to all dictionary searches unless otherwise noted. The search order begins with the last vocabulary executed and always ends with **FORTH**.

source definition

Text consisting of word names suitable for execution by the text interpreter. Such text is usually contained in textual files or arranged in blocks maintained on a mass storage device.

stack, control flow

A "last-in-first-out" (LIFO) list which contains the machine addresses of decisional words, cycle and loop words during compilation, to pass reference addresses that must be compiled into the definition, and during interpretation for temporary values passed from one word to the other. The control flow stack does not belong to **tetra**'s memory and has a separate unreachable memory space of 9765 KiB (or 9999999 bytes) of 32-bit numbers. This extra-wide control flow stack width was designed to guarantee the execution of programs written with recursive techniques which, when executed within structures, can cumulate a very high number of recursive steps. This value can be incremented by changing the constant `CFSTACKLENGTH` in '`tbas.h`', and by recompiling the sources.

stack, data

A "last-in-first-out" (LIFO) list consisting of cells. This stack is primarily used to hold intermediate values during execution of word definitions. Stack values may represent numbers, characters, addresses, boolean values, etc. The data stack belongs to **tetra**'s memory. The data stack dimension can be resized with the console option `--stack-size`.

When the name 'stack' is used, it implies the data stack.

stack, return

A "last-in-first-out" (LIFO) list consisting of cells, which contain the machine addresses of loop word definitions whose execution must be transferred in case of cycles. The return stack may cautiously be used for other values, such as loop control parameters, and for pointers for text interpretation. The return stack belongs to **tetra**'s memory. The return stack dimension can be resized with the console option `--return-stack-size`.

string

A sequence of 8-bit bytes containing ASCII characters, located in memory by an initial byte address, for at most 255 characters. When the string is represented by one value on the stack, the address, with the count on the first byte of the address, this string is defined as 'counted string' or 'packed string'. When a string is represented by two values on the stack, an address and a string count, the string is defined simply as 'string', or 'cell pair'.

A string is always case-sensitive.

If in a file the literal string is not terminated in the current line, it is automatically terminated by **tetra**. Strings thus do not extend over more lines.

string, counted

See 'string'.

string, packed

See 'string'.

string, text

A sequence of consecutive 8-bit bytes located in memory by their low memory address and length in bytes. Strings usually, but not exclusively, contain ASCII characters. When the term 'string' is used alone or in conjunction with other words it refers to text strings.

structure, control

A group of FORTH words which when executed alter the execution sequence. The group starts and terminates with compiler words. Examples of control structures: DO...LOOP, DO...+LOOP, BEGIN...WHILE...REPEAT, BEGIN...UNTIL, IF...THEN, IF...ELSE...THEN.

tetra

Tetra is a fascinating name. It's the Greek name for *four* (which undoubtedly inspired me because of its relation with *fourth*, which is the origin-name for *forth*), but it's also the name of a family of freshwater fishes, tiny sweet little fishes that we all saw in some aquarium. The mascot of the web page of **tetra** is a stylized tetra fish. You can read more about them here: <https://en.wikipedia.org/wiki/Tetra>.

tlibrary

The *tlibrary* is a library of programs and words organized in the file `tlibrary.ft` in the package. This collection can be used to transform (within certain limits) **tetra** in some other Forth version, or load in the dictionary the words you think are more useful for you. Throughout this manual, the term *tlibrary* identifies the default library, but you are free to adopt your own library and make it the default one (using the command `LIB`).

transportability

This term indicates that equivalent execution results when a program is executed on other than the system on which it was created. See 'equivalent execution'.

true

A non-zero value representing the true condition flag. Any non-zero value will be accepted by a standard word as 'true'. All words returning a flag leave 1 as a 'true' flag.

unit, memory

See 'memory'.

user area

An area in memory which contains the storage for user variables and definitions. Usually, by default, the user area is about 54 KiB cells; if the user should use the options `--stack-size` and/or `--return-stack-size`, the stacks are incremented, and the user area is correspondingly reduced. The user area includes the data and return stacks.

vocabulary

An ordered list of word definitions. Vocabulary lists are an advantage in differentiating dictionary search and in separating different word definitions that may carry the same name. More than one definition with the same name can exist in one vocabulary. The latter is called a *redefinition*. The most recently created redefinition will be found when the vocabulary is searched.

word

A sequence of characters terminated by at least one blank (or 'return'). Words are stored in input stream, from a terminal, files or mass storage device.

word definition

A named execution procedure compiled into the dictionary. Its execution is defined as a sequence of compilation addresses of other compiled words expressed in two bytes (plus any other byte necessary to complete the definition).

word name

The name of a word definition. Names are composed by any characters string up to 31 characters, and cannot contain ASCII nulls, blanks, or 'return'.¹²

word set

A group of FORTH word definitions listed by common characteristics. The sets included in **tetra** are: Required Word Set (Nucleus Words, Interpreter Words, Compiler Words, Device Words) and Extension Word Sets (Double Number Words, Editor Words, Special Addition Words). The Standard included also the Reference Word Set; some of those words are added in the Special Addition Words Set, and some are included in the library, and written in FORTH.

word set, compiler

Words which add new procedures to the dictionary or aid compilation by adding compilation addresses or data structures to the dictionary.

¹² The Standard writes "...*Standard names must be distinguished by their length and first thirty-one characters*", suggesting that names might be longer, but only the first 31 make the difference. I found this memory-consuming, and practically inapplicable. So, at the cost of nothing, **tetra** stores only up to 31 characters; if name is longer, a condition of error may exist. So be careful, with long names.

word set, devices

Words which allow access to mass storage and computer peripheral devices.

word set, interpreter

Words which support interpretation of text input from a terminal or mass storage by execution of corresponding dictionary entries, vocabularies, and terminal output.

word set, nucleus

The FORTH words generally defined in machine code that create the stacks and fundamental stack operators (virtual FORTH machine).

word set, reference

This set of words is provided as a reference document only, as a set of formerly standardized words and candidate words for standardization.

word set, required

The minimum words needed to compile and execute all Standard Programs.

word, standard

A named FORTH procedure definition, formally reviewed and accepted by the Standards Team. A serial number identifier {100..999} indicates a Standard Word. A functional alteration of a Standard Word will require assignment of a new serial number identifier.

The serial number identifier has no required use, other than to correlate the definition name with its unique Standard definition.

In this manual the number identifier is not reported, because this is not a Standard File, nor accepted by the Standard Team.

1.4. Fulfilled requirements

tetra fulfills the requirements of the Forth-79 Standard.

The Forth-79 Standard recites: *"Each Standard System and Standard Program shall be accompanied by a statement of the minimum (byte) requirements for:*

- 1. System dictionary space*
- 2. Application dictionary space*
- 3. Data stack*
- 4. Return stack*
- 5. Mass storage contiguous block quantity required*
- 6. An operator's terminal.*

Each Standard System shall be provided with a statement of the system action upon each of the error conditions as identified in this Standard."

This is the data related to this fulfillment:

1. A 54 KiB system dictionary space (included in the dictionary)

2. More than 50000 bytes of memory for the applications dictionary
3. A data stack of 2048 bytes (that can be expanded)
4. A return stack of 2048 bytes (that can be expanded)
5. A mass storage capacity (65535 blocks for each directory)
6. A UNIX/Linux operator's terminal (direct or emulated).

For what about the fifth point, the mass storage is not necessarily contiguous, and its maintenance is accomplished by the Operating System into a user-chosen directory, where blocks are textual files numbered 1 to 65535. There may be any number of such a directory, that the user may create anew. This is a minor deviation (an enhancement, actually) from the Standard requirements.

2. INSIDE THE COMPILER

2.1. The memory model

tetra runs in a single, contiguous, flat 32-bit address space. Each 32-bit address hosts the single byte of one Forth memory position; special code characters (which are 32-bit numbers) are hosted in the same 32-bit address, but this is transparent to the user.

tetra's memory is organized, as the Standard of Forth-79 requires, in bytes (i.e. an 8-bit unit); a byte or a character is stored in one byte; a (single) number is stored in two bytes (i.e. a 16-bit number, named 'cell' in Forth-79 jargon); and, as expected, a double number is organized in four bytes (i.e. a 32-bit number, thus in two cells). Cells are the basic number format, the one you see in the stack; the single 8-bit units can be accessed from memory with the Standard words `C@` and `C!` (and `C,` which I added for your ease).

As said earlier, one byte, an 8-bit unit, is hosted in a 32-bit address, using only the lower 8 bits part; the system uses the full 32-bits for control codes, markers, internal addresses and for all that is necessary for compilation; these extra bits are useless for the normal user and normally inaccessible (unless the elective word `32@` is used, but this is not a Standard word).

The memory model is organized in sections, as in fig. #1, where higher addresses are on top.

ADDRESS	TOP SECTORS	DEFAULT WIDTH
FFFF	-----	
	Return stack	2048 bytes (1024 cells)
R0 F800	-----	
	Data stack	2048 bytes (1024 cells)
S0 F000	-----	
	Buffer #3	1024 bytes
EC00	-----	
	Buffer #2	1024 bytes
E800	-----	
	Buffer #1	1024 bytes
FIRST E400	-----	
	Scratch area	512 bytes
E200	-----	
	WORD buffer	1024 bytes
W0 DE00	-----	
	Input buffer	1024 bytes
TIB DA00	-----	
	Dictionary	55807 bytes (~54 KiB)
0001	-----	
	Start of memory	1 byte (zero)
0000	-----	
	BOTTOM	

Figure #1

The labels on the left identify the constants in the dictionary to use to address specific parts of **tetra**'s memory. This memory, which is set to 65536 bytes ranging from zero to 65535, cannot be expanded (because the unsigned single number has range {0..65535}). Let's review the various memory sectors:

- The byte at position 0000 (1 byte) is reserved. It marks the start of the dictionary and cannot be different from zero. It's not allowed to be rewritten, but if you manage to do it, the dictionary is messed up! So, avoid.

- The dictionary has a total capacity of circa 54 KiB (55807 bytes from 1 to 55807). All compiled words, variables, constants and the ALLOTEd space is stored in the dictionary. The built-in words consist of about 3 KiB, so the user area is about 51 KiB. If 51 KiB seem poor for your programs, remember that each definition is stored in a few bytes.
- The input buffer (a.k.a. Terminal Input Buffer) has a total capacity of 1024 bytes (from 55808 to 56831) and hosts the current input line; the system uses this buffer to evaluate the input from keyboard, file or block.
- The WORD buffer is a dedicated space storage of 1024 bytes (from 56832 to 57855) that WORD uses to store its data; this buffer is split into four segments of 256 bytes each, available in turn, and the current segment address is returned and can be used and copied anywhere. I set the WORD buffer apart from the program memory, to avoid overwriting of any kind. The system constant W0 identifies the first byte of the first segment of the WORD buffer.
- The scratch area is used by the pictured output strings created between <# and #> and is also a dedicated buffer. It's 512 bytes wide (from 57856 to 58367), sufficient for any output format.
- Numbered buffers #1, #2 and #3 refer to the screen buffers where BLOCK loads the correspondent storage blocks in chunks of 1024 bytes (buffer 1 from 58368 to 59391, buffer 2 from 59392 to 60415, buffer 3 from 60416 to 61439). The system constant FIRST identifies the first byte of screen buffer #1.
- Finally, in the higher part of the memory system, stacks are found: the Data Stack (from 61440 to 63487) and the Return Stack (from 63488 to 65536), accessible by the tetra programmer using the usual stack words, both 2048 bytes wide (corresponding to 1024 cells); the system constants S0 (es-zero) and R0 (ar-zero) identify the first byte of the Data Stack and of the Return Stack respectively; there is a third independent Control Flow Stack, 256K 32-bits numbers wide only for system usage, destined to internal passages (mainly control flow and decisional steps taken during the compilation) and normally inaccessible;¹³ this stack is not memory-based and exists only during one session execution. Stacks are organized bottom-up.

You can use the two buffers of WORD and the Scratch Area as a private contiguous 1536 bytes buffer, for your own scopes (provided you don't use WORD and the pictured formats). To retrieve the proper addresses, the first useful character address starts from W0 and the last is FIRST-1 (the same values are shown by 2 INFO). Of course, if by means of the console options `--stack-size=<n>` and `--return-stack-size=<n>` you give different width to the stacks, the pointers of the memory will be moved to accommodate the desired width, by reducing the dictionary space, and this is reflected by the update of the respective values.

Each word (user or built-in) is saved by tetra to memory, either by the colon/semicolon mechanism or by any defining word as CREATE or VARIABLE, in the quasi-fixed scheme of fig. #2, where cells have been classified according to the fig-Forth jargon:

¹³ You can access the Control Flow Stack using CF-GET, which removes the top of the Control Flow Stack and pushes it on to the Data Stack and CF-PUT, which pushes the top of the Data Stack on the top of the Control Flow Stack; they may simulate the compilation process of words that require passing data thorough the Control Flow Stack; you can also print the content of the Control Flow Stack with the service word CF@. See the chapter "Service words."

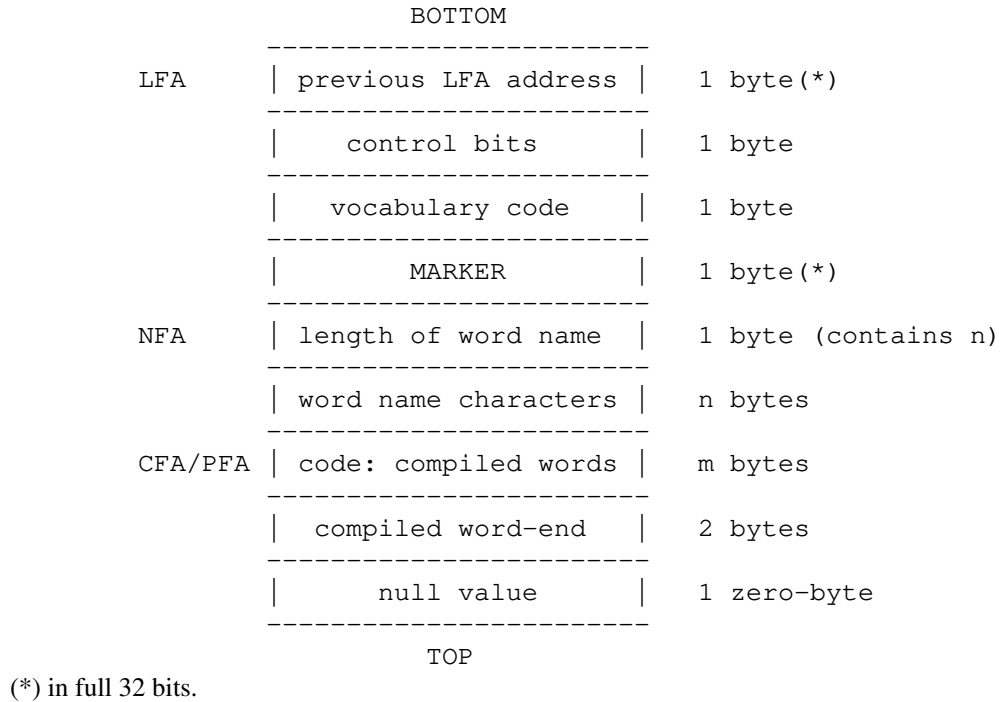


Figure #2

The *Link Field Address* is the address where the word begins, that contains the link to the previous word for backward search; it is also the word address stored in memory when that word is compiled; it's noteworthy underlining here that this address is stored in full 32-bits, for speed reasons. Control bits and the vocabulary code establish the word directives. The **MARKER**, a 32-bit number, serves as a known point for traversing words upward and downward.

The *Name Field Address* is the address where the name characters begin. Its first address contains the word count. There is a limit of 31 characters for the length of the names you can use.¹⁴

The *Code Field Address* is the start of the compiled code in words, and thus I suppose it matches the definition of 'Compilation Address' reported in the Standard; **FIND** returns the Code Field Address, and **EXECUTE** executes the Code Field Address.

The *Parameter Field Address* is the start of the parameters (i.e. values) that some items do contain. It matches quite always the *Code Field Address*, but not for all items. Read the paragraph "*A word about FIND and 'tick'*" for more. Read also the Appendix for information about **tetra** internals.

The compiled word-end appears only for colon words, vocabularies and **CREATED** words, and not for variables and constants. The compilation is performed as follows:

- **Builtin words** are usually compiled in a two bytes scheme, where in the first byte there is the builtin control code, and in the second byte (in full 32 bits) there is the builtin word code. The control flow builtin words may also add one byte (in full 32 bits) to store the branch address.
- **User words are** usually compiled in a two bytes scheme, where in the first byte there is the user word control code, and in the second byte (in full 32 bits) the Link Address of the user word.

¹⁴ This limit is established to 31 by default because it is required by the Standard. You can, anyway, change this value in `custom.h`, setting a different value. This value should not be lower than 31, because this could cause a system crash at the start or while loading items from the library. See also **WIDTH**.

- **Strings** are usually stored in memory as counted (packed) strings, i.e. the first byte holds the length L of the string (up to 255 characters for each string) and in the next bytes there are the L characters that make up the string, one byte for each. If a string terminates before its counted limit (for instance it may be typed with `EXPECT`), it is closed with the NULL character. Any typing word prints thus the string until its natural end (the characters count) or until a NULL is encountered¹⁵.
- **Vocabularies** are created in the dictionary with a link to the `CONTEXT` dictionary, so that there is virtually no limit in the vocabularies number, provided they are contained in the dictionary space.

2.2. The runtime engine

Let's review some aspects of the runtime engine.

2.2.1. The execution model

As reported also in the man page, the interpreter, before starting the Forth session (the idle cycle), executes the following processes, in order:

- 1 Populate the dictionary with built-ins and sets up system variables.
- 2 Process the `--init` option. (if given)
- 3 Parse the resource file `$HOME/.tetra.rc`. (if any)
- 4 Parse the files in the argument list. (if any)
- 5 Process the `--command` option. (if given)

The resource file is searched in `$HOME`, and its name is `.tetra.rc`¹⁶. This file (which is a Forth source, after all) is not created by `tetra`. If you want to permanently activate some options, compile words, create variables, populate the stack, print messages and banners or whatever, create the resource file by yourself and, from now on, the program/data in the resource file will be executed and be a permanent part of your system. If you remove the resource file, `tetra` won't complain and will start a normal default session.

The files in the argument list are parsed in the same order as they appear; if an error is found in one of them, the execution is stopped and control is returned to the shell, with the following data report:

```
*****
error on word <text>
error code: <n>
error type: <text>
total files in input = <n>
error was found on file <text> [#<n>]
error is located in line <n>
*****
```

The control is then returned to the shell, for the error correction, presumably reviewing the source code with an editor. If you want the Forth session continuing after the files loading, even in case of errors, use option `-Q/--no-quiet`; in this case no data report is printed, but you can inspect the system and understand what's gone wrong. If no file has a closing `BYE` and they all performs regularly, a Forth session is started. If any intermediate file has a closing `BYE`, this is ignored, to let the system process even further files. If the last file has a closing `BYE`, the control is passed to the shell at its completion. You can use option `-q/--quit` to force returning control to the shell and avoid the starting of a session. You can use option `-Q/--no-quit` to force starting a Forth session even in case a return to shell is commanded.

These options are detailed in the man page.

¹⁵ This is a minor deviation (an enhancement, actually) from the Standard.

¹⁶ Under the Windows Prompt, the file is renamed `_tetra.rc` and is not hidden. So Windows users may take care of this because they don't get the same effects running under Cygwin or under the Windows Prompt.

2.2.2. The Program Counter

When an instruction is under execution, its address is stored in the Program Counter, which is a pointer which moves through the dictionary to execute program instructions, even in case of nested words. This means that the 'next instruction code' is not pushed to the Return Stack (so common for fig-Forth and R-Forth users), but it's held in a system variable, called PC¹⁷ (Program Counter), that can be altered to modify the control flow; this process avoids dangerous usages of the Return Stack (apart from `DO . . LOOP` and `LEAVE`, which use the Return Stack for saving the intermediate limit and index, available for the word `I` to retrieve the current index value).

If you plan to use PC, remember though that the engine of **tetra** updates PC after each instruction by adding two (one for the instruction id, one for the instruction code); besides, you must pay attention while altering PC, because some words (e.g. `IF`, `ELSE`) have three cells, and others (e.g. `.`) use a variable-length cells number.

During the debugging, PC is shown at each step.

2.2.3. The 'readline' facility

tetra is built around the 'GNU readline' library facility, that lets you type a few letters of the dictionary word you want to invoke and, by hitting the TAB key, either readline completes the word (if there is one only choice) or it prints a set of possible alternatives. Typing one or two more characters, in general, shortens the possibilities of finding rapidly what you're looking for.

This makes typing faster, and long words are not a problem anymore.

One feature that I appreciate is the completion facility, which eases in typing words or file names. The facility by default starts with dictionary-word completion; the second completion is started by putting a "here-file" character (in UNIX jargon, it means using `./`): this instructs **tetra** to step to the file-completion rather than to the dictionary-words completion, which is restored at the ENTER key¹⁸. This is activated only from the second word on (that is only if at least one space is found in the input line); this assures that any first word is always searched in the dictionary. This is extremely useful with `REQUIRE`, `ENSURE` or `INCLUDE`, just to tell the more common examples.

GNU readline was sized for **tetra** to store only the last 1000 commands in the history file. See for example the `HISTORY` command, a tool for the programmer directly derived from the readline utilities. Note also that the closing command `BYE` is not recorded.

GNU readline is common in all UNIX shells, because of its versatility; it is common in Gforth too. Thanks to the guys at the Free Software Foundation project. You can find more readline news at:
<https://tiswww.case.edu/php/chet/readline/rltop.html>

2.2.4. Program interruption

Sometimes, you might find that your program hangs, due to one of the following cases:

- a) because of a compilation error from a file
- b) because of some calculation that finds wrong data on the stack
- c) because you wrote accidentally an infinite cycle
- d) because the system has become unable to accept more characters
- e) because you accidentally overwrote some dictionary part

then do one of the following actions:

¹⁷ In fig-Forth it was known as `IP`, that is the *Interpretive Pointer*. In other versions of Forth it was known under the name `DP` (*Dictionary Pointer*). I renamed it *Program Counter* because it is activated when a program (a word, actually) is invoked. Moreover, it's true the pointer traverse the dictionary (`DP`) in interpretive mode (`IP`), but I had to stand out, right?

¹⁸ Super-immediate words are not detected by readline, so that the word completion is not enabled for them.

- Press CTRL-C and then press ENTER a few times until you see ok.
- type BYE to end your session, or
- type WARM to reload the built-in dictionary (losing all changes so far), or
- type COLD to reset the forth session.
- If the error was in a compilation state, and this state is still active, type ; or [to re-enter the interpretation state and proceed.
- If you should find yourself in the 'comment' mode initiated by the open parenthesis (, type a closing parenthesis) to re-enter the interpretation state, and proceed.
- If anything fails but the system is still responsive, try with CHECK-DICT, which scans the dictionary memory from the bottom and tries to remove the words that make the system unresponsive. If even this should fail, a COLD or BYE must be invoked.

If all previous tips don't work for you, write to me. The problem may be in **tetra**'s internals. Follow the instructions in the man page.

2.2.5. Crash course

The following commands are known to crash a Gforth engine:

```
0 0 !
here execute
' catch >body 20 erase abort
' (quit) >body 20 erase
```

The last two examples cannot be used in **tetra**, because the words CATCH and (QUIT) don't belong to the current dictionary. But what about the first two cases?

I'm proud to show that **tetra** has controls that prevent the crash of the system, if the two commands are typed:

```
0 0 !
? illegal address.
0 0 !
^
```

```
HERE EXECUTE ok
```

There are other ways, though, that may crash the compiler. At present, I think I have solved all the cases that came to my attention, but I fear some other (maybe many!) remain.

I ask you to report all the crashes that happened with your code in using **tetra**, sending me also the excerpt of the culprit code. I thank you in advance.

2.3. Strict behaviour

In designing **tetra**, I deliberately decided to modify the behaviour of some commands, in order to resolve some inelegant nuisances of the Forth-79 Standard, but with the resolute intention to maintain its real essence. In particular, I decided to:

- design WORD to terminate the reading from the Terminal Input Buffer when the terminator is met, independently from its position, so that it's never ignored. This feature (also explained in the paragraph "Differences between tetra and the Forth-79 Standard") lets WORD treat also null strings, differently from what performed by the original Forth-79 behaviour. This feature lets the programmer build the ANSI PARSE command, and in general has a more modern and logic feeling. Moreover, this version does not complain if the delimiter is zero, and in this case it returns an empty string. These are not important differences, detectable only in case of null

strings followed by some text (that the original WORD would interpret as part of the parsed string). See also the dictionary definition of WORD, with all its features exposed.

- design ." (dot-quotes) to not complain if the Terminal Input Buffer is exhausted before finding the closing double quotes terminator, and terminate the string automatically.
- design FORGET to not complain if the designated word is not found, to enable the loading of files under debugging more than once, provided the start of the file is, for instance:
FORGET DISCARD
: DISCARD ;
This piece of code would work even the first time, when the DISCARD word has not been declared yet.
- design the (. .) comment to extend over multiple lines or blocks. Moreover, if the closing parenthesis is not followed by a blank, design to not consider it a closing parenthesis.

The aforementioned features depict a non-properly-standard Forth-79, because the designed behaviours change a bit what is reported in the Standard document, but ultimately they ease one programmer's life. In case a more adherent behaviour to the Forth-79 Standard is needed, option `-s/--strict` may be used, or variable `STRICT-STATE` may be set to a value different from zero. This changes the tetra's behaviour with respect to the following:

- WORD now ignores the terminator that follows the blank, so that it will continue to read characters from the Terminal Input Buffer until the end or until another terminator is found. As long as you don't treat empty strings, you won't see any difference, but beware: this WORD is not completely assimilable to the ANSI code, so you cannot use it in this sense. Moreover, if the delimiter is null, WORD will complain.
- The word ." (dot-quotes) now complains if the terminator is not found, before the Terminal Input Buffer is exhausted.
- The word FORGET now complains if the designated word is not found. This prevents the usage of the technique exposed previously but, in a certain sense, makes the words deletion a little more secure.
- The comments (. .) cannot be extended beyond the end of line (and for blocks, beyond the end of block) and the first closing parenthesis closes the comment, regardless of what follows.

This strict behaviour does not interfere in general with the fig-Forth state behaviour (regulated by `FIG-STATE`), because they refer to different unrelated features, so you can enable the strict behaviour in fig-Forth mode without any hassle.

2.4. System constants

tetra's dictionary contains some system constants that help the programmer evaluating the environment in which his programs are run. They are 16-bit constants (the predefined value, if already known, is reported in square brackets):

#ERR, the number of the default non-error condition {-99}
B/BUF, the dimension of a screen buffer in bytes [1024].
B/SCR, the number of screens per block [1].
BL, the ASCII code for the blank space [32].
C/L, the number of characters per line [64] (numbered 0..63).
ERNUM, the number of default error messages in the current version [70].
FALSE, the false value 0
FIRST, the address of the first screen buffer character. (*)
L/SCR, the number of lines per screen [16] (numbered 0..15).
LIMIT, the address just above the highest screen buffer character (*)
MEMLIMIT, the value 65535 (the last memory position)
NOOP, the code for the compiled no-op function. (*)
R/O, the Read file access method code [1].
R0, the start address of the Return Stack. (*)
S0, the start address of the Data Stack. (*)
STDERR, the standard error channel code (use with `WRITE-LINE`)

STDIN, the standard input channel code (use with READ-LINE)
STDOUT, the standard output channel code (use with WRITE-LINE)
TIB, the start address of the Terminal Input Buffer. (*)
TRUE, the true value 1
W/O, the Write file access method code [2].
W0, the start address of the WORD buffer. (*)

(*) Value calculated at start.

2.5. System variables

tetra's dictionary has some system variables that help to keep track of the dictionary status or to govern some processes. They are all 16-bit variables (the proper usable range is written in curly brackets):

#BUFFERS, the current number of active screen buffers {0..3}
#CLOCK, the milliseconds/microseconds index for CLOCK {0, non-zero}
#L, the current editing line in the current screen {0..15}
#P, the current editing position in line in the current screen#L {0..63}
#TIB, the current character count in TIB {1..1024}¹⁹
#VOCS, the current number of vocabularies in the system {2..31}
>IN, the current position in TIB {0..1023}
BASE, the current number base (default 10) {2..36}
BFR, the current screen number being EDITed or LOAded {1..3}
BLK, the current block number being loaded {0..65535}
CONTEXT, the current search vocabulary reference
CSP, a user variable containing a stack index {0..65535}
CURRENT, the current dictionary where new definitions are created
DEBUG, the debug state flag {0, non-zero}
DPL, the number of digits typed after a dot in a double number {0..255}
ERR, the last error code set by ERROR {-32768..32767}
ERPOS, the position of the error in the input line {0..1023}
FENCE, the address under which FORGET cannot actually forget {0..65535}
FIG-STATE, the fig-Forth compatibility flag {0, non-zero}²⁰
FLD, the number of characters in the last pictured output {0..255}
HLD, the address of last characters in the last pictured output {0..65535}
IP, an array of the four Internet Protocol values (each in {0..255} range)
LAST, the address of the last initiated dictionary entry {0..65535}
OFFSET, the offset from the address returned by BLOCK {0..1023}
OUT, the current printing position counter {0..255}
PC, the current program counter {0..65535}
R#, the current cursor offset in the entire block screen {0..1023}²¹
SCR, the current editing block number {0..65535}
SPAN, the characters count after last EXPECT {0..255}
STATE, the current state {0 or 1}
STRICT-STATE, the current strict-behaviour flag {0, non zero}²²
VOC-LINK, the last declared vocabulary link {0..65535}

¹⁹ The two variables >IN and #TIB refer to the same Input Buffer, with this important difference: >IN has conventionally range {0..1023}, and it is the pointer to the Input Buffer used by the text interpreter, incremented at each word invocation (starting from zero); #TIB instead is updated by the system only when the Input Buffer is filled and reports the total number of characters in the Input Buffer. After the commands in the Input Buffer have been completely evaluated, the two variables point at the same character, but #TIB has **always** pointed to it, while >IN has progressively reached that point; besides, >IN is one unit less than #TIB, because >IN is a pointer, while #TIB is a quantity.

²⁰ The FIG-STATE compatibility flag holds the forth state, zero for Forth-79, non-zero for enabling the fig-Forth features (described further in the manual).

²¹ This variable holds actually the result of: #L * 64 + #P.

²² The STRICT-STATE compatibility flag holds the strict behaviour state, zero for the default advanced non-strict Forth-79 behaviour, non-zero for enabling the strict behaviour features (described elsewhere in the manual).

WARNING, the current mode of the ERROR management {1,0,-1}
 WIDTH, the maximum length for the definition of words {1..255}

These variables have the limits reported in parentheses. Variables that have a range of 8 bit ({-127..128} or {0..255}) use only the low part of the two bytes; the other variables use the both of the bytes, that is the whole cell. When variables have a range {0..65535} they are generally addresses to be considered as unsigned.²³

2.6. Environmental queries

tetra is a UNIX program, and as such it must support all information about the underlying UNIX session (the shell data) and as a Forth system it must access the inner status for the most fussy programmer. So I added this ANSI word that expects the address and count of a string on Top of Stack; this string must match (case does not matter) an available Forth system variable name, one among those listed in the following table, returning on the stack the correspondent value (see types ahead). These data are contained in memory locations not normally accessible by the programmer, and they are generally variables and constants for the **tetra** execution. They are not variables in the Forth sense.

In the following tables, the type is *n* for single numbers, *flag* for a 1/0 result, *d* for double numbers and *addr u* for unpacked strings, *u* identifies the unsigned version of that number. The *Const?* column identifies data that won't change from query to query if *yes* is printed, otherwise they're subject to change.

The **tetra** environmental queries are:

String Name	Type	Const?	Notes
/COUNTED-STRING	n	yes	maximum size of a counted string, in characters
/HOLD	n	yes	pictured numeric output buffer size, in characters
/HOLD-START	n	yes	start of the pictured numeric string buffer
/PAD	n	yes	scratch area size pointed to by PAD, in characters
ADDRESS-UNITS	n	yes	address unit size, in bits
BLOCKS-DIR	addr u	no	blocks directory
COMPILE-ONLY-WORDS	n	no	number of compile-only words in the current dictionary
CURR-DIR	addr u	no	current working directory
DEFINING-WORDS	n	no	number of defining words in the current dictionary
DICTIONARY-START	n	no	start of available memory
EXECUTE-ONLY-WORDS	n	no	number of execute-only words in the current dictionary
FLOORED	flag	yes	returns zero (tetra is not floored-division based)
FREE-MEMORY	n	no	current dictionary memory availability
IMMEDIATE-WORDS	n	no	number of immediate words in current dictionary
LIB-FILE	addr u	no	library file path
MAX-CHAR	u	yes	maximum value of a character in the character set
MAX-D	d	yes	largest usable signed double number
MAX-D-NEG	d	yes	largest usable signed negative double number
MAX-N	n	yes	largest usable signed integer
MAX-N-NEG	n	yes	largest usable signed negative integer
MAX-U	u	yes	largest usable unsigned integer
MAX-UD	ud	yes	largest usable unsigned double number
RES-FILE	addr u	yes	resource file name
RETURN-STACK-CELLS	n	yes	maximum size of the return stack, in cells
SCREEN-BUFFER-1-START	n	yes	address of screen buffer #1
SCREEN-BUFFER-2-START	n	yes	address of screen buffer #2

²³ It's curious noting that the Standard of Forth-79 got rid of most variables that the fig-Forth openly created and used; I found this somehow weird: what harm can a variable ever do? Probably the 'STATE variable' affair may have influenced the decision. (A lot of critics wrote about STATE and the damage it can do.) I feel, instead, that system variables can increase the knowledge of the current state of the Forth engine; after all, if you don't want to use a variable, ignore it! But if you need it, here it is!

SCREEN-BUFFER-3-START	n	yes	address of screen buffer #3
STACK-CELLS	n	yes	maximum size of the data stack, in cells
SYS-DIR	addr u	yes	system directory
TOTAL-WORDS	n	no	total words number in the current dictionary
WORD-BUFFER-DIM	n	yes	WORD buffer dimension

The formulae MAX-INT (clone of MAX-N), MIN-INT (clone on MAX-N-NEG), MAX-2INT (clone of MAX-D) and MIN-2INT (clone of MAX-N-NEG) are also available.

The shell environmental queries (quite a subset of them, which are many) are:

String Name	Type	Const?	Notes

COLUMNS	n	no	number of columns in the current terminal
CPU	addr u	yes	the CPU identifier
HOME	addr u	yes	the user home directory
HOST	addr u	yes	the host (generally the user)
HOSTNAME	addr u	yes	name of the host (generally the user)
LANG	addr u	no	the shell language id
MACHTYPE	addr u	yes	the CPU-Company-System id
LOGNAME	addr u	yes	the user log name
OLDPWD	addr u	no	the previous current directory
OSTYPE	addr u	yes	the Operating System id
PATH	addr u	no	the current path (may be trimmed)
PWD	addr u	no	the current directory
SHELL	addr u	no	the shell name
TERM	addr u	yes	the terminal id
USER	addr u	yes	the user's name

An ellipsis . . . at the end of a string means that the system string (usually belonging to the shell, for example PATH) is longer than 255 characters.

These formulae can be used with EXPECT or S" (which is in the tlibrary) and treated as any other string; for instance:

```
6000 3 OVER OVER EXPECT CPU
ENVIRONMENT? DROP TYPE x86_64
S" OSTYPE" ENVIRONMENT? DROP TYPE linux
```

Or a dedicated look-ahead word that outputs a string can be written (ENV is in the library):

```
: ENV BL WORD COUNT ENVIRONMENT? IF TYPE THEN ;
```

to be used as such (watch out, this is for strings!):

```
ENV CPU x86_64 ok
```

2.7. Error management

The Forth-79 Standard specifies that an error found during an execution process must be taken into account by the Forth interpreter with a proper message, without specifying *how* this must be accomplished. So I grabbed some ideas from the fig-Forth (an environment with a very specialized set of dedicated words), and wrote all the error messages with the purpose to have them clear and complete.

2.7.1. Default Error Management

When an error condition exists, control is passed to a system error procedure, along with an error code; the system error procedure then prints the system error string corresponding to the error code.

The system proceeds now to solve the error condition by:

- setting STATE to interpret mode
- resetting all stacks
- resetting HERE in case of an error in compilation
- finally, control is returned to the keyboard.

These are called the "default error resolution operations."

This Default Error Management treats all cases and should be sufficient for all the programmer needs.

2.7.2. User Error management

If you wish to print your error messages or you want to execute different routines to resolve the error, before the default error resolution operations, the User Error Management will help. This special management can be used through the following built-in words:

The word ERROR is used to enable the User Error Management. The argument of ERROR is a number corresponding to the index of a string in the default errors string array, or a line in a screen buffer, or a user code in an alternative user procedure. The execution of ERROR forces the creation of an error instance, and a message is printed.

The user variable WARNING rules the user error management behaviour: it contains 1, 0, -1 or -2, the first three (1, 0 and -1) governing ERROR, the last (-2) governing the whole error system behaviour:

- 0 When WARNING is zero (default state), ERROR uses the argument as an error code and calls the Default Error Management; at all effects, it's like a system error procedure performed by `tetra`. You can use the system error strings to flag errors likewise the error system does on its own.
- 1 When WARNING is 1 and a screen was at least loaded (that is, BFR is not null), ERROR retrieves the output strings from the active screen buffer, with errors codes in the range {0..15} (these strings may also be simple messages and not necessarily error messages, though passed through ERROR). BFR can be used to set/unset the current screen buffer and #BUFFERS can be used to check if at least one screen was loaded. Screen 20999 offers an example of an error messages screen.
- 1 When WARNING is set to -1, ERROR calls the word (ABORT) ; this is an existing runtime procedure that in its default state simply calls ABORT, but the user is free to implement his own (ABORT) word and this last will be called by ERROR. In the user redefinition of (ABORT) , the user may read ERR to know the last used error code and then add his messages, variables resetting, all that's needed to resolve the user error condition (for instance illegal values in a math function).
- 2 When WARNING is set to -2, the Default Error Management is ignored, and (ABORT) is called. This procedure has a more deep impact on the system, because it's not related to ERROR, but influences the behaviour of the error management, that is in total power of the programmer. So, the (ABORT) procedure should be created judiciously, adopting the following rules:
 - The first operation is to set WARNING to a number different from -2, to restore the Default Error Management, to avoid the circularity of calls in case of any errors;
 - Then, intercept the errors that must be trapped;
 - Then, set up for the remaining errors, if any;
 - The final procedure is to call ABORT, that empties the stacks and reset the execution flag.

After returning from ERROR (either case), the default error resolution operations takes place, before returning control to the keyboard, and for (ABOUT) and the four rules exposed above, the default error resolution should be

restored by the programmer resetting WARNING (reset to -1, 0 or 1, that is not -2).

The word ?ERROR calls ERROR if the flag found below the top of the stack is true (conditioned error simulation).

The word MESSAGE simply prints the error message corresponding to the error code found on the stack (depending, like ERROR, on WARNING), and no default error resolution operations are performed. MESSAGE can be used to test your error routines, without the need of causing a real error.

The variable ERR contains the last error code. This variable is set by ERROR and by the error routine of the general error resolution, and should only be read, not written, by your (ABORT) procedure.

The word ABORT (and also ABORT" in the tlibrary) can be used, to reset the stack, and perform the functions of QUIT.

You can get the system error list are by typing

```
$ tetra --errors-list
```

from the console (outside tetra's environment).

2.8. Extensions for Block-treatment

To ease the block programming, I introduced the following low-level non-Forth-79 procedures:

```
READ-BLOCK
WRITE-BLOCK
DELETE-BLOCK
```

They are low level procedures (mostly incorporated in high-level commands such as BLOCK or LOAD) that are build to add some tools to the programmer for writing commands for blocks management, with the obvious consideration that blocks are files, under Unix.

2.9. Extensions for File-treatment

UNIX® is an Operating System dedicated to files. It would be strange programming under UNIX without accessing any kind of (textual) file located anywhere in the file system, and not only in BLOCK files. Since Forth-79 is not a file-oriented language, I introduced the following low-level non-Forth-79 procedures:

```
OPEN-FILE
CLOSE-FILE
READ-LINE
WRITE-LINE
DELETE-FILE
```

and the following procedure to return a file access method:

```
R/O
W/O
```

(in all they are a subset of the ANSI 1994 File Support Words) to give the programmer a bunch of low-level tools, useful for accessing files on a UNIX Operating System.

OPEN-FILE needs a string address and a string count (which may be retrieved through EXPECT or S" in the tlibrary), and a file access method. The file name must reside on the file system, of course, and the path (if necessary) should be contained in the file name.

If you open a file in reading mode (using R/O), the file must exist. The file pointer is positioned to the beginning of the file for reading the file content from the start. Reading is done through READ-LINE, which needs a string address and a string count to store the line read (it suffices a CREATED and some ALLOTEd space), but it also needs the file-id that OPEN-FILE returns in case of a successful opening. I suggest saving this file-id (which is an integer) into a variable, avoiding some stack noise.

Opening a file in write mode (using W/O) causes the file to be erased if existing or created empty if not existing. Opening a file in append mode (using R/O + W/O) needs the file to exist. The file pointer is positioned at the end of the file so that subsequent writing adds contents to a file (append-mode). Writing, in either case, is done through WRITE-LINE which, as for READ-LINE, needs a string address, a string count and a file-id.

READ-LINE recognizes all three kinds of newlines (LF, CR, and CRLF), and CR and WRITE-LINE outputs the standard newline of the output channel (for files: LF on Unix, CRLF on Windows, CR on Mac).

CLOSE-FILE does not take into account the opening mode: it simply closes the connection with the file, saving permanently the information therein contained.

DELETE-FILE deletes the file from the file-system.

These words are sufficient to create a file, open a file, read its lines, closing or deleting a file; it's up to the programmer to separate the content of the lines and manage their content (for example creating WRITE-FILE or READ-FILE).

2.10. Using computer ports

Warning: if the port words do not return the desired values, perhaps they are under control of the firewall, which could prevent the use of some of them, or the computer is not listening the port you are requiring. So, before using ports, assure that a connection with the ports in use is possible

It would be strange if a Forth interpreter couldn't talk to external devices. It would be a blind, self-looping environment. So I enabled `tetra` to be aware (at least potentially) of computer ports. It's the programmer's responsibility to talk to the right port for the right scope, but if done right, computer ports can be very useful.

The connection is opened with a talk to 127.0.0.1 by default (that is the basis computer ports). If you need another address, see `IP` that can host the address to open. These are the characteristics of the connection:

- The connection type is `SOCK_STREAM`: it provides sequenced, reliable, bidirectional, connection-mode byte streams, and may provide a transmission mechanism for out-of-band data.
- The protocol is `AF_INET`, that is it uses the Internet Protocol 4 (IP4).
- The reading or sending proceeds by bytes (one byte for each reading).

To enable the connection to a port, the `PORT` word must be used, to access the port connection. The port number is expected on the stack, along with a true flag:

```
un true PORT
```

This command will try to open and connect port number `un`, returning 0 if a connection is possible, -1 if the socket could not be created, and -2 if the socket was created but no connection is possible; in case of -1 or -2, also the error number `errno` of the `socket ()` and `connect ()` procedures is returned. The figure:

```
un false PORT
```

will close the connection with port `un`, returning always -1. At most 64 contemporary connections can be established by `PORT` word in open mode. If you please, you can create two constants:


```
1 CONSTANT OPEN
0 CONSTANT CLOSE
```

and use the more familiar commands (e.g. on port 13):

```
13 OPEN PORT
13 CLOSE PORT
```

which in a program has a good effect.

To receive and send bytes from/to the port, the words `LISTEN` and `SEND` may be used.

`LISTEN` needs the port number on top of the stack, and returns the received byte. a value of 0 means an empty receiving or an error in reading. A positive value means correct receiving²⁴. E.g.

```
13 LISTEN
```

`SEND` also needs the port number on top of the stack, and the value to send underneath. For instance, to send the letter 'A' (65 in ASCII) to port 13:

```
65 13 SEND
```

YOu can also systematize the process, if the call to `SEND` becomes frequent (this is only an example):

```
: SEND> 13 SEND ;
65 SEND>
67 SEND>
...
92 SEND>
```

This site offers a list of all ports of a common PC:

https://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers

2.11. Making sounds with tetra

The dictionary of `tetra` includes a couple of words for producing sounds. The first is `BELL`, which was also in the Forth-79 dictionary, though only in the Reference Word Set, probably due to the different computer architectures at those times, some with sounds capabilities and some without. The second is `SOUND`, which produces a tone variable in duration and pitch. Let's review both of them in details.

2.11.1. Using `BELL`

`BELL` is available (with identical purposes and effects) as `BEEP` (guaranteeing compatibility with other Forth-79 or fig-Forth implementations).

In the `tetra.h` header file there are five definitions that are useful to choose the right method for the current architecture: the `SPEAKER1` to use the Sox software, the `SPEAKER2` to use the speaker-test software and the third to use the ASCII Bell System. There is also a `SPEAKER4` version for doing nothing under Linux, Unix or Cygwin and `SPEAKER5` for doing nothing under the Windows Prompt²⁵. The general variable `SPEAKER` is defined in `custom.h` and set to one of `SPEAKER1`, `SPEAKER2`, `SPEAKER3`, `SPEAKER4` (on Linux/Unix/Cygwin) or

²⁴ Warning: sometimes the port does not return zero but the last available code repeatedly (generally the value 10 for `\r` or 13 for `\n` from ports that don't end transmission, like port 13); this is not an error in transmission, so, in these cases, you should check not only for zero but for 10 or 13 also, or stop reading if a non-printing character is met. See for instance the word `GETPORTDATA` defined in the library.

²⁵ This is the case for producing no nasty sound or graphical effects for `BELL` or `BEEP`, under Linux, Unix or Cygwin, or even under the Windows© Prompt.

SPEAKER5 (on Windows only) before compiling (default is SPEAKER1).

2.11.2. Using SOUND

The second versatile sound word is SOUND, which is capable of producing a sinusoidal tone variable in duration and frequency. This is an experimental word that is not really portable on all Operating Systems. So use it with care.

Again, in the `tetra.h` header file there are four definitions that are useful to choose the right method for the current architecture: the SOUND1 to use the Sox software, the SOUND2 to use the speaker-test software. There is also a SOUND3 version for doing nothing under Linux, Unix or Cygwin and SOUND4 for doing nothing under the Windows Prompt²⁶. The general variable SOUND is defined in `custom.h` and set to one of SOUND1, SOUND2, SOUND3 (on Linux/Unix/Cygwin) or SOUND4 (on Windows only) before compiling. (Default is SOUND1.)

The word SOUND uses two values on the stack, the topmost is the frequency in Hertz and the second the duration in milliseconds. (1000 is one second.)

Use this feature with care, because it's still experimental. In particular, the sound at present blocks the execution of the program, and so, if you create a piece of music to be performed by your program, the program won't proceed until the entire music is over.

2.12. Pipes with tetra

Piping into `tetra` is possible through the reading with READ-LINE from STDIN, and output to a pipe can be done with TYPE, EMIT, CR and the like. Here's a detailed exposition with pipes²⁷.

2.12.1. Reading a pipe

The reading from a pipe is done using READ-LINE repeatedly from STDIN; the end of the pipe is signaled with the return value 0 as the second value on stack, which is 1 for a correct reading and 0 for an error or the end-of-pipe signalling. If you want to be more specific (for example testing the difference between the end-of-pipe and the failing reading from a pipe) you should also check for the first value on stack - the `ior` in ANSI 1994 jargon - which is 0 for a correct execution, 1 if the pipe was never opened, and 2 in case of errors in transmissions (see READ-LINE).

Here's an example of an invocation of `tetra` usable in a pipe:

```
tetra --init=": foo begin pad dup 84 stdin read-line drop while type repeat ;  
foo bye"
```

This example works like the UNIX shell tool `cat`, to copy the pipe and type it on the screen. An example of a real usage of such a piping is²⁸:

```
$ cat fibo.ft | tetra --init=": foo begin pad dup 84 stdin read-line drop  
while type repeat ; foo bye"
```

2.12.2. Writing to a pipe

As said earlier, the writing to a pipe is performed simply by using the printing words that output ASCII text to the screen, like TYPE, CR, EMIT, the dot family and the like.

An example of an output to a pipe is a reworking of the previous example, that reads from a pipe, and passes the output to the UNIX shell tool `head` to print the first 10 lines.

²⁶ This is the case for producing no nasty sound or graphical effects for SOUND, under Linux, Unix or Cygwin, or even under the Windows® Prompt.

²⁷ I derived much of this from the GForth manual.

²⁸ GForth creators: forgive me if I choose your exact name for this procedure, but you set somehow a definitive word in this subject, so I simply retrace your steps...

```
$ cat fibo.ft | tetra --init=": foo begin pad dup 84 stdin read-line drop  
while type repeat ; foo bye" | head
```

This chunk uses `READ-LINE` to read *from* a pipe and `TYPE` to output *to* a pipe, intercepted by `head`. As a UNIX program, `tetra` behaves naturally with pipes, which are a UNIX essential feature.

2.13. Super-immediate words

Some words, due to the particularly delicate role they play in the runtime architecture of `tetra`, were marked as *super-immediate*. This means that they are intercepted before the interpreter. Since they therefore don't need to be in the dictionary, they can be used even in case the dictionary is messed-up, or in case the system gets unstable due to some strange experiments (I speak by experience).

The super-immediate words cannot be compiled, not even with `COMPILE` or `[COMPILE]`. They act always, immediately, and need no stack values or buffers to operate. They are nonetheless described in the manual, because of their utility. Their usage and specifics are reported in the chapter "Service words".

3. WORDS REFERENCE

3.1. Notation in the Reference Manual

Each word is structured according to the following rules:

- The word name is in **CAPITAL** in ASCII alphabetical order. **tetra** is anyway case insensitive.
- When the *stack notation* is reported, it shows the stack situation (before -- after); if an R: is reported, it refers to the return stack (R: before -- after); if CF: are reported, it refers to the control flow stack (CF: before -- after). The top of the stacks is always to the right.
- Occasionally, when relevant, the *word pronunciation name* is written in double quotes.
- The items in the stack notation may be one of the following *types*:²⁹

c	Any 0..127 character (unsigned 7-bit integer)
b	Any 8-bit byte (signed or unsigned 8-bit integer)
n	Any signed 16-bit integer (one cell, two bytes)
d	Any signed 32-bit double integer (two cells, four bytes)
addr	Any memory reference as an unsigned 16-bit integer
index	A postfix numeric index may be applied (n1,ud2,..)
u	Prefixes any unsigned number (ub,un,ud)
f	A boolean flag (zero, non-zero)
<name>	A dictionary word name
<file>	A file name (following own OS rules- not explained here)
<text>	Text following word in the buffered input stream
cccc	Any arbitrary series of ASCII characters (string)
file-id	An unsigned integer as a file channel identifier
fam	A constant as file access method (1=Read, 2=Write)

- Some *attributes* may be applied to the word:

C	The word may only be used during compilation of a colon definition.
I	Indicates that the word is immediate and will execute during compilation, unless special action is taken.
E	Intended for execution only, unless special action is taken.
S	The word is super-immediate .
U	A user variable (range is always specified) or a user constant .

- Some *source specifier* is applied to words:

ANS	The word belongs to the ANSI Standard.
APP	The word belongs to the Apple II Forth-79.
COM	The word is a common usage word.
D79	The word belongs to the Forth-79 Double Number Word Set.
F79	The word belongs to the Forth-79 Standard.
F83	The word belongs to the Forth-83 Standard.
FIG	The word is peculiar to fig-Forth.
RWS	The word belongs to the 79-Standard Reference Word Set
TFT	The word is peculiar to tetra .

See also the **tetra** man page, which lists all the built-in words.

²⁹ In the rest of this chapter, the term 'cell' will always refer to a couple of 8-bit bytes (that is to 16 bits). Thanks to **Ian Jones** for pointing out this subject to me during the development phase.

3.2. Required Word Set

Any Forth-79 Standard system must provide, as established in the final document, some keywords in some specification core sets; it's attested in fact that *"The words of the Required Word Set are grouped to show like characteristics. No implementation requirements should be inferred from this grouping."* Here follow the core sets:

Nucleus Words

```
! * */ */MOD + +! (+loop) - /
/MOD 0< 0= 0> 1+ 1- 2+ 2- <
= > >R ?DUP @ ABS AND (begin) C!
C@ CMOVE (compile) (constant)
D+ D< DEPTH DNEGATE (do) (does>)
DROP DUP (else) EXECUTE EXIT FILL I
(if) J LEAVE (loop) MAX MIN
MOD MOVE NEGATE NOT OR OVER PICK
R> R@ (repeat) ROLL ROT
SWAP (then) U* U/MOD30 U< (until)
(variable) (vocabulary) (while) (word) XOR
```

Please note that the parenthesized entries refer to just the runtime code corresponding to a compiling word³¹.

Interpreter Words

```
# #> #S ' ( -TRAILING .
79-STANDARD <# >IN ? ABORT BASE BLK
CONTEXT CONVERT COUNT CR CURRENT
DECIMAL EMIT EXPECT FIND FORTH HERE
HOLD KEY PAD QUERY QUIT SIGN SPACE
SPACES TYPE U. WORD
```

Compiler Words

```
+LOOP , ." : ; ALLOT BEGIN
COMPILE CONSTANT CREATE DEFINITIONS DO
DOES> ELSE FORGET IF IMMEDIATE
LITERAL LOOP REPEAT STATE THEN UNTIL
VARIABLE VOCABULARY WHILE [ [COMPILE] ]
```

Device Words

```
BLOCK BUFFER EMPTY-BUFFERS LIST
LOAD SAVE-BUFFERS SCR UPDATE
```

³⁰ The Standard curiously here writes U/ and not U/MOD... a Copy&Paste from the fig-Forth manual?

³¹ With reference to the original scheme, in *tetra* LITERAL, abd DLITERAL have only a runtime part, because they are immediate, so when you compile them, you will see only the result of thier work (to force, e.g. LITERAL compilation, [COMPILE] must be used); analogously, CREATE has only a run-time part, because it always uses the terminal input buffer to look for the creating name. Besides, the colon has only a runtime part, because it should act always in interpret mode, and the semicolon has only a compile-time part, because it should act always in compile mode. Finally, COMPILE, VOCABULARY and WORD in the original Forth-79 Standard were not due to present a runtime part, while in *tetra* they are.


```

'          -- addr          I "tick"          F79

```

Used in the form:

' <name>

If executing, leave the Parameter Field Address of the next word accepted from the input stream. If compiling, compile this address as a literal; later execution will place this value on the stack. An error condition exists if `<name>` is not found after a search in the CONTEXT through FORTH vocabularies chain. Within a colon-definition `' <name>` is identical to `[ ' <name> ] LITERAL`.³² When interpreting, `' xyz EXECUTE` is equivalent to `xyz`.

(-- I "paren" F79

Used in the form:

(<text>)

Accept and ignore characters from the input stream, until one next right parenthesis is found followed by a white space or the end-of-line³³. A blank after the leading parenthesis is required. If the input stream is exhausted before the right parenthesis, the comment mode remains active until a manual right parenthesis is typed.

```
(+LOOP)          n1 --                      C                      F79
                  R: n2 n3 -- n3 n2+n1 |
                  R: n2 n3 --
```

The runtime procedure compiled by `+LOOP`: increment the loop index by `n1` and test for loop completion. See `+LOOP`.

```
(?DO)          n1 n2 --          C          ANS
R: -- n1 n2 |
R: --
```

The runtime procedure compiled by ?DO: passes control to the instruction after the correspondent LOOP or +LOOP in case n1=n2, removing them from the stack, otherwise behave like DO and move the loop control parameters to the return stack for LOOP or +LOOP to use. See ?DO.

```
(2CONSTANT)          d  --                                TFT
```

The runtime procedure compiled by 2CONSTANT for creating a double constant.

```
(2VARIABLE)          d  --                                TFT
```

The runtime procedure compiled by 2VARIABLE for creating a double variable.

³² As said earlier in this manual, the Parameter Field Address and the Code Field Address depends on the item type (words, variables, CREATED words, colon words, etc.). See also FIND.

³³ The right parenthesis directly followed by a character is part of the comment, and does not close it. This lets using (and) inside the comment, in such a way:

```
( comment (by M.E. Myself); this is a comment too )
```

with no risks. This is a minor difference with the Standard, which does not require this. This is, nonetheless, a feature that A. Winfield presents in his "*The Complete Forth*", and that probably was a feature of R-Forth.

(ABORT)

—

FIG

Executed by ERROR when WARNING is -1. This is a default runtime procedure that simply calls ABORT.³⁴ This is not a runtime procedure, but a real procedure, that has the form of a runtime procedure. The programmer may write his own (ABORT) routine, which will be the one called by the error management system in case of an existing error condition. See also ABORT.

(BEGIN)

C

F79

The runtime procedure compiled by BEGIN. It marks the BEGIN position. See BEGIN.

(CASE)

C

ANS

The runtime procedure compiled by CASE. It marks the position of the current CASE clause. See CASE.

(COMPILE)

C

F79

The runtime procedure compiled by `COMPILE`: physically compiles the word address. See `COMPILE`.

(CONSTANT)

n --

TFT

The runtime procedure compiled by `CONSTANT` for creating a single constant.

(DO)

$$n_1 \quad n_2 \quad \dots$$

C

F79

R: -- n1 n2

The runtime procedure compiled by DO: move the loop control parameters to the return stack for LOOP and +LOOP to use. See DO.

(DOES>)

C

F79

The runtime procedure compiled by DOES>. The Code execution token of this word is compiled just before the DOES> part of the defining word, signaling both the start of the following DOES> section (that will be copied to the defined word when the defining word is executed) and also the end of the executable part of the defining word. See DOES>.

(ELSE)

$$f \quad \text{---} \quad f$$

C

F79

The runtime procedure compiled by `ELSE`: test a copy of the flag `f` and if true jumps to `THEN`, otherwise continues execution. See `ELSE`.

(ENDCASE)

n --

C

ANS

The runtime procedure compiled by ENDCASE: drop the comparing value n. See ENDCASE.

(ENDOF)

CF: f --

C

ANS

The runtime procedure compiled by ENDOF: test the flag `f` in the control flow stack and if true jump to END-CASE, otherwise continue execution. See ENDOF.

³⁴ It's been built as a safe measure against forgotten implementation of (ABORT) in case the alternative error management is chosen.

(FORGET) -- F79

The runtime procedure compiled by FORGET, that performs the word deletion.

(IF) n -- C F79
CF: -- f

The runtime procedure compiled by IF: test the value n; if true, continue execution leaving a true flag on the control flow stack; if false, jump to ELSE pushing a false value on the control flow stack, or to THEN in case ELSE is not available. See IF.

(LOOP) -- C F79
R: n1 n2 -- n2 n1+1
R: n1 n2 --

The runtime procedure compiled by LOOP: increment the loop index by 1 and test for loop completion. See LOOP.

(OF) n1 n2 -- n1 C ANS
CF: -- f

The runtime procedure compiled by OF: test n2 against a copy of n1; if they match, continue execution, otherwise jump to the relative END OF. See OF.

(REPEAT) C F79

The runtime procedure compiled by REPEAT: jump back unconditionally to its proper BEGIN. See REPEAT.

(S -- I "paren" COM

This form of comment is a variant of (that skips text until the first closing and unique), treating the inner text as a comment. The difference is not operational, but it's useful for sources readers, to distinguish a stack comment from any other comment, and it may have a practical usage if you want to read files and parse the stack comments differently.

(SLITERAL) addr u -- C ANS

The runtime procedure compiled by SLITERAL; return the address and count of the string that is compiled immediately following. See SLITERAL.

(THEN) CF: f -- C F79

The runtime procedure compiled by THEN: drop the flag on the control flow stack and continue execution. See THEN.

(UNTIL) f -- C F79

The runtime procedure compiled by UNTIL: test f, and if true continue execution, otherwise jump back to its proper BEGIN. See UNTIL.

(VARIABLE) -- TFT

The runtime procedure compiled by VARIABLE for creating a single variable.

(VOCABULARY) C F79

The runtime procedure compiled by VOCABULARY: set CONTEXT to the compiled vocabulary code. See VOCABULARY.

(WHILE) f -- C F79

The runtime procedure compiled by WHILE: test f, and if true continue execution, otherwise jump forward to its proper REPEAT. See WHILE.

(WORD) c -- addr C F79

The runtime procedure compiled by WORD: set c as the delimiter character; parse the input stream until the delimiter character is found (or until the end of stream if c=1) and copy the characters starting from addr+1; set addr to the string count; return addr, the counted string. See WORD.

) F79

Revert to interpretation or compilation if the character) is followed by a space; it does not need to be separated by the text that precedes, but if the character that follows is not a blank, the comment does not end³⁵; for instance:

```
( comment that ends)
( comment (by someone); ending only after second closing )
```

* n1 n2 -- n3 "times" F79

Leave the arithmetic signed product n3 of n1 times n2³⁶.

*/ n1 n2 n3 -- n4 "times-divide" F79

Multiply n1 by n2, divide the result by n3 and leave the quotient n4. n4 is rounded toward zero. The product of n1 times n2 is maintained as an intermediate 32-bit value for greater precision than the otherwise equivalent sequence: n1 n2 * n3 /.

*/MOD n1 n2 n3 -- n4 n5 "times-divide-mod" F79

Multiply n1 by n2, divide the result by n3 and leave the remainder n4 and quotient n5. A 32-bit intermediate product is maintained as for */MOD for greater precision than the otherwise equivalent sequence: n1 n2 * n3 /MOD. The remainder has the same sign as n1 * n2.³⁷

+ n1 n2 -- n3 "plus" F79

Leave the arithmetic sum n3 of n1 plus n2. Sum is always modulo 65536 using signed values.

+! n addr -- "plus-store" F79

Add n to the 16-bit value at the address addr, by the convention given for +.

³⁵ I notice that GForth doesn't have this property yet (which maybe isn't Standard, but I believe it's useful.).

³⁶ A curious typo in the Standard replicates this entry (last at page 16) as first at page 17, totalling two consecutive same definitions.

³⁷ The standard reports here: "*The remainder has the same sign as n1*". It's clearly a typo, since the first operation is a multiplication, and if n2 is negative, it changes the sign of n1 ($-n1 * -n2 = n1 * n2$, $n1 * -n2 = -n1 * n2$), so that in these cases, the remainder must have the same sign as $n1 * n2$ rather than as n1 alone, and this is in accordance with what is written for /MOD. Thanks to **Ian Jones**, who pointed out this problem.

```
+LOOP          n --          I,C "plus-loop"      F79
```

Add the signed increment n to the loop index using the convention for $+$, and compare the total to the limit. Return execution to the corresponding `DO` until the new index is equal to or greater than the limit ($n > 0$), or until the new index is less than the limit ($n < 0$). In case the increment n is zero, the cycle executes once only. Upon the exiting from the loop, discard the loop control parameters, continuing execution ahead. Index and limit are signed integers in the range $\{-32768..32767\}$.³⁸

/	n --	"comma"	F79
---	------	---------	-----

Store n into the next available dictionary memory cell, advancing the dictionary pointer.

-	n1 n2 -- n3	"minus"	F79
---	-------------	---------	-----

Subtract n2 from n1 and leave the difference n3. Difference is always modulo 65536 using signed values.

```
-->                                --                                I "next-block"      RWS
```

Continue interpretation on the next sequential block (e.g. if `-->` is found on block file 100, block file 101 is loaded). No error condition exists if no more block files are present in **tetra**'s directory, or if the next block file does not exist: the compilation will simply stop: this is a difference with the Forth-79 Standard. It must NOT be used within a colon definition that crosses a block boundary: this is another difference with the Forth-79 Standard.³⁹

```
-TRAILING      addr n1 -- addr n2  "dash-trailing"      F79
```

Adjust the character count `n1` of a text string beginning at `addr`, to exclude from the output the final trailing blanks. If `n1` is zero or the entire string consists of spaces, `n2` is zero. An error condition exists if `n1` is negative.

. n -- "dot" F79

Display `n` converted according to `BASE` in a free field format with one trailing blank. Display a negative sign when `n` is negative.

³⁸ The Standard reports here for +LOOP: "*It is a historical precedent that the limit for $n < 0$ is irregular. Further consideration of the characteristic is likely.*"). Well, the article by Robert L. Smith titled "*FORTH Standard Corner - DO, LOOP, and +LOOP*", on *Forth Dimensions* vol. III N. 6, presents a number of problems in the LOOP/+LOOP words as reported by the Standard, mainly the missed zero increment case in +LOOP and the use of memory addressing with loop indices, which in the Standard are explicitly signed. See also the chapter "DO, LOOP and +LOOP problems".

³⁹ Far from being a programming misconception, this last difference is instead a moral rule that Leo Brodie exposes in "Thinking Forth" (ch. 5, page 143): "...Some people consider the arrow to be useful for letting definitions cross screen boundaries. In fact --> is the only way to compile a high-level (colon) definition that occupies more than one screen, because --> is "immediate". But it's NEVER good style to let a colon definition cross screen boundaries. (They should never be that long!)".

1+	<code>n -- n+1</code>	<code>"one-plus"</code>	F79
-----------	-----------------------	-------------------------	-----

Increment n by one, according to the operation of +.

1-	<code>n -- n-1</code>	<code>"one-minus"</code>	F79
-----------	-----------------------	--------------------------	-----

Decrement n by one, according to the operation of -.

2!	<code>d addr --</code>	<code>"two-store"</code>	D79
-----------	------------------------	--------------------------	-----

Store d in two consecutive cells beginning at addr.

2+	<code>n -- n+2</code>	<code>"two-plus"</code>	F79
-----------	-----------------------	-------------------------	-----

Increment n by two, according to the operation of +.

2-	<code>n -- n-2</code>	<code>"two-minus"</code>	F79
-----------	-----------------------	--------------------------	-----

Decrement n by two, according to the operation of - .

2@	<code>addr -- d</code>	<code>"two-fetch"</code>	D79
-----------	------------------------	--------------------------	-----

Leave on the stack the contents of the two consecutive cells beginning at addr, as a double integer.

2CONSTANT	<code>d --</code>	<code>"two-constant"</code>	D79
------------------	-------------------	-----------------------------	-----

A defining word used in the form:

`d 2CONSTANT <name>`

to create a dictionary entry for <name>, leaving d in two consecutive cells of its parameter field. When <name> is later executed, d will be left on the stack.

2DROP	<code>d --</code>	<code>"two-drop"</code>	D79
--------------	-------------------	-------------------------	-----

Drop the top double number on the stack.

2DUP	<code>d -- d d</code>	<code>"two-dupe"</code>	D79
-------------	-----------------------	-------------------------	-----

Duplicate the top double number on the stack.

2OVER	<code>d1 d2 -- d1 d2 d1</code>	<code>"two-over"</code>	D79
--------------	--------------------------------	-------------------------	-----

Leave a copy of the second double number on the stack.

2ROT	<code>d1 d2 d3 -- d2 d3 d1</code>	<code>"two-rote"</code>	D79
-------------	-----------------------------------	-------------------------	-----

Rotate the third double number to the top of the stack.

2SWAP	<code>d1 d2 -- d2 d1</code>	<code>"two-swap"</code>	D79
--------------	-----------------------------	-------------------------	-----

Exchange the top two double numbers on the stack.

2VARIABLE "two-variable" D79

A defining word used in the form:

2VARIABLE <name>

to create a dictionary entry of <name> and assign two consecutive cells for storage in the parameter field, setting the variable contents to zero. When <name> is later executed, it will leave the address of the first cell of its parameter field on the stack.

32! d addr - "32-store" TFT

Store the double number on top of stack on a single memory unit (remember that one byte is seen as 8-bit but the system knows it as a 32-bit unit). Used to adjust the dictionary in relation to the special markers and link addresses present in a **tetra** compiled entry. This word is a **tetra** special word serving as a system helper.

32, d -- "32-comma" TFT

Allot the double number on top of stack in one unit in the dictionary, storing it as a 32-bit number (remember that one byte is seen as 8-bit but the system knows it as a 32-bit unit). Used to compile special markers and link addresses. This word is a **tetra** special word serving as a system helper.

32@ addr -- d "32-fetch" TFT

Leave on the stack as a double number the number contained in the unit `addr` (remember that one byte is seen as 8-bit but the system knows it as a 32-bit unit). Used to parse the dictionary entries and look for the special markers and link addresses present in a **tetra** compiled entry. This word is a **tetra** special word serving as a system helper.

79-STANDARD -- F79

Assure that the environment executing the program is a Forth-79 Standard system, otherwise an error condition exists (it's supposed to be missing in a non-compliant system). Also, reset the fig-Forth compatibility flag `FIG-STATE` to zero, assuring that a normal Forth-79 behaviour is performed from now on.

: -- E "colon" F79

A defining word executed in the form:

: <name> ... ;

Select the `CONTEXT` vocabulary to be identical to `CURRENT`. Create a dictionary entry for <name> in `CURRENT`, and set compile mode. Words thus defined are called 'colon-definitions'. The Link-Field-Addresses of subsequent words from the input stream which are not immediate words are stored in the dictionary to be executed when <name> is later executed. `IMMEDIATE` words are executed as encountered.

If a word is not found after a search from `CONTEXT` to `FORTH` vocabularies, conversion and compilation of a literal number is attempted, with regard to the current `BASE`; that failing, an error condition exists, an error message is printed and interpret mode is set, removing all effects established by current compilation.

```
;          --          I,C  "semi-colon"  F79
```

Terminate a colon definition and stop compilation. If compiling from mass storage and the input stream is exhausted before encountering `;` an error condition exists.

```

;S                                --                                E  "semi-s"                                RWS

```

Stop the interpretation of a block. In `tetra`, the function of this word has been enhanced to allow also stopping the interpretation of a file.

If used into a compilation word, acts as a word ender (compiles the word ender token and a final 0); the code after ; S is compiled and occupies its memory, but it's never executed, nor shown by SEE. It can thus be used to shorten the execution of a word under design, avoiding the unnecessary code for the current investigation.

```
<          n1 n2 -- f          "less-than"          F79
```

True if n1 is less than n2.

`-32768 32767 <` returns true.

`-32768 < 0` returns true equally, and thus the two numbers are distinguished (as required by the Standard).

```
<#          --          "less-sharp"          F79
```

Initialize pictured numeric output. The words:

```
<# # #S HOLD SIGN #>
```

are variously used to specify the conversion of a double-precision number into an ASCII character string stored in right-to-left order.

```
=          n1 n2 -- f          "equals"          F79
```

True if n1 is equal to n2.

```
> n1 n2 -- f "greater-than" F79
```

True if n1 is greater than n2.

```
>IN      -- addr      U  "to-in"      F79
```

Leave the address of a variable which contains the current character offset within the input buffer in the range {0..1023} bytes. See: WORD, (, ., ", FIND and #TIB.

```
>R      n --      C  "to-r"      F79
      RS:  -- n
```

Transfer n to the return stack. Every >R must be balanced by a R> in the same control structure nesting level of a colon-definition.

?	addr --	"question-mark"	F79
---	---------	-----------------	-----

Display the number at address, using the format of . (the dot).

?CFS -- f "query-c-f-s" TFT

Leave the true flag in case the Control Flow Stack is empty, and zero otherwise. Used in conjunction with ?ERROR to issue a particular error code related to the CF stack: ?CFS 16 ?ERROR will set error 16 in case the Control Flow Stack is empty.

?CSP -- "query-c-s-p" FIG

Issue an error message, followed by (ABORT), if stack position differs from value saved in CSP. It can be used during the compilation phase of a program, to check if stack value in compilation changes (and shouldn't). In conjunction with WARNING and (ABORT), ERROR can issue a proper and personal error message and/or run a proper and personal user procedure.

?DO n1 n2 -- I, C ANS

Used in a colon-definition:

```
?DO ... LOOP or
?DO ... +LOOP
```

If n1=n2, pass control to the next instruction after the correspondent LOOP or +LOOP, otherwise begin a loop which will terminate based on control parameters. As for DO, the loop index begins at n2 and terminates based on the limit n1 (being n1-1 the last evaluated number). See also all the notes in DO, which apply also here.

?DUP n -- n n "query-dupe" F79
 0 -- 0

Duplicate n if it is non-zero.

?ERROR f n -- "query-error" FIG

In case flag f is true, call ERROR, that will issue the error message n; otherwise drop both f and n and continue execution. See also the chapter "Error management".

?TERMINAL -- n | 0 "question-terminal" COM

Check if a key was pressed from keyboard, and return zero if not; otherwise return the character code of the key that was pressed (in the assumption that the NULL character cannot be obtained from a common keyboard, this value is always different from zero). This word is non-blocking.

@ addr -- n "fetch" F79

Leave on the stack the number contained in the cell starting at addr.

ABORT -- F79

Clear the data, return and control flow stacks, setting interpret mode Print a proper message (only in fig-Forth mode). Return control to the terminal without printing the 'ok' string.

ABS n -- |n| "absolute" F79

Leave the absolute value of the number n.

ALLOT n -- F79

Reserve *n* bytes in the dictionary. If it follows a recently defined word (with `CREATE`, for instance), this means extending its `parameters` field by *n* bytes.⁴³

AND n1 n2 -- n3 F79

Leave the bitwise logical 'and' of n1 and n2 as n3.

```

ASCII                -- c (executing)      I "askee"          RWS
                        -- (compiling)

```

Leave (as a 16-bit number) the 7-bit ASCII character value of the next non-blank character in the input stream. If compiling, compile it as a literal, which will be later left when executed⁴⁴.

```
B/BUF      -- 1024      U "buffer-bytes"      RWS
```

A constant leaving 1024, the number of bytes per block buffer, that is the byte count read from disc by BLOCK.

```
B/SCR          -- 1          U "screen-blocks"  FIG
```

A constant leaving the number of blocks per editing screen. By convention, an editing screen is 1024 bytes organized as 16 lines of 64 characters each. Each screen can hold one entire block (no shadow screen, that is).

BANNER -- TET

Print the following banner:

Forth-79

to mark your program a **tetra** product!

In case of fig-Forth emulation, a proper string is appended (see FIG-STATE).

BASE	-- addr	U	F79
-------------	---------	---	-----

Leave the address of a variable containing the current input-output numeric conversion base in the range {2..36}.⁴⁵ If a value lesser than 2 or greater than 36 is stored into BASE, it's restricted to the proper interval without any warning. Use the library function ?BASE to inspect the base in decimal if needed.

⁴³ The n bytes reserved by ALLOT can be store at most n 8-bit numbers or characters, $n/2$ single numbers and $n/4$ double numbers.

⁴⁴ The Reserved Word Set indicates that ASCII is immediate and may be used for compilation only (it's mark as I,C). This must be a typo, because in the text there is also the description of the execution out of compilation, so the word is not to be marked as "Compilation only".

⁴⁵ The Forth-79 standard specifies that BASE should have the range {2..70} (Forth-83 even {2..72}!); in modern computing, bases greater than 36 (which is the sum of 10 ASCII digits plus 26 ASCII letters, uppercase or lowercase) have no meaning and are practically not implementable with the ASCII set 33-127 (printable character set), unless you want to distinguish, for example, "a" from "A", which is the best way to confuse things; furthermore, nowhere in the Standard is it written what the ordering of these 70/72 characters is (ASCII? EBCDIC? Other?); creating one specifically would result in a really messy and non-portable criterion for number recognition. This is another (minor, I hope) difference with the Forth-79 Standard, confirmed on the other hand also by the more recent Forth Standard, which sets BASE in the range {2..36}.

BEEP	--	COM
-------------	----	-----

Play a beep. This is a synonym of BELL (added for compatibility reasons). See BELL for all the technical stuff.

BEGIN	I, C	F79
--------------	------	-----

Used in a colon-definition in the form:

```
BEGIN ... f UNTIL or
BEGIN ... f WHILE ... REPEAT
```

Mark the start of a word sequence for repetitive execution. A BEGIN . . UNTIL loop will be repeated until flag *f* is true. A BEGIN . . WHILE . . REPEAT loop will be repeated until flag *f* is false. The words after UNTIL or REPEAT will be executed when either loop is finished. The flags of WHILE and UNTIL are always dropped after being tested.

At compile time, BEGIN compiles the runtime word (BEGIN) , and leaves its return address on the control flow stack.

BELL	--	RWS
-------------	----	-----

Play a beep⁴⁶. The default beep is a sound of 880 Hz for about 140 milliseconds. See also BEEP that performs an identical task.

BFR	--	U	COM
------------	----	---	-----

Leave the address of a variable containing the number of the current edited screen buffer. Every screen buffer/block word (BLOCK, BUFFER, LIST, EDIT) when invoked, set BFR to the next available screen buffer if the block was loaded, or to the correspondent screen buffer in case the block is already in memory. The range is {1..3}. The user can manually change BFR to edit another screen.

BL	-- 32	"b-l"	RWS
-----------	-------	-------	-----

A constant that leaves the ASCII character value for blank (decimal 32).

BLK	-- addr	U "b-l-k"	F79
------------	---------	-----------	-----

Leave the address of a variable containing the number of the mass storage block being interpreted as the input stream. If the content is zero, the input stream is taken from the terminal. Range is {0..65535} as an unsigned single number (see BLOCK).

⁴⁶ About this matter read the chapter "Making Sound with tetra".

```
BLOCK          un -- addr          F79
```

Leave the address of the first cell in screen buffer for the block `un`. If the block is not already in memory, it is transferred from mass storage into whichever screen buffer has been least recently accessed. If the block occupying that screen buffer was `UPDATED` (i.e. modified), it is rewritten onto mass storage before the new block-file `un` is read into the screen buffer. If mass memory reading is not possible (not-existing block or denied directory access), the screen buffer is filled with blanks⁴⁷. If correct mass storage writing is not possible, an error condition exists. Only data within the latest screen buffer referenced by `BLOCK` is valid by cell address, due to sharing of the screen buffers. At all effects, `BLOCK` sets `BFR`, and `un` is associated to the screen addressed by `BFR`.⁴⁸

BUFFER	un -- addr	F79
---------------	------------	-----

Obtain the next screen buffer, assigning it to block un. The block is not read from mass storage. If the previous contents of the screen buffer has been marked as UPDATED, it is written to mass storage to the correspondent block number. If correct writing to mass storage is not possible, an error condition exists. The address left is the first cell within the screen buffer for data storage. BUFFER sets BFR, and un is associated to the screen addressed by BFR.

BYE -- I COM

Immediately terminate the current **tetra** session and return control to the shell. This word has a service super-immediate version in case of unresponsive dictionary (and in this case, the string "(service)" appears when you type **BYE**; this is a sure sign of dictionary corruption, so be careful with your programs and review them in case.

```
C!          n addr --          "c-store"          F79
```

Store the least significant 8-bits of n in the byte at addr.

C,	n --	"c-comma"	RWS
----	------	-----------	-----

Store the least significant 8-bits of n in the dictionary, advancing HERE by one.

```
C/L          -- 64          "chars-per-line"    COM
```

A constant containing the number of characters per line in a buffer screen.

```
ca      addr -- ub      "c-fetch"      F79
```

Leave on the stack the unsigned content `ub` of the byte at `addr` (with higher bits zero in a 16-bit field).

CASE -- ANS

At compile-time, mark the start of a `CASE . . OF . . END OF . . ENDCASE` structure. At runtime, continue execution.

⁴⁷ This is an important difference with the Standard, though applicable only in case of not-existing blocks, and not applicable in normal processing.

⁴⁸ **Important note:** all blocks are saved in a hidden directory, to prevent accidental deletion. This directory is `$HOME/.tetra/blocks/` and is created by the option `--config` (see the man page for the Windows© configuration). In this directory, blocks are saved as pure textual files of 16 lines with 64 characters each, in total 1024 bytes (plus the line terminators) for each block. A very compact storage. You can copy, rename, move, delete, send, print and do any regular operation upon these files, since they are normal textual files. You can use any directory for saving blocks through the word `USE`.

```
CF-GET      -- n                                TFT
CFS:        n --
```

Remove n from the top of the control flow stack and put it on top of the data stack.

CF-PUT	n --	TFT
	CFS: -- n	

Remove n from the top of the data stack and put it to the top of the control flow stack.

```
CLOCK      -- ud      COM
```

Return the quantity of milliseconds or of microseconds (1 millionth second) from the start of `tetra`. The returned value of `CLOCK` is a double number in the range `{0..4294967295}` to ensure the maximum range. The difference is governed by the variable `#CLOCK`: if it contains zero, milliseconds are returned, otherwise microseconds are returned.

In the default milliseconds configuration (`#CLOCK` contains zero), this ensures a maximum runtime length of 4294967 seconds, which correspond to 71582 minutes, or 1193 hours, that is more than 49 days!

In the microseconds configuration (`#CLOCK` does not contain zero), this ensures a maximum runtime length of 4295 seconds, which correspond to about 71 minutes, or 1 hour and 11 minutes, but assures a greater precision with faster programs and very small intervals.

For an example of usage, take a look at the set of `TIMING`, `RESET-TIMER`, `TIMER` and `NTIMER` in the default library, with `CLOCK` as the base-word showing how to implement a benchmark tester. Change at will `#CLOCK` to contain 0 or 1, and see the difference (for instance the program `ybench.ft` in the `files/` directory).

[illegible]

Close the file identified by `file-id`. Return zero on success, one otherwise.

```
CMOVE      addr1 addr2 n --      "c-move"      F79
```

Move the specified quantity `n` of bytes beginning at address `addr1` to memory at `addr2`. The contents of `addr1` is moved first, proceeding toward high memory. If `n` is zero or negative, nothing is moved.

COLD i*x -- I TFT

Initialize or reinitialize the dictionary from the default configuration⁴⁹ in pristine state. All block buffers that have not yet been written back to mass storage are discarded. The memory and the stacks are completely reset. After the cold start, the base vocabulary is FORTH, which is also CONTEXT and CURRENT. The numerical base is decimal. Show the login message banner (*cold start*) and the session is restarted via ABORT. This word has a service super-immediate version in case of unresponsive dictionary.

It's safer typing `BYE` and restarting a `tetra` session than using `COLD`, but in some cases, this may reveal the only practical way to exit the session anyway.

⁴⁹ Remember that only the default built-in dictionary is reinitialized, not the words you might have loaded through `LOAD`, `ENSURE`, `REQUIRE` or `INCLUDE`, compiled with the colon mechanism or via the resource file `$HOME/.tetra.rc`.

COMPILE C F79

When a word containing `COMPILE` executes, the cell value following the compilation address of `COMPILE` is copied (compiled) into the dictionary. i.e., `COMPILE DUP` will copy the compilation address of `DUP`.

`COMPILE [ 0 , ]` will copy zero.⁵⁰

At compile time, `COMPILE` compiles the runtime word (`COMPILE`) and the compilation address of the next word in the input stream⁵¹.

COMPILE-ONLY E COM

Mark the most recently made dictionary entry as a word which will be usable only into definitions and not directly from console.⁵²

CONSTANT n -- F79

A defining word used in the form:

n `CONSTANT <name>`

to create a dictionary entry for `<name>`, leaving n in its parameter field. When `<name>` is later executed, n will be left on the stack. When compiled, `<name>` will leave its value that will be literally compiled in the code.

CONSUMES addr -- n TFT

This word returns the number of bytes that a word consumes in the dictionary, given its Parameter Field Address, that is the value returned by `'` (tick).

CONTEXT -- addr U F79

Leave the address of a variable specifying the address of the index of the vocabulary that starts the chain in which dictionary searches are to be made, during interpretation of the input stream.

CONVERT d1 addr1 -- d2 addr2 F79

Convert to the equivalent stack number the text beginning at `addr1+1` with regard to `BASE`. The new value is accumulated into the double number `d1`, being left as `d2`. `addr2` is the address of the first non-convertible character. This word accepts negative numbers, that are added to `d1` according to the usual laws of arithmetic.⁵³

⁵⁰ This is required and explicitly specified in the Standard, but I observe here that this definition is equivalent to `[ 0 , ]` (`COMPILE` omitted), because here `COMPILE` simply ignores what follows. The only practical consequence that can be derived from this is that `COMPILE` cannot compile the `[` (or at least this is what I understood, here!).

⁵¹ To be clear, builtin words are compiled with a reference to their compilation address, while user words are compiled with a reference to their link address.

⁵² See the related word `BUFFERED` in the tlibrary, and study how it is implemented. `COMPILE-ONLY` could be written in Forth in a similar way with 8 (the compilation bit) as the bit selector.

⁵³ I note that nowhere in the Standard is reported if dots in the string are accepted or not. Anyway, I enabled this feature in `tetra` because it seems logical that, if a number from keyboard contains dots and is accepted as a double number, the same must be said if the source of the number is a string in memory (it may have been typed by the user). Moreover, the Standard does not specify if the leading minus (`-`) is accepted (even if it seems logical it does). In any case, in `tetra`, a leading minus or even a leading plus (as for the common keyboard input) is accepted and works as expected (producing a negative number in case of minus and ignored in case of plus).

COUNT	addr -- addr+1 ub	F79
--------------	-------------------	-----

Leave the address `addr+1` and the character count of text beginning at `addr`. The first byte at `addr` must contain the unsigned character count `ub`, but if `ub` is zero or greater than 255, a condition of error exists.

CR	--	"C-r"	F79
----	----	-------	-----

Transmit a carriage-return and line-feed to the terminal.

CREATE -- F79

A defining word used in the form:

CREATE <name>

to create a dictionary entry for <name>, without allocating any parameter field memory. When <name> is subsequently executed, the address of the first cell of <name>'s parameter field is left on the stack.⁵⁴ See also DOES>.

CSP	-- addr	U	FIG
------------	---------	---	-----

Leave the address of a variable temporarily specifying the stack pointer position, for compilation error checking. See !CSP and ?CSP. The range of CSP is {0..65535}. This variable does not update automatically and is only an aid to the programmer to check if the stack maintains its integrity.

```
CURRENT      -- addr      U      F79
```

Leave the address of a variable specifying the address of the index of the vocabulary into which new word definitions are to be entered.

D*	d1	d2	---	d3	"d-times"	COM
----	----	----	-----	----	-----------	-----

Leave the signed product d3 of the two signed double numbers d1 and d2.

D+	d1 d2 --- d3	"d-plus"	F79
-----------	--------------	----------	-----

Leave the arithmetic signed sum d_3 of d_1 plus d_2 .

D-	d1 d2 -- d3	"d-minus"	D79
-----------	-------------	-----------	-----

Subtract d2 from d1 and leave the difference d3.

D.	d --	"d-dot"	D79
----	------	---------	-----

Print `d` converted according to `BASE` in a free field format, with one trailing blank at the end. Display the sign in case it's negative.

⁵⁴ The compile-time version of this word was known, before the Forth-79 Standard (the 1978 fig-Forth model, actually), as <BUILDS, and this explains the "strange" format of DOES>; the original construction was in fact made with the tokens

... <BUILDS ... DOES> ...

and this makes a perfect sense. Now, <BUILDS and DOES> in fig-Forth bear some differences with the Forth-79 counterparts; in particular, <BUILDS allocates one cell in the Parameter Field Address of the newly created words, while CREATE does not allocate anything. Besides, DOES> in Forth-79 is IMMEDIATE, while in fig-Forth it's not.

D.R	d n --	"d-dot-r"	D79
-----	--------	-----------	-----

Print `d` converted according to `BASE`, right aligned in a field `n` characters wide. Display the sign in case it's negative. If `n` is negative or smaller than the characters required for `d`, no leading spaces are given.

D/	d1 d2 --- d3	"d-divide"	COM
-----------	--------------	------------	-----

Divide d1 by d2 and leave the quotient d3. d3 is rounded toward zero.

```
D0=          d -- f          "d-zero-equals"      D79
```

Leave true if d is zero.

```
D<      d1 d2 -- f      "d-less"      D79
```

True if d1 is less than d2.

D=	d1 d2 -- f	"d-equal"	D79
-----------	------------	-----------	-----

True if d1 equals d2.

DABS	d -- d	"d-absolute"	D79
-------------	---------	--------------	-----

Leave the absolute value of a double number d.

DATE	--	n1	n2	n3	COM
------	----	----	----	----	-----

Leave on stack current year n1, current month n2 and current day n3 on top. The year is in four digits (e.g. 2019), n2 is in the range {1..12}, n3 in the range {1..31}.

```
DEBUG                -- addr                U                TFT
```

Leave the address of a variable containing the current debug state, where 0 means debug off, and non-zero means debug on. This value may read or changed, and the change will influence the debug. The most common usage is `DEBUG ON` to activate, and `DEBUG OFF` to disable.

Please, note that if TRACE is active, the activation of DEBUG will be without effect because TRACE prevails over DEBUG. See also TRACE.

DECIMAL -- F79

Set the input-output numeric conversion base to 10.

DEFINITIONS -- F79

Set CURRENT to the CONTEXT vocabulary so that subsequent definitions will be created in the vocabulary previously selected as CONTEXT.

DELETE-BLOCK un1 -- f TFT

Erase the block file un1 from the current blocks directory. In case of success, leave true, otherwise leave zero. The cases of failure include the impossibility to remove the file or a non-existent file. This is a low-level procedure that is incorporated in any word that erases a block from the file system. The screen buffers are not touched, and BLK, SCR and BFR are not changed.

DELETE-FILE addr u -- 1 | err 0 ANS

Delete the file named in the character string specified by addr and long u characters. The flag f is true in case of success (file deleted) or false in case of error

DEPTH -- n F79

Leave the number of the quantity of 16-bit cells contained in the data stack, before n is added (that is the half of the number of occupied bytes).

DLITERAL d -- I, C FIG

If compiling, then compile the double stack value d as a 32-bit literal, which when later executed will leave d on the stack as a double number.⁵⁵

DMAX d1 d2 -- d3 "d-max" D79

Leave d3, the greater of the two double numbers d1 and d2.

DMIN d1 d2 -- d3 "d-min" D79

Leave d3, the lesser of the two double numbers d1 and d2.

DNEGATE d -- -d F79

Leave the two's complement of a double number d, i.e. the difference 0 less d.

DO n1 n2 -- I, C F79

Used in a colon-definition:

DO ... LOOP or
DO ... +LOOP

Begin a loop which will terminate based on control parameters. The loop index begins at n2, and terminates based on the limit n1 (being n1-1 the last evaluated number).⁵⁶ At LOOP or +LOOP, the index is modified by a positive or negative value. The range of a DO ... LOOP is determined by the terminating word.

⁵⁵ More recent versions of Forth use the syntax 2LITERAL, maybe for assonance with 2VARIABLE and 2CONSTANT. But I decided to use the fig-Forth version of the word. If you need to use 2LITERAL, please ENSURE it. It is contained in the tlibrary.

⁵⁶ The index of the DO ... LOOP cycle can be used to inspect the whole memory, both in signed and unsigned mode:
(signed mode) 32767 -32767 DO I 32767 + .. LOOP
(unsigned mode) 65535 0 DO I .. LOOP
In both cases, the memory range is {0..65534}, due to the peculiar structure of DO ... LOOP, that does not evaluate last limit value. And being 65535 the higher unsigned number, the last memory slot must be inspected out of the DO ... LOOP cycle.

DO...LOOP may be nested. There are no limits (except the stacks overflow) to the loop nesting. Index and limit are 16-bit signed integers.

At compile time, DO compiles the runtime word (DO) and leaves its address on the control flow stack.

DOES> I, C "does" F79

Define the runtime action of a word created by a high-level defining word. Used in the form:

: <name> ... CREATE ... DOES> ... ;

and then

<name> <name>

Mark the termination of the defining part of the defining word <name> and begins the defining of the runtime action for words that will later be defined by <name>. On execution of <name> the sequence of words between DOES> and ; are executed directly into the compilation address of the created word <name>.

DPL -- addr U "f-l-d" RWS

A user variable containing the number of digits to the right of the decimal on double integer input. It may also be used to hold output column location of a decimal point, in user generated formatting. The default value on a single number input is -1. The range of DPL when greater than zero, is {0..255}, interpreted as an unsigned value.⁵⁷

DROP n -- F79

Drop the top number from the stack.

DU< ud1 ud2 -- f "d-u-less" D79

True if ud1 is less than ud2. Both numbers are unsigned.

DUMP addr un -- RWS

List the contents of un bytes starting from the address at addr, with addresses and values expressed in the current BASE. Each line of values is preceded by the address of the first value and followed by a pictured image of the ASCII characters in the cells (a dot for each non-printable value). Each BASE defines its own number of bytes per row (from 1 in BASE=2 to 8 in BASE=36). Here's an example of output in hexadecimal:

⁵⁷ The Reference Word Set reports the following definition for DPL: "A variable containing the number of places after the fractional point for output conversion. If DPL contains zero, the last character output will be a decimal point. No point is output if DPL contains a negative value. DPL may be set explicitly, or by certain output words, but is unaffected by number input".

This means that for the Reference Word Set, DPL is a variable that controls the *output*, not the input. In *tetra* I set up DPL to follow the fig-Forth and R-Forth (according to some examples in the Alan Winfield book, in chapters 8.1 and 8.5), that is, DPL is updated after each double number input, for checking the user input and acting consequently. This can be considered a minor difference with the Forth-79 Standard. Some modern compilers (Swiftforth, for instance) set DPL to return the number of digits to the right of the rightmost punctuation character (that is, the last is the good one). In *tetra*, instead, it represents the number of digits to the right of the leftmost punctuation character (that is, the first dot is the good one, and also FForth also follows this rule). Of course, if there's only one dot, the two systems coincide.

Leave a copy of the top stack number.

Start an editing session with the screen editor of block un (which corresponds to a block number), setting SCR to contain un. un is unsigned. EDIT uses BLOCK to load the block in memory, in the first available screen buffer, whose value is yield by BFR. At all effects, EDIT updates SCR and BFR.

The EDIT program is a standalone application, and it has no relation, nor common operational commands, with the built-in line editor contained in the EDITOR vocabulary.

See the chapter "The screen editor in details" ahead.

The name of the editor vocabulary. When this name is executed, EDITOR is established as the CONTEXT vocabulary and all the line editor commands become available.

Used in a colon-definition in the form:

IF . . . ELSE . . . THEN

ELSE executes after the true part following IF. The word ELSE forces execution to skip till just after THEN in case the flag left by *F* is true. It has no effect on the stack. (see IF).

At compile time, ELSE compiles the run-rime word (ELSE), placing its address on the reserved byte following (IF), and leaves its own address on the control flow stack.

Transmit the character `b` to the terminal (as 8-bit bytes with higher bits set to zero), according to the ASCII table.⁵⁸

⁵⁸ The Standard specifies for EMIT the characters range in the {0..127} region of the ASCII table. I built EMIT to use the full {0..255} range, but with the advice that the ASCII table in the range {1..127} is standard, and surely portable among various system, while the range {128..255} is system-dependent. This is a minor deviation from the Standard.

EMPTY-BUFFERS -- F79

Mark all block buffers as empty, without affecting their actual contents. UPDATED blocks are not written to mass storage.

ENDCASE addr -- I, C ANS

At compile-time, resolve all branches in the case structure, such that the address beyond ENDCASE becomes the destination of all branches after each ENDOF. At runtime, if used, discard the original comparison value and continue execution.

ENDOF -- I, C ANS

At compile-time, provide the location address following ENDOF as the destination address needed by the corresponding OF, and place a new forward reference to the next OF or default structure. At runtime, jump to the next OF or default structure.

ENSURE -- E TFT

Used in the form:

ENSURE <name>

Just like REQUIRE, parse file tlibrary (looking for it in the default library file) then, if the word <name> is not present in the dictionary, read and compile it, along with all the dependencies words that are in the same library and that are not yet in the dictionary. ENSURE cannot be compiled, and is executable-only.

See the note in REQUIRE, which contents apply to ENSURE too.

ENVIRONMENT? addr u -- n*x 1 | 0 ANS

addr is the address of a character string and u is the string's character count. u may have a value in the range from zero to 31. The character string should contain a keyword identifying an environmental query to inspect the inner status of tetra. If the system does not recognize the attribute, the returned flag is false; otherwise, the flag is true and the n*x returned is of the type specified in the table for the attribute queried. The present configuration includes also system variables of the hosting shell. See the paragraph "Environmental queries".

ERNUM -- n U COM

A constant leaving the number of the default error messages hardwired in the current tetra implementation. It may be used to know the cycle values needed to print or check all error messages (see MESSAGE and the word ERRORS in the library).

ERRPOS -- n U COM

Leave the address of a variable containing the position, in the input line, of the last character of the word that caused the last occurred error. This value contains zero at start, and is never reset.

ERR -- addr U COM

Leave the address of a variable containing the last error code retrieved by `ERROR` or by the error system. The default value is -99, that means no error. The range of this variable depends on the contest: if `WARNING` is 1, it is set to a value in the range {0..15}; if `WARNING` is 0, it is a positive value in the range of the default error strings array; if `WARNING` is -1, the range is {0..32767} and the user is free to use his own positive codes to manage his error routines. This variable should not be changed by the user, because it is changed by `ERROR` pr ny the error system and reset to -99 in case of range or buffer error in `ERROR`.

ERROR n -- FIG

Execute the user error notification. Set `ERR` to the positive value `n`. `WARNING` is then examined. If `WARNING` = 1, the text of line `n`, relative to the most recently loaded screen buffer, is printed, with `n` in the range {0..15}. If `WARNING` = 0, the error condition `n` is passed to the system error management procedure, simulating a real error condition, with `n` in the range established in `errors.h`. If `WARNING` = -1, (`ABORT`) is executed, which executes in turn the system `ABORT`. The user may change this by creating his own (`ABORT`), using `ERR` to retrieve the user error code. If `n` is negative, the error state is set without any message. In case `n` is not in the defined ranges (if an error message cannot be printed), a real error condition exists! See also the chapter "Error management".

EXEC-ONLY E TFT

Mark the most recently made dictionary entry as a word which will be usable only from console and not into definitions.⁵⁹ Such words, with the notably exceptions of `IMMEDIATE`, `COMPILE-ONLY` and `EXEC-ONLY` itself) may be compiled using [`COMPILE`], but their behaviour is not guaranteed to work on runtime in the same way it runs when directly invoked.

EXECUTE addr -- F79

Execute the dictionary entry whose Code Field Address is on the stack (that is the start of a compiled region)⁶⁰.

EXIT -- C F79

When compiled within a colon-definition, terminate execution of that definition, at that point, by immediately reaching the end of the word definition. May not be used inside a `DO . . LOOP`⁶¹. This is not controlled by `tetra`, and is left to the responsibility of the programmer.

EXPECT addr n -- F79

Transfer characters from the keyboard, beginning at `addr`, upward, until a "Return" or the count of `n` has been received. Take no action for `n` less than or equal to zero. A condition of error exists if `n` is greater than 255. Nulls are added at the end of the text up to the length `n`. Each typed key is echoed; the backspace can be used to retype mistaken text. The actual count of the typed characters (not necessarily equal to `n`) is stored in the user variable `SPAN`.

⁵⁹ See the related word `BUFFERED` in the tlibrary, and study how it is implemented. `EXEC-ONLY` could be rewritten in Forth in a similar way with 64 (the `exec` bit) as the bit selector.

⁶⁰ See the relative article in this manual devoted to `EXECUTE`.

⁶¹ In these cases, `LEAVE` should be used to exit the loop anticipatedly; `ABORT` would be even more useful in case of nested cycles, when the need of exiting the word should arise, because `ABORT` will clean all stacks, and thus all values remained on the Return Stack.

FALSE -- 0 U COM

A constant leaving the false value.

FENCE U COM

Leave the address of a user variable containing an address below which the user is not allowed to FORGET. In order to use FORGET on an entry below this point, it is necessary to alter the contents of FENCE. If FENCE is null, the limit for FORGET is fixed to the last built-in word. The range of FENCE, since it's an address, is {0..65535} (unsigned number).

FIG-STATE -- addr U TFT

Return the address of the fig-Forth compatibility flag; when this contains zero, tetra behaves in the default way, as a Forth-79 environment; when it contains a value different than zero, tetra behaves more like fig-Forth for the differences that do exist between the two (to execute, if possible, programs written for fig-Forth avoiding much rewriting).

The word 79-STANDARD resets the value of the variable putting 0 in FIG-STATE.

See also the chapter "fig-Compatibility".

FILL addr n ub -- F79

Fill the memory beginning at address addr with a sequence of n copies of character ub. If the quantity n is less than or equal to zero, take no action⁶².

FIND -- addr | 0 F79

Used in the form

FIND <name>

Leave the Link Field Address (a.k.a. the Compilation Address) of the next word <name>, which is accepted from the input stream. If that word cannot be found in the dictionary after a search of CONTEXT through FORTH dictionaries, leave zero.

FIRST -- addr U FIG

A constant leaving the address of the first (lowest) byte of screen buffer #1⁶³.

FLD -- addr U "f-l-d" RWS

A variable containing the field length of the latest output conversion through the <# . . . #> sequence. The range of FLD is {0..255} as an unsigned number.

⁶² The memory is filled byte by byte, but the definition is ambiguous, being n a cell; the modern Forth Standard is less ambiguous asserting that (paraphrasing) ub is stored in each of n consecutive characters of memory beginning at addr, so from addr to addr+n-1.

⁶³ This constant is calculated at start and may vary if the stack or the return stack are re-dimensioned though the relative console options.

FORGET

E

F79

Execute in the form:

FORGET <name>

Delete <name> from the dictionary (<name> must be in the CURRENT vocabulary) and all words added to the dictionary after <name>, regardless of their vocabulary. If the word <name> is not found in the CURRENT through FORTH dictionaries, no error condition is generated.⁶⁴

The fig-Forth standard required that CURRENT and CONTEXT coincide in order to proceed to forget words; this is not strictly required by the Forth-79 Standard but it should be good practice; if the CURRENT dictionary is not in the chain shown by ORDER, you won't find the words to be deleted, and on the contrary, if you specify a word in the CONTEXT chain dictionaries, you could delete words just defined in the CURRENT dictionary.

So, even if not specified by the Forth-79 Standard, my advice is to follow the fig-Forth golden rule.

FORTH

I

F79

The name of the primary vocabulary. Execution makes FORTH the CONTEXT vocabulary. New definitions become a part of the FORTH vocabulary until a different CURRENT vocabulary is established. User vocabularies conclude by ultimately 'chaining' to FORTH, or to the CONTEXT vocabulary, so it should be considered that FORTH should always base each user's vocabularies chain.

H.

n --

RWS

Print n in hexadecimal.

HERE

-- addr

F79

Return the address of the next available dictionary location.⁶⁵
HERE is not a variable, but a regular word returning an address.

HEX

--

RWS

Set BASE to decimal 16 (hexadecimal base). See H..

HISTORY

n --

TFT

Print the list of the last n commands that were input from keyboard. The HISTORY command itself is not counted (it will be in the next history search). If n is zero or negative, nothing is printed, if greater than 1000, it is resized to 1000.

⁶⁴ This is a minor difference with the Standard, that explicitly prescribes that "Failure to find <name> in CURRENT or FORTH is an error condition.". I set it as such in order to use FORGET as a simple MARKER (ref. ANSI 1994); simply FORGET a specific marker word, and then re-define it at the very beginning of your program; this will let you reload unlimited times the same dictionary without redefinitions and without the necessity to manually FORGET specific sections of the dictionary. This is useful during the implementation phase, e.g.:

```
FORGET MARKER
: MARKER ;
```

Of course, in the design and implementation phase, FENCE must be avoided; it becomes necessary only when you are sure your program works (when you can also decide to remove the MARKER section). See also option --strict or the flag STRICT-STATE.

⁶⁵ As said earlier in this manual, since the program dictionary starts at location 1 of the memory, the quantity of occupied cells in the program memory can be calculated with HERE-1.

HLD	-- addr	U	FIG
0	0	0	0
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5
6	6	6	6
7	7	7	7
8	8	8	8
9	9	9	9
10	10	10	10
11	11	11	11
12	12	12	12
13	13	13	13
14	14	14	14
15	15	15	15
16	16	16	16
17	17	17	17
18	18	18	18
19	19	19	19
20	20	20	20
21	21	21	21
22	22	22	22
23	23	23	23
24	24	24	24
25	25	25	25
26	26	26	26
27	27	27	27
28	28	28	28
29	29	29	29
30	30	30	30
31	31	31	31
32	32	32	32
33	33	33	33
34	34	34	34
35	35	35	35
36	36	36	36
37	37	37	37
38	38	38	38
39	39	39	39
40	40	40	40
41	41	41	41
42	42	42	42
43	43	43	43
44	44	44	44
45	45	45	45
46	46	46	46
47	47	47	47
48	48	48	48
49	49	49	49
50	50	50	50
51	51	51	51
52	52	52	52
53	53	53	53
54	54	54	54
55	55	55	55
56	56	56	56
57	57	57	57
58	58	58	58
59	59	59	59
60	60	60	60
61	61	61	61
62	62	62	62
63	63	63	63
64	64	64	64
65	65	65	65
66	66	66	66
67	67	67	67
68	68	68	68
69	69	69	69
70	70	70	70
71	71	71	71
72	72	72	72
73	73	73	73
74	74	74	74
75	75	75	75
76	76	76	76
77	77	77	77
78	78	78	78
79	79	79	79
80	80	80	80
81	81	81	81
82	82	82	82
83	83	83	83
84	84	84	84
85	85	85	85
86	86	86	86
87	87	87	87
88	88	88	88
89	89	89	89
90	90	90	90
91	91	91	91
92	92	92	92
93	93	93	93
94	94	94	94
95	95	95	95
96	96	96	96
97	97	97	97
98	98	98	98
99	99	99	99

A user variable that holds the address of the latest character of text during numeric output conversion.

HOLD C -- F79

Insert char c into a pictured numeric output string. May only be used between <# and #>.

I -- n C F79

Copy the current loop index onto the data stack⁶⁶. May only be used in the form:

DO ... I ... LOOP or
DO ... I ... +LOOP or

the word `I` always refers to the most inner loop in case of nested `DO...LOOPS`.

IF f -- I,C F79

Used in a colon-definition in the form:

```
f IF ... ELSE ... THEN or
f IF ... THEN
```

In the first form, if flag `f` is true, the words following `IF` are executed and the words following `ELSE` are skipped; if `f` is false, the words following `IF` are skipped and the words after `ELSE` are executed. The second form shows that the `ELSE` part is optional; in this case, if `f` is false, words between `IF` and `THEN` are skipped and never executed. `IF . . ELSE . . THEN` conditionals may be nested.

At compile-time, IF compiles the runtime word (IF) and reserves space for a 32-bit address, that will be later used by ELSE or THEN.

IMMEDIATE F79

Mark the most recently made dictionary entry as a word which will be executed when encountered during compilation rather than being compiled.⁶⁷

This method allows definitions to handle unusual compiling situations, rather than build them into the fundamental compiler. The user may force compilation of an immediate definition creator by preceding it with `[COMPILE]`.

```
INCLUDE          --                                ANS
```

Used in the form:

```
INCLUDE <file>
```

⁶⁶ In his book *"The complete Forth"*, in paragraph 9.6. Alan Winfield presents his STEP . . DOWN procedure, which is a sort of a one-way DO . . LOOP. In this scheme he never uses DO . . LOOP, but his compiler lets him use I inside a word and outside of a DO . . LOOP. This contradicts the Standard Forth-79, which specifies that I (and J) must be used *"only in the form"* of a DO . . LOOP. Anyway, the task of I outside of a DO . . LOOP scheme can be safely performed by R@, and at all effect, I is equal to R@ for all licit usages. This is confirmed indirectly by the Forth definition of I: *"Used within a DO-LOOP to copy the loop index to the stack."*. Other usages were left to implementors, but the Forth-79 later removed this faculty.

⁶⁷ See the related word `BUFFERED` in the library, and study how it is implemented. `IMMEDIATE` could be rewritten in Forth in a similar way with 1 (the precedence bit) as the bit selector.

Parse and load the file <name>, that can be given with the whole absolute or relative path if located in a different directory. Please note that, in case the name contains a space, this must be preceded by the escape character \, in order to let INCLUDE to parse the space, that usually stops the file name reading⁶⁸. It is the equivalent command to LOAD, with the difference that INCLUDE uses files of the Files System, while LOAD uses blocks (also located in the Files System, but not perceivable as files by the user). If an error in the file is found, the situation before executing INCLUDE is restored.

INCLUDED addr u -- ANS

This word expects on Top of Stack the address and count of a string <name>, and proceed to load it. As for INCLUDE, this is another equivalent command to LOAD, with the difference that INCLUDED uses files of the Files System, while LOAD uses blocks (also located in the Files System, but not perceivable as files by the user). If an error in the file is found, the situation before executing INCLUDED is restored.

INFO n -- TFT

Print some data about **tetra** settings and characteristics, in the following sections:

- *Running mode*, specifying if **tetra** works in Forth-79 mode, the default, or in fig-Forth emulation mode.
- *Memory settings*, with all the width and starting addresses of **tetra**'s memory sectors (dictionary, buffers, stacks); also, the current free memory amount in bytes is reported.
- *Interpreter status*, with news about last compiled word, last accessed block, last accessed buffer and the total number of compiled words.⁶⁹
- *Execution limits*, with the extents limits for strings, numbers, the PAD, the division type, the characters width and so on.
- *System dictionary status*, with the list and count of the following words classes: immediate words, compile-only words and executable-only words.
- *User dictionary status*, with the list of user words and user vocabularies.
- *List of system directories and files*.

The number n on top of stack drives the printing features:

- * If n = 0 print in four screens, after each (but the last) you are required to type ENTER to continue, or the process is stopped.
- * If n = 1 print all info sections continuously
- * If n = 2 print only the sections *Running mode* and *Memory settings*
- * If n = 3 print only the section *Interpreter status* (accessed also by WHERE)
- * If n = 4 print only the section *Execution limits*
- * If n = 5 print only the section *System dictionary status*
- * If n = 6 print only the section *User dictionary status*
- * If n = 7 print only the section *System directories and files*

Any other value of n is ignored.

The INFO word is only informative, but can be of great help when you want to program **tetra** near its limit.

⁶⁸ For example, the file 'two cases.ft' should be invoked as INCLUDE two\ cases.ft, without quotes. If you omit the escape character, the statement is equivalent to INCLUDE two followed by the word cases.ft; in both cases, it's almost certain that an error will occur.

⁶⁹ This is the output of the word WHERE, which can be found as a word by itself, belonging to the Reference Word Set of Forth-79.

INTERPRET

--

RWS

Begin interpretation at the character indexed by the contents of >IN relative to the block number contained in BLK, continuing until the input stream is exhausted. If BLK contains zero, begin interpretation of characters from the terminal input buffer, resetting >IN.⁷⁰

If the word name cannot be found after a search in CONTEXT through FORTH dictionaries, it is converted to a number according to the current base. If a decimal point is found as part of a number, a double number value will be left. The decimal point has no other purpose than to force this action. That also failing, an error message echoing the name with a " ?" will be given. See also "Error management".

IP

U

TFT

IP is a user array holding the 4-numbers Internet Protocol address; by default they hold respectively 127, 0, 0 and 1 (PORT opens by default the local 127.0.0.1 address), but the programmer can choose another address by specifying new values for the array using IP! (writing directly into memory, using ! and appropriate offsets from IP, is prohibited).

The array is composed by four 8 bits bytes (in the range 0÷255); IP marks the address of the first byte.

IP can be used to quickly inspect the four IP sub-addresses numbers, for instance using DUMP:

```
IP 4 DUMP
00425  0000000127  0000000000  0000000000  0000000001  ....
```

The memory locations are accessed with:

```
IP C@
IP 1+ C@
IP 2 + C@
IP 3 + C@
```

and new values are storer with:

```
<n1> IP C!
<n2> IP 1+ C!
<n3> IP 2 + C!
<n4> IP 3 + C!
```

See also the library programs IP! and IP@ that may be use for this scope, and the utility MAKEADDR that prints the IP address in memory.

J

-- n

C

F79

Return the index of the next outer loop. May only be used within a nested DO . . LOOP in the form:

```
DO ... DO ... J ... LOOP ... LOOP
```

⁷⁰ The Standard does not specify the resetting of >IN but it seems a natural choice.

KEY -- C F79

Leave the ASCII value of the next available character from the current input device. The typed character is not echoed, but its ASCII code is left on the stack as a single number with 8 higher bits set to zero. This procedure is blocking.

L/SCR	-- 16	U	COM
-------	-------	---	-----

A constant leaving the number of lines per screen. Lines are numbered from 0 to 15, thus from 0 to $L/SCR-1$.

LAST	-- addr	U	RWS
-------------	---------	---	-----

A variable containing the address of the beginning of the last dictionary entry made, which may not yet be a complete or valid entry. Being an address, its range is $\{0..65535\}$ as an unsigned number.

LEAVE C F79

Force termination of a DO . . LOOP at the next LOOP or +LOOP by setting the loop limit equal to the current value of the index. The index itself remains unchanged, and execution proceeds normally until the loop terminating word is encountered.

LIB -- TET

Used in the form:

LIB <text>

Set <text> (which must be a file name) as the current library.

If `<text>` is empty (LIB is invoked with no arguments), the library file⁷¹ is reset to the default one (\$HOME/.tetra/tlibrary.ft for Linux, Unix or Cygwin, and C:\Users\<uname>\AppData\Roaming\tetra\tlibrary.ft for the Windows® Prompt), and the current library file (with its directory) is printed to screen.

LICENSE -- TFT

Print the licensing text.

LIMIT U FIG

A constant leaving the address just above the highest memory available for a screen buffer. In **tetra** this corresponds to the Data Stack start (and incidentally has the same value of `S0`).

LIST un -- F79

List the ASCII symbolic contents of block un (which corresponds to a block number), setting SCR to contain un. un is unsigned (as in the original Forth-79 Standard)⁷². LIST uses BLOCK to load the block in memory, in the first available screen buffer, whose value is yield by BFR. At all effects, LIST updates SCR and BFR.

⁷¹ I'm very grateful to Bruce Axtens who wrote the routine for setting the default library file under the execution of the `--config` option.

⁷² See also what is written for `LOAD`. See also the library word `SLIST` ("screen-list") that lists the ASCII symbolic content of screen un-
(which corresponds to a screen in memory, numbered 1 to 3), in the very same output of `LIST`.

```
LISTEN          un1 -- un2          TFT
```

Listen to port un1 (which must be opened with PORT), and leave the received byte un2 (which is a value between 1 and 255); in case un2 = 0 it may mean an error in reading.

LITERAL	n --	I,C	F79
----------------	------	-----	-----

If compiling, then compile the stack value `n` as 16-bit literal, which when later executed, will leave `n` on the stack.⁷³

LOAD un -- F79

Begin interpretation of screen `un` (unsigned⁷⁴) by making it the input stream; preserve the locators of the present input stream (`>IN` and `BLK`) and the buffers, setting temporarily `BLK` to `un` and `>IN` to zero. If interpretation is not terminated explicitly, it will be terminated when the input stream is exhausted. Control then returns to the input stream containing `LOAD`, determined by the original input stream locators `>IN` and `BLK`. At all effects, `LOAD` sets `BFR` and preserves `SCR` and `BLK`.

LOAD-SYSTEM -- f E TET

This word is used in the form:

LOAD-SYSTEM <file>

to load the dictionary previously saved in <file> by SAVE-SYSTEM, substituting the current dictionary. If <file> contains spaces, it must be enclosed in double quotes.

The returned value is 0 if loading was correct, and 1 in case `<file>` couldn't be loaded.

LOOP	--	I,C	F79
------	----	-----	-----

Increment the DO . . LOOP index by one, terminating the loop if the new index is equal to or greater than the limit. The limit and index are signed integers.

At compile-time, LOOP compiles the runtime word (LOOP) and the branch address left on the control flow stack by DO⁷⁵.

LSEE	--	"library-see"	TFT
-------------	----	---------------	-----

Used in the following form:

LSEE <name>

Print <name> source, as written in the current library (by default `tlibrary.ft`). Variables and constants are shown along the following colon definition.

⁷³ The Standard does not specify the compile-only status for `LITERAL`, but the definition seems to suggest this status, as explained in the paragraph "A word about `LITERAL` (and `DLITERAL`)". This is a minor difference with respect to the Standard (actually more a regularization rather than an enhancement).

⁷⁴ In the Forth-79 Standard, `LIST` explicitly uses unsigned values; for `LOAD`, it's not specified if the value is signed or unsigned. `tetra` treats consistently the both of them, so that block references are unsigned for `LOAD` too (as it is also for `BLOCK` and `BUFFER`); this guarantees that blocks from 1 to 65535 can be created in the File System (remember that `tetra` is a UNIX tool).

⁷⁵ In **tetra**, the call to **LOOP** is in reality a call to **+LOOP**, passing to it the increment 1.

M*/ d1 n1 n2 -- d2 "m-star-slash" ANS

Multiply d1 by n1 producing the triple-cell intermediate result t. Divide t by n2 giving the double-cell quotient d2. The definition in the ANSI 1994 use a positive n2 and leave indefinite (*ambiguous condition*) when this is zero or negative. In **tetra**, n2 can also be negative, and an error message of division by zero is returned in case it is null.

MAX n1 n2 -- n3 "max" F79

Leave n3, the greater of the two numbers n1 and n2.

MAXBUF n1 n2 -- n3 "max" APP

A constant leaving the maximum number of block buffers which would currently fit in memory.

MESSAGE n -- FIG

Print the error message corresponding to code n. WARNING is first examined. If = 1, the text of line n, relative to the most recently loaded screen buffer, is printed, with n in the range {0..15}. In all other cases, the system error message n is printed, with n in the range established in `errors.h`. In case n is not in the defined ranges, an error condition exist. See also the chapter "Error management".

In the library, you can see for instance the word `PERRORS` (that is "Print ERRORS"), that uses `MESSAGE` to print all errors, cycling in the list using `ERNUM`.

MIN n1 n2 -- n3 "min" F79

Leave n3, the smaller of the two numbers n1 and n2.

MOD n1 n2 -- n3 "mod" F79

Divide n1 by n2, leaving the remainder n3, with the same sign as n1.

MONITOR i*x -- i*x FIG

Start a shell session⁷⁶. The shell is the one defined in `custom.h` and by default it is set to `/bin/bash`. The shell starts from the current directory; before the shell session, all buffers are saved; during the shell session, the directory can be changed but when the shell command `exit` is typed, the **tetra** session is restored in the original directory. At all effects, apart the intervening shell session, nothing changes in the **tetra** session⁷⁷. The fig-Forth named this word `MON` but I prefer the complete name, and `MON` was also the name of the super-immediate word of the Alpha version, with the same identical properties, now promoted to regular and compilable word starting from the Beta version.

MOVE addr1 addr2 n -- F79

Move the specified quantity n of cells beginning at `addr1` to memory at `addr2`. The contents of `addr1` is moved first, proceeding toward high memory. If n is zero or negative, nothing is moved.

⁷⁶ Technical speech: the shell is started using the C function `system()`, which creates a fork in the executable, and the created fork starts the execution of a shell session. As such, the shell does inherit the last saved shell state. If you have started **tetra** from the shell, and after a while you type `MONITOR`, the current shell history will not contain the commands typed so far, because the hosting shell, which is underneath **tetra**, has not rewritten the history file yet.

⁷⁷ Why not setting **tetra** as a shell environment? Add it to the shell list in `/etc/shells` as root, and have fun with **tetra**! For all specific shell needs, type `MONITOR` and you're done! You can also start your session with `$ tetra --init=monitor` to begin a shell session *even before* Forth!

MS u -- RWS

Wait for at least `u` milliseconds. This lets wait from 0 to circa 65 seconds for something to complete (from 0 to 65535 milliseconds). The measure unit is always in milliseconds and is independent from `#CLOCK`.

NEGATE n -- -n F79

Leave the two's complement of number n, i.e., the difference of zero less n.

NOOP	-- n	TFT
------	------	-----

A constant leaving the opcode of TASK (the word that does nothing); useful for detecting CREATE objects. See for instance the PFA word in the tlibrary.

NOT	f1 -- f2	F79
------------	----------	-----

Reverse the boolean value of flag `£1`. This is identical to `0=` (and in effect it **is** `0=`)⁷⁸.

OF	n1 n2 -- n1	I,C	ANS
-----------	-------------	-----	-----

At compile time, OF compiles the runtime word (OF) and reserves space for a 32-bit address which will be later filled by ENDOF; this forward reference in the control stack marks the beginning of a conditional branch. At runtime, perform the test by comparing the original value and the current comparison value; if test is true, continue execution, or else jump to the next OF or to the default section.

OFF	addr --	COM
------------	---------	-----

Set the value of the variable at `addr` to `false`.

OFFSET	-- addr	U	RWS
---------------	---------	---	-----

Leave the address of a variable whose content is interpreted as a block offset (with respect to the default value returned by `BLOCK`, which is the position of the first character in the screen buffer). The value in `OFFSET` is always treated as an unsigned number, with range $\{0..65535\}$, but since `OFFSET` is related to a screen block, whose dimension is 1024 bytes ranging from 0 to 1023, the proper values for it are in the range $\{0..1023\}$.

OFFSET influences LOAD, that skips OFFSET characters of the screen block during interpretation. The default value is zero, and the user may add any value (in the range from 1 to 1023) to skip text in the block that's being interpreted. A value of 1024 means that the whole block is skipped.

ON	addr	--	COM
----	------	----	-----

Set the value of the variable at `addr` to `true`.

⁷⁸ The Forth-83 Standard (and later) changed the NOT behavior to a more strict bits inversion (the one's complement); this was the main reason (or the main effect) of the turning the True value from 1 to -1, because it is the inversion of zero - all bits set).

OPEN-FILE `addr n fam -- file-id 0` ANS
 `addr n fam -- 0 1`

Open the file whose name is given by the character string at `addr` with length `n`, with file access `fam`. If the file does not exist, and `W/O` is specified as access mode, the file is created empty. Set the file position to the start of file if reading, or the to end of file if writing. Return values are in order:

If opening was successful, `file-id` is a non-zero unsigned integer in the range {1..6}; otherwise return zero. If a connection could be established return zero, otherwise return one.

OR `n1 n2 -- n3` F79

Leave the bitwise logical inclusive-or `n3` of `n1` and `n2`.

ORDER `--` ANS

Display the names of the vocabularies in the search list, in their present search order sequence and the vocabulary in which new definitions will be placed (the `CURRENT` vocabulary).

OUT `-- addr` U FIG

Leave the address of a variable holding the current output position on the terminal. It is incremented by all printing words `EMIT`, `.` (dot-quote), `SPACE`, `SPACES`, `U.R`, `D.`, `D.R`, `U.`, `UD.`, `.` (dot), `TYPE` and it's set to zero by `CR`, `PAGE` and before each command invocation from console. The word `GOTOXY` in the tlibrary sets `OUT` to the value of the `x` coordinate. The user may alter and examine `OUT` to control display formatting.

OVER `n1 n2 -- n1 n2 n1` F79

Leave a copy of the second number on the stack.

PAD `-- addr` F79

Leave the address of a temporary scratch area, used to hold character strings for intermediate processing. The capacity of `PAD` is 84 characters⁷⁹ and is located always 200 cells after `HERE`⁸⁰. If there is no more room for the sum of `PAD` plus the fixed distance from `HERE` in dictionary memory, `PAD` is moved to the Scratch Area and possibly resized to no more than 512 characters.

`PAD` is not a variable, but a regular word returning an address.

PAGE `--` RWS

Clear the terminal screen, and position cursor on the upper left screen at coordinates 0,0.

The screen clearing uses VT100 codes that should apply to all monitors, included the monitors emulated in Desktop applications (for instance, it works on Linux Terminals).

⁷⁹ The standard prescribes 64 bytes as the minimum quantity. `tetra` has a default greater quantity, but you can anyway change the pad length for the current session (in the range 64..1024) using the console option `--pad=<n>`. See the man page for details.

⁸⁰ The standard prescribes a *scratch area*, without more specification. So I followed an old Forth tradition, and set up a dynamic `PAD` that is located at a fixed distance from `HERE`, assuring that it is always available. Anyway you can anyway change the pad distance (in the range 200..1024) using the console option `--pad-distance=<n>`. See the man page for details.

PC -- addr U TFT

Leave the address of a variable containing the current Program Counter, which is not null during a program execution. PC refers to the dictionary position currently in execution, and can be altered to direct the next execution elsewhere (vectored execution). Being an address, its range is {0..65535}.

If PC is set to 0 (zero), program execution will stop and return control to console. This trick can be used inside DO . . LOOP cycles, where EXIT cannot be used.

PC is updated, after every token turn, by 2 units. All instructions involving more than two bytes, update their own count by themselves, but if you want to set the Program Counter before those instructions, you have to record PC at the proper point or calculate by yourself the real offset. Take a look at the Appendix for notes about compilation formats.⁸¹

PICK n1 -- n2 F79

Return the contents of the n1-th stack value, not counting n1 itself. An error condition results for n1 less than one.

2 PICK is equivalent to OVER.

1 PICK is equivalent to DUP.

The range of n1 is {1..n}, where n = DEPTH.⁸²

PORT n1 f1 -- f2 TFT

If f1 is not null, try to open port n1, leaving flag f2 as a zero on success, or -1 if port could not be opened (i.e. socket could not be created) or -2 if a connection couldn't be established.

A warning: ports may be available or not, on your computer, depending on the firewall or/and on your network setup. So, to make tetra be aware of ports, you should enable them, otherwise the connection will always fail.

If f1 is null, closes the connection (established or not) and return f2 = -1.

QUERY -- F79

Accept input of up to 80 characters (or until a 'return') from the terminal, into the terminal input buffer. WORD may be used to accept text from this buffer as the input stream, by setting >IN and BLK to zero. Query has a limited keyboard control (echo for the current key-press, Backspace and Return).

QUIT -- F79

Clear the return stack, setting interpret mode and return control to the terminal. No message is given.

R0 -- addr "r-zero" FIG

A constant leaving the address of the bottom of the Return Stack.

⁸¹ Look also at the file `stepdown.ft`, included in the package, to see how `tetra` uses PC to alter the program flow. Please, remember that, in `tetra`, the current execution address is NOT saved to the Return Stack but saved in PC.

⁸² The Forth-83 Standard (and later) changed the index n1 reference from {1..n} to {0..n}, so that the two versions are not compatible.

R> -- n C "r-from" F79
 RS: n --

Remove n from the return stack to the data stack.

R@ -- n C "r-fetch" F79

Copy the number on top of the return stack to the data stack.

READ-BLOCK un1 un2 -- addr | 0 TFT

Read the block file un1 from the current blocks directory and copy it to the screen buffer un2. In case of success, leave the screen buffer address addr for subsequent usages, otherwise leave zero. The cases of failure include the impossibility to read, or a wrong buffer number (the range of un2 is {1..3}). This is a low-level procedure that is incorporated in LIST, LOAD, BLOCK, BUFFER and EDIT and in general in any word that retrieves a block from the file system. The previous content of the screen buffer is lost, because this procedure doesn't control if the destination screen buffer is updated or not, a type of control usually performed by BLOCK; SCR is not updated.⁸³ BFR is set to un2 in case of success, to zero otherwise. See BLOCK.

READ-LINE addr n1 file-id -- n2 n3 n4 ANS

Read one line of text from the file identified by file-id and store it at address addr, for at most n1 characters. Terminate the reading if a line terminator was found, or n1 characters were read; the return values are in order:

- if the reading was successful, n2 is the actual count of read characters; otherwise return zero;
- in case of successful reading, if the end of file was reached after at least one READ-LINE execution, n3 is 1, otherwise zero.
- in case of successful reading, n4 is 0; in case something went wrong, return 1 in case the file was never been opened, and 2 in case the file connection wasn't correctly initiated.

REPEAT I, C F79

Used in a colon-definition in the form:

BEGIN ... WHILE ... REPEAT

At runtime, REPEAT returns to just after the corresponding BEGIN.

At compile time, REPEAT compiles the runtime word (REPEAT), reserves one 32-bit cell, retrieves two addresses from the control flow stack, the topmost left by WHILE and the innermost by BEGIN; compiles its address on the address left by WHILE, and compiles the address of BEGIN on the reserved cell.

REQUIRE E TFT

Used in the form:

REQUIRE <name>

Parse file tlibrary.ft⁸⁴ (looking for it in the default library file), read and compile the word <name> and

⁸³ I must warn you that, since SCR is not updated by the usage of READ-BLOCK, an improper use of this word may cause overwriting a buffer that you may not have saved yet.

⁸⁴ Please note that a) since the reading of the definition starts from the : <name> token and proceeds until the next colon is met, all intermediate direct Forth code is executed (included VARIABLE, IMMEDIATE etc...); b) only colon definitions are loaded through REQUIRE; c) constants in the library are defined as colon definitions to let REQUIRE find them; for what about all the library variables, you have to declare them on your own before requiring the words that use them (e.g. TIMING - see ahead), or define them through CREATE.

all dependencies word that are in the same library⁸⁵. At all effects, it's like the definition of <name> is typed from keyboard, along with all the other definitions that occur in it. REQUIRE cannot be compiled, and is executable-only.

This word works like ENSURE, with the difference that REQUIRE will always load the word, existing or not in the dictionary, while ENSURE will do it only if the word does not exist in the dictionary.

Note: the Standard uses REQUIRE in the same sense of INCLUDE, that is *require file*; in **tetra**, instead, the sense is *require word*.

ROLL n1 - F79

Extract the `n1`th stack value to the top of the stack, not counting `n1` itself, moving the remaining values into the vacated position. An error condition results for `n1` less than one.

```
3 ROLL = ROT
2 ROLL = SWAP
1 ROLL = null operation
```

The range of `n1` is $\{1..n\}$, where $n = \text{DEPTH}$.⁸⁶

```
ROT          n1 n2 n3 -- n2 n3 n1  "rote"          F79
```

Rotate the top three values on the stack, bringing the third to the top.

```
RP!          addr --          "rp-store"          COM
```

Store `addr` in the return stack pointer. If `addr` is lower than `r0` or greater than the memory limit⁸⁷, it is resized accordingly.

S0	-- addr	"s-zero"	RWS
-----------	---------	----------	-----

A constant leaving the address of the bottom of the Data Stack (and incidentally this is the same value returned by `LIMIT -1`).

SAVE-BUFFERS F79

Write to mass-storage all blocks that have been marked as UPDATED. An error condition results if writing to mass-storage is not possible.

Since the library is written in DECIMAL, REQUIRE (and ENSURE) compile the word in DECIMAL, restoring current BASE afterwards. So that you must not care of setting DECIMAL before loading a library word.

If you plan to enhance the library by adding variables, constants or CREATED objects outside definitions, remember to group them and precede them with a dummy definition; when this dummy definition is REQUIRED, all the variables/constants/CREATE words that follow will be added to the dictionary; otherwise, they must be explicitly typed from keyboard or written to file/block. See for instance the TIMER words in the library.

⁸⁵ The file `ttlibrary.ft` is a collection of additional dictionary words, written in Forth, in form of colon definitions, without unresolved dependencies.

⁸⁶ The Forth-83 Standard (and later) changed the index range from $\{1..n\}$ to $\{0..n\}$, so that the two versions are not directly compatible.

⁸⁷ The memory limit is 65535.

```
SAVE-SYSTEM      -- f          E          TFT
```

This word is used in the form:

SAVE-SYSTEM <file>

to save the current dictionary in <file>; the file will contain the entire dictionary (variables, compiled words, constants), but will not include the stacks or the buffers. <file> can be lately loaded in memory with `LOAD-SYSTEM`. If <file> contains spaces, it must be enclosed in double quotes. This word is execute-only (cannot be compiled).

The returned value is 0 if saving could get accomplished, and 1 in case of error on file opening or writing.

SCR	-- addr	U	F79
------------	---------	---	-----

Leave the address of a variable containing the number of the screen most recently listed with LIST or edited through EDIT.

SEARCH	E	TFT
--------	---	-----

Used in the form:

SEARCH <text>

Search in every block for the string `<text>`, and print every block number, along with the line number (enclosed in parentheses) where it occurs.

The search is case insensitive (that is, if FORTH is the search pattern, FORTH, forth and Forth will all match the search).

```
SEARCHLIB      -- n      E      TFT
```

Used in the form:

SEARCHLIB <text>

List all the colon definitions in the current library that contain the pattern `<text>`; at the end, `ok` is printed, returning the words count, or zero if the pattern could not be found.

If the pattern <text> is empty (not given), print the names of all words in the library that remain to be ensured, as with WORDS, returning the library words total count on the stack.

The search is case insensitive (that is, if FORTH is the search pattern, FORTH, forth and Forth will all match the search).

SOUND	n1 n2 --	TFT
--------------	----------	-----

A procedure to produce a sinusoidal sound with a given frequency and a given duration. Parameters are interpreted this way: n1 is the duration in milliseconds, while n2 is the frequency in Hertz.

For instance, to produce the A note, useful to tune a guitar, use the following command:

```
2000 440 SOUND
```

This sequence will produce a 2 seconds A tone. Remember that, to use this, the system must not be use the *mute* mode.

This word, quite experimental and not portable, was added to let the programmer create its own music or bell words.

SP !	addr --	"sp-store"	COM
-------------	---------	------------	-----

Store `addr` in the data stack pointer (SP stands for Stack Pointer). If `addr` is lower than `s0` or greater to or equal to `r0` it is resized accordingly.

SPACE	--	F79
--------------	----	-----

Transmit an ASCII blank to the terminal screen.

SPACES	n --	F79
---------------	------	-----

Transmit `n` spaces to the terminal screen. Take no action for `n` = zero or less.

SPAN	-- addr	U	F83
-------------	---------	---	-----

Leave the address of a variable containing the number of characters typed during last `EXPECT` execution, in the range {0..255} as an unsigned byte.

STATE	-- addr	U	F79
--------------	---------	---	-----

Leave the address of a variable containing the compilation state, which is non-zero for the compilation state, and 0 (zero) for the interpretation state.

STDERR	-- n	U	TFT
---------------	------	---	-----

A constant leaving the code for the standard output channel `stderr` (the UNIX standard error channel) that may be used with `WRITE-LINE`.

STDIN	-- n	U	TFT
--------------	------	---	-----

A constant leaving the code for the standard input channel `stdin` (the UNIX standard input channel) that may be used with `READ-LINE`.

STDOUT	-- n	U	TFT
---------------	------	---	-----

A constant leaving the code for the standard output channel `stdout` (the UNIX standard output channel) that may be used with `WRITE-LINE`.

About this matter read the paragraph "Making Sound with tetra".

```

STRICT-STATE          -- addr          U          TFT

```

Leave the address of a variable containing the strict-behaviour state, which is non-zero for the strict-behaviour state, and 0 (zero) for the advanced-features state (the default).

SWAP	n1 n2 -- n2 n1	F79
-------------	----------------	-----

Exchange the top two stack values.

SYSTEM	--	I	COM
--------	----	---	-----

Used at console level to parse the entire string that follows in the input stream and pass it unchanged to the underlying shell as a shell command; this enables using the shell from within **tetra**, in a fixed directory state⁸⁹. This word is the normal version of the super-immediate word **SH** that will execute even in case of messed up dictionary.

SYSTEM arguments extend until the end of the input buffer, so that it must not be followed by any other Forth word (that will be interpreted as an argument).

Remember also that while `SYSTEM` can be written in lower or upper case, the rest of the command is passed as-is, because the shell is case-sensitive.

TASK	FIG
------	-----

A no-operation word⁹⁰ which can be used as safe null operation (for instance in referenced functions, to reset some feature to null, avoiding dangerous zeroes).

Originally, in fig-Forth, TASK was used to mark the boundary between applications. By forgetting TASK and re-compiling, an application could be discarded in its entirety. *tetra* does not need such a feature, because of its peculiar FORGET behaviour.

TETRA FIG

This informative word⁹¹ prints the Operating system on which it is executed and the version of the program. For example, on my laptop it prints:

```
tetra  x86_64-suse-linux 1.0-Beta Jan 11 2025 ok
```

⁸⁹ Technically speaking, each `SYSTEM` call is an independent fork, passed through the C function `system()` to the shell but, on returning, the forked program ceases to exist, and all the changes it performed are maintained only if they are physical changes; so, listing directories is possible, and also creating and deleting files and directories, but the base directory remains fixed, and cannot be changed: in facts, the shell command `cd` changes the directory for the forked instance, and on returning, this change is lost.

⁹⁰ I underline here that a no-operation word does nothing, has no arguments, and returns nothing.

⁹¹ Swiftforth has a similar word, called of course swiftforth.

THEN -- I, C F79

Used in a colon-definition in the form:

IF ... ELSE ... THEN or
IF ... THEN

THEN is the point where execution resumes after ELSE or IF (when no ELSE is present).

At compile time, THEN compiles the runtime word (THEN), retrieving from the control flow stack the address left there by IF or ELSE, and compiling there its proper address.

TIB	-- addr	U	FIG
------------	---------	---	-----

A constant leaving the address of the first character of the Terminal Input Buffer.

TIME	--	n1	n2	n3	COM
------	----	----	----	----	-----

Leave on stack current seconds n1, current minutes n2 and current hour n3 on top. n1 is in the range {0..59}, n2 is in the range {0..59}, n3 in the range {0..23}.

TRACE	E	TFT
-------	---	-----

Used in the form:

TRACE <name>

Enable the debugger on the user word <name>, by setting a trace-point at it; the debugger exits when <name> has ended its duty. The trace-point is not removed until the end of session, or until another different word is TRACED. To manually disable the trace-point, use the parameter-less version:

TRACE

Built-in words cannot be TRACED, or an error condition exists. Note that TRACE prevails over DEBUG, so that if you activate debugging while in trace mode, the debug will be disabled. To assure to be in debugging mode, disable the tracing.

TRUE	-- 1	U	COM
------	------	---	-----

A constant leaving the true value.

TYPE	addr n --	F79
-------------	-----------	-----

Transmit *n* characters beginning at address *addr* to the terminal. No action takes place for *n* less than or equal to zero. If the zero string terminator is found before the *n*th character, stop.⁹²

```
U*      un1 un2 -- ud      "u-times"      F79
```

Perform an unsigned multiplication of `un1` by `un2`, leaving the double unsigned number product `ud`. All values are unsigned.

⁹² I don't know if this is a **tetra**'s feature or if this is common to all Forth-79 interpreters and compilers. I only know that this is an useful feature.

U. un -- "u-dot" F79

Display un converted according to BASE as an unsigned number, in a free field format, with one trailing blank following.

U.R un n -- "u-dot-r" RWS

Display un converted according to BASE, right aligned in an n-characters wide field. If n is negative or smaller than the characters required for un, no leading spaces are given.

U/MOD ud1 un2 -- un3 un4 "u-divide-mod" F79

Perform the unsigned division of double number ud1 by un2, leaving the remainder un3, and the quotient un4. All values are unsigned.

U< un1 un2 -- f "u-less-than" F79

Leave the flag f representing the magnitude comparison of $un1 < un2$ where un1 and un2 are treated as 16-bit unsigned integers.

UD. ud -- "u-d-dot" COM

Display ud converted according to BASE as an unsigned double number, in a free field format, with one trailing blank following.

UD/ ud1 ud2 -- ud3 "u-d-slash" COM

Divide the double number ud1 by the double ud2 and leave the double quotient ud3. All values are unsigned. See / (division).

UNTIL f -- I,C F79

Within a colon-definition, mark the end of a BEGIN . UNTIL loop, which will terminate based on flag f. If f is true, the loop is terminated. If f is false, execution returns to the first word after BEGIN. BEGIN . UNTIL structures may be nested.

At compile time, UNTIL compiles the runtime word (UNTIL), and reserve one 32-bit unit to store the address left on the control flow stack by BEGIN.

UPDATE -- F79

Mark the most recently referenced block as modified. The block will subsequently be automatically transferred to mass storage, should its memory buffer be needed for storage of a different block, or upon execution of SAVE-BUFFERS.

USE	--	TFT
-----	----	-----

Used in the form:

USE <text>

Set <txt> (which must be a legal directory path) as the current directory where to search for blocks. The directory is created empty in case it is not found.

If <text> is a relative path, it will be created in the current directory; if <text> is empty (USE is invoked with no arguments), the system directory⁹³ is reset to the default one (\$HOME/.tetra/blocks/ for Linux, Unix and Cygwin, and C:\Users\<uname>\AppData\Roaming\tetra\blocks\ for the Windows® Prompt), and the current blocks directory is printed to screen.

VARIABLE -- F79

A defining word executed in the form:

VARIABLE <name>

to create a dictionary entry for <name> and allot two bytes (one cell) for storage in the parameter field, setting the variable contents to zero. When <name> is later executed, it will place the starting storage address on the stack.

```
VERSION          -- n1 n2                                TFT
```

Leave on the stack two 16-bit numbers: the minor version number `n1` and the major version number `n2` (on top), for versioning checking of the user programs.

```
VOC-LINK          -- addr          U          FIG
```

Leave the Address of a variable holding the link field of the most recently created vocabulary. All vocabulary names are linked by these fields, and VOC-LINK, through NFA, allows printing their name, or through LFA controls the FORGETTING through multiple vocabularies. All vocabularies in the current chain start from VOC-LINK down to FORTH (the value of VOC-LINK, an address, may be safely stored for later intervention, considering that it is updated at each vocabulary creation).

VOCABULARY -- F79

A defining word executed in the form:

VOCABULARY <name>

to create (in the CURRENT vocabulary) a dictionary entry for <name>, which specifies a new ordered list of word definitions. Subsequent execution of <name> will make it the CONTEXT vocabulary. When <name> becomes the CURRENT vocabulary (see DEFINITIONS), new definitions will be created in that vocabulary. All vocabularies ultimately chain to FORTH. By convention, vocabulary names are to be declared IMMEDIATE⁹⁴. When compiled, vocabularies are referenced by their link address.

⁹³ I'm very grateful to Bruce Axtens who wrote the routine for setting the default blocks directory under the execution of the `--config` option.

⁹⁴ I want to underline here anyway that FORTH and EDITOR, the default vocabularies, are IMMEDIATE words, as required by the Standard (to compile the reference for those, [COMPILE] must be used), and this is an indirect confirmation that vocabularies are to be declared IMMEDIATE. See also the chapter "VOCABULARY problems".

W/O -- fam "w-oh" ANS

A constant which returns the Write File Access Method fam = 2.

W0 -- addr "w-zero" YTH

A constant leaving the address of the first byte of the WORD buffer.

WARM i*x -- I TFT

Initialize or reinitialize the dictionary from the default configuration⁹⁵ in pristine state. After the warm start, the base vocabulary is FORTH, which is also CONTEXT and CURRENT. The numerical base is decimal. Stacks and buffers are preserved for further treatment. Print no message. This word has a service super-immediate version in case of unresponsive dictionary.

WARNING -- addr U FIG

Leave the address of a variable containing a value controlling ERROR messages output. If = 0 (default case), messages will be retrieved from the default errors strings array and the default error message management is simulated. If WARNING = 1, and at least one block was loaded, strings will be retrieved from the most recent screen buffer (ruled by the BFR variable that you can control). If WARNING = -1, (ABORT) is called, for a user specific procedure. After their completion, all cases are followed by the default error resolution operations. See also the entry for ERROR in this manual and the chapter "Error management".

WHERE -- RWS

Print information about the status of tetra, (e.g., after an error abort), printing⁹⁶:

- the last word name compiled
- the last block number accessed
- the last buffer number accessed
- the total words number compiled

WHILE f -- I, C F79

Used in the form:

BEGIN ... f WHILE ... REPEAT

where f is a flag. Select conditional execution based on flag f. On a true f, continue execution through to REPEAT, which then returns back to just after BEGIN. On a false f, skip execution to just after REPEAT, exiting the structure.

At compile time, WHILE compiles the runtime word (WHILE), push its address on the control flow stack and reserves one 32-bit unit.

⁹⁵ As for COLD, only the default built-in dictionary is reinitialized, not the words you might have loaded through LOAD, ENSURE, REQUIRE or INCLUDE, compiled with the colon mechanism or via the resource file \$HOME/.tetra.rc.

⁹⁶ This information, along with many others, is also printed by INFO.

WIDTH	-- addr	U	FIG
--------------	---------	---	-----

A user variable containing the maximum number of letters saved in the compilation of a definition's name. It's by default set to 31 and has a (theoretical) range {1..255}. The name character count and its natural characters are saved, up to the value in WIDTH. The value may be changed at any time within the above limits.⁹⁷

WORD	c -- addr	F79
------	-----------	-----

Receive⁹⁸ characters from the input stream according to the delimiter character *c* and place the characters in a string beginning at *caddr*+1 (in WORD's buffer). If *c* = 1, receives all characters from the input stream. The characters count is placed in the byte position at *addr*. An error condition results if *c* is the ASCII null or if the count exceeds 255. Initial occurrences of *c* in the input stream are ignored only when they are blanks⁹⁹. If *c* appears in the input stream as a terminating character, it is appended to the string but not included in the count. If the input stream is exhausted before *c* is encountered as a terminating character, the terminating null character is appended instead of *c*. A zero length will result if the input stream is empty when WORD is called.¹⁰⁰

WORDS -- COM

List all the definition names belonging to the list presented by ORDER, that is from the CONTEXT vocabulary down to FORTH, in the created chain. As said for VOCABULARY, the FORTH word-list is always active, because all dictionaries are chained to it.¹⁰¹

```
WRITE-BLOCK      un1 un2 -- f                                TFT
```

Write the content of the screen buffer `un2` to the current blocks directory, as block file `un1`. In case of success, leave true, otherwise leave zero. The cases of failure include the impossibility to write to the file system, or a wrong buffer number (the range of `un2` is {1..3}). This a low-level procedure that is incorporated in `SAVE-BUFFERS`, and in general in any word that saves a block to the file system. `BLK`, `SCR` and `BFR` are not changed. See `SAVE-BUFFERS`.

```
WRITE-LINE      addr n1 file-id -- n2                                ANS
```

Write at most `n1` characters from `addr` to the file identified by `file-id`, adding a line terminator after `n1` characters or after the natural character-list end. `n2` is 1 in case of success, zero otherwise.

⁹⁷ **Watch out!** An improper name length may cause irregular functioning or memory segmentation errors; if the length is fixed to a value lesser than the maximum value already compiled (in a user/system word), the system may crash! Change this variable only if strictly necessary, preferably increasing it up to and not beyond 255 (and never decreasing it less than 31).

⁹⁸ This definition is not the one contained in the Standard, but a mix of the one proposed by Robert L. Smith in his "*Forth Standards corner*", about the WORD procedure (contained in *Forth Dimensions* vol. III, N. 3, September/October 1981) mixed with the one designed by me. This new definition, while does not change the Standard effect of WORD, tries to resolve some problems in the Standard definition: no error condition specified in case the delimiter is null; unclear definition of resulting packed string length; unclear text in case of empty stream. And since, in according to Robert L. Smith observations (cit), the skipping of the leading occurrences of the delimiter prevents WORD from treating empty strings, please note that *tetra* does not skip any leading occurrences when they are not blanks. This is a minor difference with the Standard (an enhancement, actually).

⁹⁹ This is a deviation from the Standard.

¹⁰⁰ In the Standard it's not clear if WORD has a dedicated buffer or if it uses a volatile memory portion (like PAD). In building **tetra**, I assigned to WORD a proper and independent buffer, to avoid collisions between the WORD buffer and the dictionary. This buffer is 1024 bytes wide, and is split into four segments, active one at a time. Each segment is 256 wide (255 characters + 1 terminator) and its address is returned in `addr`, so that you can compare two strings got through WORD (because it always returns two distinct addresses in two consecutive calls), or even three or four.

¹⁰¹ See for contrast the `VLIST` word, in the `tlibrary`, that lists all definitions of the `CONTEXT` dictionary only.

XOR	n1 n2 -- n3	"x-or"	F79
------------	-------------	--------	-----

Leave the bitwise exclusive-or of two numbers.

```
[      --      I  "left-bracket"  F79
```

Stop the compilation mode. The words after [are executed, not compiled. This allows calculation or compilation exceptions before resuming compilation with]. See] and COMPILER.

```
[COMPILE]                                I,C  "bracket-compile" F79
```

Used in a colon-definition in the form:

[COMPILE] <name>

Force compilation of the following word. This, for instance, allows compilation of an IMMEDIATE word when it would otherwise be executed. In case the word EXEC-ONLY is applied to a word (to signal it must not be directly compiled), [COMPILE] can override this bit state and compile the word that follows¹⁰²

```

\      --      "slash"      COM

```

Ignore all text from the character following the space that follows to the end of the input line in any regular file, and to the end of line in a block (this means that this comment extends until the last character of current line in a screen 16 x 64 characters - see LOAD)

```
1      -- "right-bracket" F79
```

Set the compilation mode. The text from the input stream is subsequently compiled¹⁰³. See [1]. Read also very carefully the note appearing at the bottom of this page.

¹⁰² As for what about `COMPILE`, builtin words are compiled with a reference to their compilation address, while user words are compiled with a reference to their link address.

¹⁰³ in the ANSI Standard Forth site, at <https://forth-standard.org/standard/core/COMPILEComma>, Anton Ertl reports the following curious code bit:

```
] + [ \ compiling outside a colon definition in compile state.
```

The comment is his own. This example is defined by Mr. Ertl as *non-standard*. Actually, such a definition has no constraints imposed by the Standard (least of all by the Forth-79 Standard): it's a legal and perfectly executable code, because the closing `]` cannot be compile-only (being it always executes in interpret state). Unfortunately, the cumulative effect of this call is to ruin the dictionary by inserting spurious compilation code between definitions and/or allocated memory areas always updating `HERE`. Therefore, although possible, this procedure is not advisable in all contexts, and the words `[` and `]` should only be used into colon definitions, never outside, unless you know very very exactly what you are doing....

3.4. The Line Editor in detail

The Forth-79 Standard does not specify commands for the line editor, or what must be the editor like. It only specifies (in the Reference Word Set) the `EDITOR` vocabulary, as if suggesting that all possible editing words must be put in there, without saying what these words should be.

So the editor commands added to **tetra** are mainly those described in "EXTENDED fig-Forth", by Patrick L. Mullarky, dated 1981. Some additions were also grabbed from "Forth On The BBC Microcomputer", by Richard De Grandis Harrison, dated January 17, 1983, published by Acornsoft.

The editor words are available only after `EDITOR` has been typed. If you often forget this (as I do), define `LIST` to enable editor by default:

```
: LIST    LIST    EDITOR ;
```

A word expecting a list of characters <text> must be used as the last word on the command line; words expecting only numbers on stack or acting without arguments or parameters can be cumulated on the command line.

CLEAR n --

Clear screen *n* (with *n* = 1,2 or 3) by filling it with blanks (ASCII 32), and destroying any previous information.

COPY n1 n2 --

Copy screen *n1* onto screen *n2* (with *n1* and *n2* = 1,2 or 3). It overwrites any previous content of screen *n2*. This operation does not change the current buffer screen references (default buffer and cursor position).

MARK

Mark the current screen as modified. A subsequent `SAVE-BUFFERS` command will cause the entire screen to be written out. It's a synonym of `UPDATE`.

STAMP

Substitute line 0 with a stamp line with date and initials, that must be successively integrated by the programmer.

TEXT

Wait for the input of text from keyboard, up to 64 characters (or until the Return key is pressed) and put them into `PAD`. Text may be used for example by `C` or `I` to insert it in a specific line of the screen buffer.¹⁰⁴ The remaining part of `PAD`, not affected by `TEXT`, is erased each time.

¹⁰⁴ This command, specifically targeted for the Line Editor, is not the equivalent to the command `TEXT` in the Reference Words Section; in particular, this word does not use `EXPECT` to get the string into `PAD` and thus `SPAN` is not affected by this word. If you should need `SPAN`, you can rewrite `TEXT` in the `EDITOR` vocabulary like this:

```
EDITOR DEFINITIONS
: TEXT    PAD 64 EXPECT
```

with identical effects and with `SPAN` containing the characters actually typed by the user.

TILL

Used in the form:

TILL <text>

Delete all text from the current cursor position until end of the matching string <text>. The deletion involves only the current line, and does not act on the rest of the screen.

TOP

Position the cursor at the upper left corner of the current screen (start of screen at row=0 and line=0, or position=0 and cursor=0)

A

Used in the form

A "cccc1" "cccc2"

Look for character string cccc1 in the current screen, starting from the current cursor position, then changes the first occurrence of cccc1 with cccc2, saving cccc1 to PAD for another search with N. Patterns must always be included in double quotes.

B

Move the cursor backward for as many characters are in PAD, until the first null character. The cursor movement is performed by an inner call to M.

C

Used mainly in the two forms:

C <text>

C

Insert text into the current line at the current cursor position. If the argument <text> is given (first form), use <text> as the insertion text; if the second form without argument is used, use the text in PAD as the insertion text.

D

n --

Delete line n in the current screen and moves up all following lines, creating a new (empty) 15th line. This word also saves the contents of the deleted line into PAD, so that you can use the editor word I later, if needed.

E

n --

Erase line n in the current screen, i.e. filling it with blanks, without moving up the following line.

F

Used in the form

F <text>

Look for character string <text> in the current screen, starting from the current cursor position. This words also saves the string <text> in PAD.

H

n --

Copy the content of line n to PAD, leaving the line untouched. It is equivalent to the sequence

n D

n I

I

n --

Insert the content of PAD in a new line created immediately above line n in the current screen, and then moves all following lines, including old nth, down one line (15th line is lost).

L

List current screen. The current screen is changed by n LIST which will list out screen n and make it the current screen. The current screen may also be changed by setting variable BFR.

M

n --

Move the cursor n characters forward or backward (backward if n is negative).

N

Look for next occurrence of the text in PAD. If this text was left by F, N can be used as a faster second instance of F for the same search pattern.

P

n --

It is used in the form

n P <text>

Put the character string <text> into line n, and erase the previous content, if any. Only one space after P is discarded, so that <text> may contain even leading blanks.

R

n --

Replace the content of line in the current screen with the text in PAD.

S

n --

Spread current screen at line n, originating a new empty line immediately preceding line n, and moving all following lines down one line (15th is lost).

T n --

Type out line n of current screen; the cursor position is pointed by a caret (^) put under the correspondent character in case it belongs to line n. This word is called by all the editor words that alter the text and print the modified or search line.

x

It is used in the form

X <text>

Extract the first character string occurrence of <text> and shorten up the line. The extracted text is copied to PAD. This command, as F, begins the search from the current cursor position.

To these, the following variables (also belonging to the EDITOR vocabulary) were added, to ease the user for the management of the screen buffers (with the note that screens are numbered 0 to 15 and positions 0 to 63):

#L U

The value of the current editing line in the current editing buffer. It is updated automatically by the various editor commands. The range is {0..15}.

#P U

The value of the position of the cursor in the current editing row. It is updated automatically by the various editor commands. The range is {0..63}.

R# U

The value of the current cursor position in the current editing buffer as an absolute reference in the range {0..1023}. It is updated automatically by the various editor commands and by `LOAD`.

3.4.1. Some Line Editor techniques

Here is some tips for a proper usage of the Line Editor commands.

Working on the screen

List the screen: use L
Clean the screen: use CLEAR
Copy one screen on another: use COPY
Reset cursor position: use TOP
Mark screen as updated: use MARK

Working on a line

Show a line: use T
Set line text: use P
Clean the line: use E
Remove the line: use D
Move line to another row: use D; use I
Insert a void line: use S
Replace line with PAD content: use R
Copy the line to PAD: use H
Copy a line to another row: use H; use R
Insert a line: use I
Insert a line: use S; use R (assuming line content is in PAD)

Searching text and set cursor

Finding a string: use F (cursor set at the end of the match)
Finding next occurrence: use N
Setting cursor at the beginning of the match: use F; use B
Setting cursor at a specific point: use M (some calculation is involved)

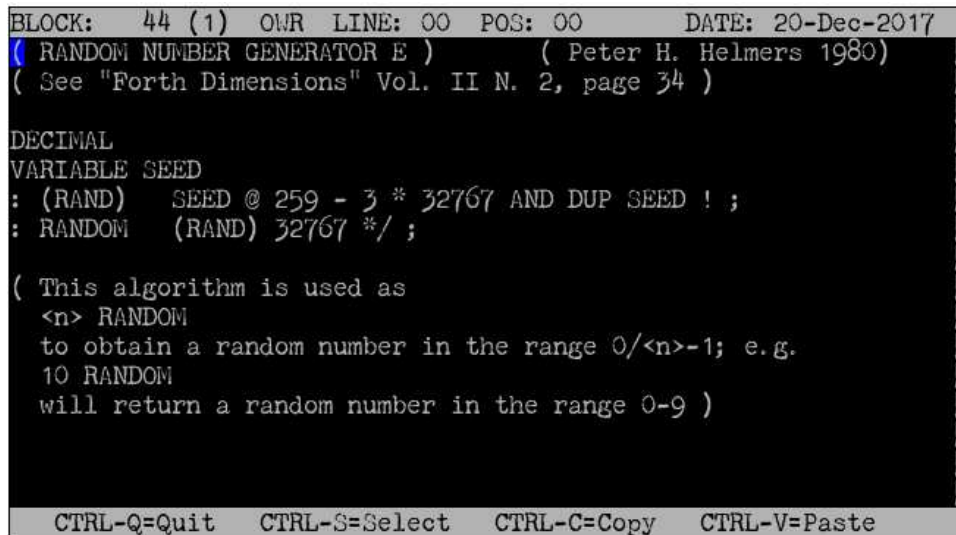
Changing a line content

(Note: these words depend on the cursor position; in case you have to consider the screen from the start, use TOP before any of them)

Delete from cursor to a given pattern: use TILL
Delete pattern: use X
Substitute text: use A old new
Substitute text: use X old; use C new
Substitute text: use X old; use TEXT typing new; use C (no args)

3.5. The Screen Editor in detail

Following some Forth traditions (thanks to old Forth environments like F-PC, a powerful Forth for DOS, which had some very advanced instruments, like for instance the screen editor), and listening to the proper advices of tester **Ian Jones** about such subject, I decided to add a built-in Screen Editor, suitable for the editing of Forth blocks (1024 bytes in 16 rows of 64 characters each), invoked though the word `EDIT`, as in the following figure:



```
BLOCK: 44 (1) OWR LINE: 00 POS: 00 DATE: 20-Dec-2017
( RANDOM NUMBER GENERATOR E ) ( Peter H. Helmers 1980 )
( See "Forth Dimensions" Vol. II N. 2, page 34 )

DECIMAL
VARIABLE SEED
: (RAND) SEED @ 259 - 3 * 32767 AND DUP SEED ! ;
: RANDOM (RAND) 32767 */ ;

( This algorithm is used as
  <n> RANDOM
  to obtain a random number in the range 0/<n>-1; e.g.
  10 RANDOM
  will return a random number in the range 0-9 )

CTRL-Q=Quit CTRL-S=Select CTRL-C=Copy CTRL-V=Paste
```

`EDIT` must find the block number to be edited on top of stack. It works in the same vein of `LOAD`. Don't expect `EDIT` to be a sort of Microsoft Word, or Libreoffice Writer, though. It's a simple tool, with many features, yes, but its most important feature is that it's an editor targeted for Forth.

`EDIT` has two states: the overwrite state (the default, safer mode), and the insertion state, each with obvious scopes. These states can be toggled with `CTRL-J`.

The cursor can go anywhere in the screen but not past the dominion of the 1024 characters. This helps in maintaining correctly dimensioned lines. Moreover, all places where you don't see printable characters are not really empty, but filled with blanks, so the screen **always** contains 1024 bytes: this assures that anytime you decide to save it to the storage device, no less and no more than 1024 bytes are saved. In any case, the editor can load any lines up to 16, with length up to 64 characters, terminated by the End-Of-Line character (in all, less than 1024 bytes), filling the rest with spaces; when you will save this block, it will be correctly set to 1024 bytes.

Warning: since the block is loaded in its entirety, using the last character of a line and the first character of the following line may lead to error in compilation, because the two are seen as contiguous (unless this is intended). So, you can use the last character of a line if the next line is indented, or use the first character of a line if previous line terminates with a blank.

The Screen Editor disables the standard interrupt state, to let using all commands through the `CTRL` key¹⁰⁵ (the standard state is restored when exiting). All the remaining non-printable usual keys do work (I hope for you) as expected, except `TAB`, which acts like `CTRL-I`. The copy command `CTRL-C` uses a private buffer, not shared with the Operating System.¹⁰⁶

¹⁰⁵ Please note that in the Screen Editor `CTRL-Z` does not work here as the shell 'suspend' interrupt.

¹⁰⁶ On emulated terminals, you can do Copy&Paste operations with the mouse; first, moving the mouse, select the portion you want to copy (right-click/Copy); this operation can be done on any window (for instance, from a site page in the browser or the `EDIT` window itself); then you switch to the `EDIT` window, and move to the place where to paste (with the arrows, not the mouse); then you paste (right-click/Paste). The `CTRL-C` and `CTRL-V` are not related with the right-clicks of the Operating System; that is, either you use the mouse, or you use `CTRL-S/CTRL-C/CTRL-V` from keyboard; the two systems don't use the same memory buffer, but the arrows (for positioning) must be used in both cases.

3.5.1. Status lines

The EDIT screen has an upper line and a bottom line (both not editable and in reverse colours) which serve as information centers.

The top line reports the following data:

- BLOCK: block number (the next number in brackets is the buffer number)
- INS/OWR insertion/overwrite state
- LINE: POS: line and position of the cursor (lines are numbered 0..15 and positions 0..63)
- If the block buffer was changed after loading, the string UPD appears following the POS number; UPD disappears when the block is saved or mark as not updated.
- DATE: the current date in a common and unambiguous format: e.g. 10-Nov-2018

The bottom line reports useful data, like a brief help in normal mode, runtime messages and, when asked for input, it turns to a user typing field.¹⁰⁷

Anytime, the least accessed screen is the chosen one for editing, starting from screen #1.

3.5.2. Screen Editor commands

All commands are driven by the CTRL key:

CTRL-A Select all text in the current buffer.

CTRL-B or Page Down

Boot next block. Start editing a new buffer screen (even empty) attributing it the current block number + 1. It may also be used to 'read' several consecutive blocks in the Screen Editor.

CTRL- or Page Up

Boot previous block. Start editing a new buffer screen (even empty) attributing it the current block number - 1. It may also be used to 'read backward' several consecutive blocks in the Screen Editor.

CTRL-C Copy selected text; if no selection was made, nothing is done.

CTRL-D Delete current line, shifting up lower lines; a 15th line is created empty.

CTRL-E Edit another block in the next available screen and set the relative buffer as the current buffer. The block number is asked on the bottom line.

CTRL-F Find a string in the current screen; the matching string is asked on the bottom line. The search starts from the current cursor position. After search, the cursor position is set to the start of the matching string, if found, or remains in the original position if not found (the message "Text not found" appears on the bottom line).

CTRL-G Repeat last search made with CTRL-F, starting search on character after current cursor position (to avoid finding the same last search).

CTRL-H Show a short help screen; ESC to return to the current editing screen. Editing is disabled while the help screen is active.

CTRL-I Insert an empty line, shifting down current and lower lines; 15th line is lost. The TAB key performs the same task, because it's bound to the same CTRL command.

CTRL-J Toggle the insert/overwrite state (overwrite being the default). Labels INS or OWR appear on the top line, to signal the proper active state.

CTRL-K Erase current line filling it with spaces.

¹⁰⁷ During input in the typing field, keep in mind that: for commands that ask for a number, only characters in the range {0..9} can be typed, for at most 5 digits; for commands that ask for a string, only ASCII characters in the range {32..126} can be typed, for at most 50 characters. Of all the other control characters, only BACKSPACE and ENTER are evaluated during the typing: the first deletes last character, the second confirms the whole input.

CTRL-L	Quit EDIT (the Screen Editor) and enter the Line Editor mode (enabling the EDITOR vocabulary and listing current buffer).
CTRL-M	Equivalent to the ENTER key.
CTRL-N	Clear current buffer, setting it as new and not updated. All content is lost. The clearing involves only the memory buffer, not the original file.
CTRL-O	Re-open the block file, reloading all the block with the data on file, and losing all changes applied to the buffer. Use this command with care, because it will lose all your changes, though sometimes this is just the thing to do, when the text gets cluttered and we feel that the changes we've made don't satisfy us any more. Useful also when re-entering EDIT (in this case the buffer is already engaged, so the block is not re-loaded) and we want to re-edit from start.
CTRL-P	Put a default stamp on the first line of the screen (the stamp is the same of the command STAMP of the line editor).
CTRL-Q	Quit the editor and get back to tetra's console, without saving. Watch out! EDIT does not ask you to save an updated screen, so, if saving is your intention, remember to type CTRL-W to save the block, before quitting, or type SAVE-BUFFERS once back to the console.
CTRL-R	Replace the latest string occurrence searched with CTRL-F with a new string; the new string is asked on the bottom line only for the first substitution (in case no search was performed, the match string is also asked, priorly, on the bottom line). The search starts from the current cursor position. After the substitution, the cursor position is set to the start of the substituted string, if found, or remains in the original position if not found (the message "Text not found" appears on the bottom line, and the new string is reset). Multiple substitutions can be applied simply repeating the CTRL-R command.
CTRL-S	Toggle selection state. Selection always starts from current cursor position and is extended forward or backward, upward or downward with the arrow keys.
CTRL-T	Set the cursor to Top, i.e. line 0, position 0 (the upper left corner); if the cursor is already there, put it at line 15, position 63 (the lower right corner).
CTRL-U	Undo last operation. Watch out! There's only one undo level for all commands.
CTRL-V	Paste the selected text to the current cursor position; if no text was copied, nothing is pasted.
CTRL-W	Write the current editing block to disk; other buffers are not touched by this operation. Of course, the same operation can be done cumulatively on all buffers with SAVE-BUFFERS, after getting back to tetra's console (the message "Block file was saved to file system." appears on the bottom line). CTRL-W is not executed after a CTRL-K, which sets current buffer as updated.
CTRL-X	Copy and cut the selected text; if no selection is made, nothing is done.
CTRL-Z	Set current block as not updated, so that a further SAVE-BUFFERS command won't save the block to disk, and physically remove the file from the file system (the message "Block file was removed from file system." appears on the bottom line). It is somehow equivalent to an EMPTY-BUFFERS directed on a single buffer. CTRL-Z disables also a following CTRL-W on the same buffer. Any editing on the buffer will reset this command (the block would be marked updated) and on a subsequent saving (with CTRL-W or with SAVE-BUFFERS), while the file is marked updated, the file is recreated anew.

The ENTER key moves to the beginning of the next line. If ENTER is pressed in insert mode, the text is broken and the part following ENTER (including current cursor position and up to the end of line) is moved on next line created as new (the 15th is lost). In overwrite mode, no breaking is done.

The control Keys available on editing are:

- The arrow keys
- ESC (only for Help Screen exit)
- CANC - delete current character
- HOME - go to the first position of current line

- END - go to the last position of current line
- BACKSPACE - remove current character and shift left

Other control Keys are left to the Operating System (for instance, the Stamp key that captures the screen, present on some OS).

The Screen Editor is still experimental, and I expect some bugs. Please, if you find one, write to me immediately. Thanks.

A note for computers having a keyboard with a dedicated numerical keypad: this pad generally works in two modes: with the block-number ON or with the block-number OFF. If it is ON, the numeric keys emit their numerical value. If it is OFF, the pad works as a directional tool, with keys 8,4,6,2 as arrows, 9,3 are page-up and page-down, 7 is used as page-top, 1 as end-of-line and 0 as INS, with the 5 key disabled. When the Screen Editor is used and the block-number is deactivated, the pad works normally, but some keys (namely 7, 6, 5), if used in combination with the CTRL key, are disabled because they interfere with the editor commands.¹⁰⁸

BTW: there's an Easter egg, in the editor. Can you find it? Peeping into the code is forbidden! ;-)

3.6. fig-Forth Compatibility

The compatibility flag FIG-STATE is a variable that influences the behaviour of **tetra** with respect to the fig-Forth compatibility; if it contains zero, **tetra** behaves as a Forth-79 system; if it contains a number different from zero, in order to make **tetra** behave like fig-Forth, the following words change their behaviour:

- +LOOP (include the lesser limit in case of negative increment)
- 2VARIABLE (use the double number on the stack as starting value)
- ABORT (print an appropriate message, required by fig-Forth)
- FORGET (execute only if CONTEXT is equal to CURRENT)
- I (available also outside a DO . . LOOP and working as R@)
- SIGN (operate a ROT before evaluating the sign)
- VARIABLE (use the number on the stack as a starting value)
- WORD (place the counted string to HERE)

The compatibility does not automatically load the missing fig-Forth words. Block 20000 contains the library reference of words related to fig-Forth; to turn your system in a (quasi) fig-Forth environment, type thus

```
FIG-STATE ON
20000 LOAD
```

and you're done. See also 79-STANDARD.

3.7. Service words

In developing **tetra** I often needed some service words that revealed useful to inspect the memory and the stack, the state and so forth, even in case of unresponsive dictionary (and God knows if I found myself in this situation...). So, to avoid clutching the dictionary with sparse words, I added them as super-immediate not dictionary-based words (because they had to work even when the dictionary was messed up). Some of these would eventually be copied in **tetra**'s dictionary, due to their utility: SH (as SYSTEM), MON (as MONITOR), BYE, COLD and WARM. The remaining were to be deleted, but I ultimately thought that if they were useful to me, they might be useful for you too. So I decided to maintain them, to signal also you cannot name a word with the same characters combination of these (even if I tried to use improbable names), because your word will never be seen. And remember that they are not compilable, they are immediately executed when met, there is no way to use [COMPILE] with them, so they remain pure keyboard console tools.

¹⁰⁸ If you use an emulated terminal on your desktop, maybe you can use the cute commands CTRL/+ and CTRL/- to expand or reduce the terminal fonts. These commands are not intercepted by the Screen Editor.

I expose them here, in the same manner of the rest of the dictionary, but remember that they are not contained in the dictionary and quite certainly they are not portable.

@STAT S TFT

Enable the statistics to be printed at the end of the session. It's equivalent to the option `--statistics` typed at start.

CF@ S TFT

Print the control flow stack signed content in a non-destructive way. In normal usage, it should always show an empty control flow stack. I built it to check when built-in words under development didn't correctly work, leaving some scattered values back.

CHECK-DICT S TFT

Try to fix the database in case it gets damaged. **Watch out!** This command tries to repair the database, starting from the bottom and proceeding upward, by removing the first item which does not match the link addresses references with the previous word; `HERE` and `LAST` are re-addressed in case. This normally implies the loss of some user words (the faulty one and all that follow). But if the fault is found in the built-in section (due to improper memory writing), using `CHECK-DICT` can cause the deletion of some built-in words. So, in case of cluttered and faulty dictionary, use `CHECK-DICT`, but if the system remains unresponsive, type `COLD` and restart.

BYE i*x -- S COM

The service word `BYE` works exactly as its dictionary-based counterpart; when the dictionary is corrupted, typing `BYE` will not activate the normal version, but this Service word, that will perform the termination of `tetra` by printing " (service) " to let you know which version was called. See also the ordinary `BYE` in the dictionary..

COLD i*x -- S COM

The service word `COLD` works exactly as its dictionary-based counterpart.

DS-> -- addr S TFT

Leave the memory position of the next free unit of the top of stack; the last value was stored in `DS-> - 2` and can be checked with:

`DS-> 2- @ .`

I built it to test the stack and the stack-based words without using the `@`-family words. Note that in the Alpha version, this command was named `DS`, but this is a too common name so, to avoid conflicts, starting from the Beta version, I decided to rename it `DS->`, which far less common.

DS@ S TFT

Print the Data Stack unsigned content in a non-destructive way. I built it to check the stack state after a system freeze caused by wrong functions during the implementation phase; since this is a super-immediate word, it needs no dictionary search and so it's likely to work when `.S` cannot. And since it prints unsigned numbers, it could reveal strange unwanted numbers, because compilation codes are always positive.

GET-STATE S TFT

Show the current STATE on the screen, followed by a Carriage Return. It prints 0 or 1, with the meaning that

0 = interpret state and
1 = compile state

This was done because, sometimes, when the system was under construction, I needed a fast way to check the STATE, without interfering with the running system and the stacks.

MON S FIG

Start a shell session. It behaves just like the dictionary word MONITOR, but as a super-immediate word it can start a shell session even if the dictionary is messed up.

PREV-HERE -- addr S TFT

Leave the address of the previous compiled word. Differently from LAST, which returns the address of the last word under compilation, regardless of the compilation success, PREV-HERE returns always the last **successful** compiled address, and it's the same value used internally by tetra to traverse the whole dictionary. It was built when testing the word creation process, to dump memory in a fast way.

RS@ S TFT

Print the Return Stack unsigned content in a non-destructive way. In normal usage, it should always show an empty Return stack. I built it to check the values left by the built-in DO . . LOOP family words, or to check scattered values left behind in all remaining words.

SH S TFT

Used in the form:

SH <name> <text>

to invoke the command <name> (which is any tool in the path or a shell command) and passes it to the shell, along with all arguments <text>. This is the super-immediate word equivalent to CSYSTEM, but it's not programmable.

SH arguments extend until the end of the input buffer, so that it must not be followed by any other Forth word (that will be interpreted as an argument)r.

Remember also that while SH can be written in lower or upper case, the rest of the command is passed as-is, because the shell is case-sensitive.

WARM i*x -- S COM

The service word WARM works exactly as its dictionary-based counterpart.

3.8. The integration file tlibrary.ft

The package includes the file tlibrary.ft, a library file that contains a great number of useful words definitions in Forth (for instance INDEX, S", CLEARSTACK); some of them fill some holes in the Forth-79 word set (for instance <>, ><, m*) and some are ANSI 1994 words in a Forth-79 translation (not always fully respondent to the ANSI 1994 Standard). Some are Forth-83 renditions, but be careful because the latter overwrite the Forth-79 versions.

The library file is copied in the user `$HOME/.tetra/` directory by the procedure `--config` (see the file `INSTALL`).

You can safely load the entire library with `INCLUDE` (the library protects itself from the Forth-83 versions), or you can load only the words you need with `ENSURE` or `REQUIRE`. See `INCLUDE`, `ENSURE` and `REQUIRE`. You can search into the library for word names with `SEARCHLIB`.

All library words, being written in Forth, can be fully disassembled with `SEE`.

A word of caution: all variables in the library, that cannot be loaded with `ENSURE` or `REQUIRE` (because they are built to load only colon words), are built as pseudo variables, with the following mechanism:

- 16-bit

A normal 16 bit variable is compiled as

```
: <name> [ here 2 + ] literal 0 , ;
```

and is equivalent to `variable <name>`.

- 32-bit

A normal 32 bit variable is compiled as

```
: <name> [ here 4 + ] literal 0. , , ;
```

and is equivalent to `2variable <name>`.

These forms are peculiar to `tetra`, and quite surely not directly usable in other Forth systems.

The constants, instead, are simply built storing the constant value in the body:

- 16-bit

A normal 16 bit constant with `val <val>` is compiled as

```
: <name> <val> ;
```

and is equivalent to `<val> constant <name>`.

- 32-bit

A normal 32 bit constant with `val <2val>` is compiled as

```
: <name> <2val> ;
```

and is equivalent to `<2val> 2constant <name>`.

If you have built your own library, you can use it as the current source by setting the library reference with `LIB`. Remember that, to work smoothly, libraries should only have colon words definition, with variables and constants declared as above, but consider that any directly executable code is "seen" only if you include the whole library, not if you `ENSURE` or `REQUIRE` specific words. See also `LIB`.

4. THE BUILT-IN DEBUGGER

tetra has a built-in debugger engine, a simple words-tracking with stack content picturing, enabled with the words `DEBUG ON` and `DEBUG OFF` to enable/disable the debug on the whole program and with the word `TRACE` to enable/disable the debug on a single user word.

Don't expect a very complex debugger. Its aim is to signal where a word fails by printing the flow of procedures involved and the stack content at each step.

4.1. Using `DEBUG`

Suppose for instance that we have a word defined as such:

```
: DE-TEST  ( d1 n1 -- n2 )
  ROT ROT SWAP /
  SWAP / ROT
;
```

(it's a useless word, just to show how the debugger works). Now let's try it:

```
56783344. 567 DE-TEST
? empty stack.
56783344. 567 DE-TEST
^
```

What we know so far is that `DE-TEST` fails. But where? We have no other suggestions by the standard error detection system.

The debugger, when enabled, at each step shows us some more information, namely the Program Counter, the Data Stack in signed mode, along with the name of the word or the number under evaluation; the following shows the entire debug session:

```
DEBUG ON  ok
56783344. 567 DE-TEST
<3> 29168 866 567

PC:03639  Executing built-in word --> rot <--

<3> 866 567 29168

PC:03641  Executing built-in word --> rot <--

<3> 567 29168 866

PC:03643  Executing built-in word --> swap <--

<3> 567 866 29168

PC:03645  Executing built-in word --> / <--

<2> 567 0

PC:03647  Executing built-in word --> swap <--

<2> 0 567
```



```
PC:03649  Executing built-in word --> / <--
<1> 0
PC:03651  Executing built-in word --> rot <--
? empty stack.
56783344. 567 DE-TEST
      ^
```

The problem is on the third ROT command, which finds less than three items on the stack, and thus rightly complains. Of course, at this point, it's up to us programmers review the algorithm and find a solution for this, but at least we now know where the problem lies. Sometimes, this is just the thing to know, isn't it?

4.2. Using TRACE

If you want to select a specific user word for debugging, you can use TRACE; this word will trigger debugging only on the word whose name has been given as argument- Consider for instance the case when previous word DE-TEST is used inside another word:

```
: WHAT    ( -- )
  GET-VALUE DUP IF DE-TEST THEN RESULT
;
```

Enabling debugging with DEBUG and executing WHAT would print the debugging information for all the user words WHAT, DUP, GET-VALUE, DE-TEST and RESULT; if instead you invoke:

```
TRACE DE-TEST
```

before executing WHAT, the debug will be active only during the execution of DE-TEST; this lets inspecting the program to find more exactly where it fails, without the need to traverse all words, ending in a long debugging output, where the useful information must be searched.

If the tracing word has some user words in its body, these words are not traced (that is the debugger does not 'enter' into these) so that, in the assumption that these inner words work regularly, you can concentrate on the word under TRACE (DE-TEST in the example), without bothering for the rest.

To disable the tracing, invoke TRACE alone:

```
TRACE
```

TRACE, when invoked with an argument, sets also DEBUG to false. So, after you disable TRACE, you have to re-enable debugging.

A final remark: TRACE cannot be set on built-in words (variables included); moreover, as you can easily see by yourself, tracing user variables or constant is useless, because they don't perform any operation. I must also observe here that CREATED words always perform the operation of returning their PFA, and this pushing is shown by the debugger; all DOES> operations (if any) are shown afterward.

4.3. Debug Commands

At each step, the debug waits for one keyboard key (lowercase or uppercase are the same, no ENTER key):

C - Continue in debug mode

The program continues in debug mode (the trace-point is not removed), but without steps interruptions, showing the debug lines until the natural end.

D - Data stack

The Data Stack is printed (in the same style of `.S`) with unsigned values. Useful for inspection of addresses in the stack. The stack is identified with `DU:` (for Data Unsigned).

E - Execute user code

The system suspends the current word and requests a line of text (the string `(suspended)` appears, the prompt becomes `(D)`); the input text is evaluated by the interpreter as Forth code (by calling the line interpreter in debug-off mode); this feature can be used to inspect variables content, modify the current stack or the system state, `DUMP` memory or change the program behaviour during runtime, or any other legal Forth operation¹⁰⁹. After executing the user code, the user can type another command if needed; in case the command is empty (the void string), the string `(resumed)` appears, the debugger resumes control, restoring the system parameters and the previous input line, waiting for a key press to execute the suspended word or another debug command.

Warning: Please be aware that typing values or removing values on the stacks can alter the debug. Do that only if you are sure it's the right thing to do.

F - Control flow stack

The Control Flow Stack is printed (in the same style of `.S`) with signed values. The stack is identified with `CF:` (for Control Flow).

G - Exit debug and go on

The trace-point is removed, the `DEBUG` flag is disabled and the program continues in normal off-debug mode, until its natural end. The debug is reset, so if you want to debug again, re-enable it with `TRACE` or `DEBUG ON`.

H - Help

A brief help is shown with the titles of this help. Refer to this manual for more info.

P - Print the T.I.B.

The content of the Terminal Input Buffer is printed as string to terminal.

R - Return stack

The Return Stack is printed (in the same style of `.S`) with unsigned values; in case of `DO...LOOP` indices, the negative values are shown as unsigned, so be aware of this. The stack is identified with `RU:` (for Return Unsigned).

X - Stop and exit debug

The program and the debugger stop; the trace-point is not erased and the `DEBUG` flag is maintained, for a subsequent debug session. The string `(terminated)` appears on the screen.

Any other key gets to the next step.

¹⁰⁹ Debugging uses its own circuit to get the input commands; therefore, if you invoke `BYE` as the user code, the system won't exit until the next iteration resumes. Using `COLD` as the user code is instead legal, since debugging or tracing are disabled.

5. DIFFERENCES BETWEEN **tetra** AND THE FORTH-79 STANDARD

Here are collected all the differences between **tetra** and the official Forth-79 Standard, split in two categories: the changes (that modify the Standard behaviour) and the enhancements (that try to surpass the Standard limits). Let's begin.

5.1. Changes

5.1.1. Word sets

tetra has the following word sets hardwired as built-in:

- The **DOUBLE** word set (defined in the Forth-79 standard as Double Number word set in the Extension Word Sets chapter - page 32) is implemented by default and belongs to **FORTH**'s vocabulary.
- The **EDITOR** vocabulary is the only other system vocabulary added, at present.

The **ASSEMBLER** word set (defined in the Forth-79 standard as Assembler word set in the Extension Word Sets chapter - page 34) is currently not present.

5.1.2. **-->**

This word does not belong to the core set of the Forth-79 Standard; it belongs to the Reference Word Set, where it's stated that it *"May be used within a colon definition that crosses a block boundary"* (i.e. the colon is on one block, the semicolon on the next); this is not true for **tetra**: any definition must be accomplished into the block, and thus the effect of **-->** is merely 'append' the next block.

Moreover the Standard does not specify what should happen in case the block does not exist or is not found. I chose to ignore the error condition, after the returning of a block filled with blanks by **BLOCK**, which means the compilation will simply stop, and this may mean another difference, if not with the Standard, with many Forth-79 compilers.

5.1.3. **BASE**

The Forth-79 Standard specifies the limit of {2..70} for the available bases for numbers. **tetra**, in a more modern feeling (and in the assumption that this is enough) limits this range to {2..36}, adhering thus to the modern languages range, that has been proved to be adequate and robust.

5.1.4. **BLOCK** and **BUFFER**

The Forth-79 Standard specifies for **BLOCK** that *"...if correct mass storage read or write is not possible, an error condition exists"*, without specifying if this error condition is originated by the absence of the file relative to that block, or by a disk error or what. As a result, I built **BLOCK** to return a full empty block (filled with blanks) if the relative file can't be loaded (because it does not exist or because you haven't the access rights); the problem reported in the Standard arises when the *saving* of the block is not possible: whence the error condition. Besides, in **tetra** **BLOCK** and **BUFFER** use an unsigned argument, to let the user treat up to 65535 block files (excluding zero).

5.1.5. **CONTEXT**

The definition of **CONTEXT** in the Standard is: *"Leave the address of a variable specifying the vocabulary in which dictionary searches are to be made, during interpretation of the input stream."*. **fig-Forth** defined both **CONTEXT** and **CURRENT** to be pointers to the vocabulary word within which dictionary search will first begin, and this, somehow, has set the behaviour of the following Forth-79 compilers. In **tetra**, **CONTEXT** returns an address of the dictionary point where a control-bit byte is stored, specifying a given vocabulary; this value is checked, in runtime, against a given word's definition, to see if it matches the dictionary control-bit value, in which case it is executed. Any dictionary has linked the father-dictionary address, to extend the search in case. This is a feature, more than a difference with the Standard, but this does not prevent the usage of **CONTEXT @ @** to return the reference of the context vocabulary.

5.1.6. CURRENT

The definition of CURRENT in the Standard is: "*Leave the address of a variable specifying the vocabulary into which new word definitions are to be entered.*". In **tetra**, CURRENT returns an address of the dictionary point where a control-bit byte specifying a given vocabulary is stored; this value is put into the definition being created in the dictionary control-bit, value that will be checked in runtime to see if it responds to the CONTEXT control-bit value, in which case the word is executed. This is a feature, more than a difference with the Standard, but this does not prevent the usage of CURRENT @ @ to return the reference of the current vocabulary.

5.1.7. DPL

In **tetra**, DPL is affected by the user during a number input, *à-la* fig-Forth. This differs from the definition of DPL in the Forth-79 Reference Word Section, where it's stated "...DPL may be set explicitly, or by certain output words, but is unaffected by number input"¹¹⁰.

5.1.8. FORGET

In **tetra**, FORGET does not complain if the word to be forgotten is not found, doing nothing in return. The error instead is explicitly ruled in the Standard, where it's stated "*Failure to find <name> in CURRENT or FORTH is an error condition*". However, the imposed error condition in the Standard prevents its usage as an automatic tool. See the note for the FORGET entry in this manual.

5.1.9. LITERAL and DLITERAL

The Standard specifies for LITERAL the compilation behaviour without specifying any execution behaviour, but also without setting LITERAL as compile-only (a feature later enabled in Forth-83). So I turned to the Forth-83 scheme, which seems smarter, and enabled LITERAL as compile only, with the consideration that typing from console, e.g., 24 LITERAL, is meaningless. The same for DLITERAL.

5.1.10. PAGE

In the Reference Word Set (page 8), PAGE is set to "*clear the terminal screen or perform an action suitable to the output device currently active*". In **tetra**, PAGE acts always on the terminal screen.

5.1.11. WORD

The Standard requires WORD to ignore leading occurrences of the terminator character. **tetra**, instead, ignores the terminator only when it's a blank, letting treating the empty string case. See the WORD entry in the dictionary.

This difference with the Forth-79 Standard is confirmed by what exposed by the Rationale of the PARSE word at page <https://forth-standard.org/standard/core/PARSE>. The example herein contained reports what follows:

"The traditional Forth word for parsing is WORD. PARSE solves the following problems with WORD:

1. WORD always skips leading delimiters. This behavior is appropriate for use by the text interpreter, which looks for sequences of non-blank characters, but is inappropriate for use by words like (, . (, and .". Consider the following (flawed) definition of . (:

```
: . ( [CHAR] ) WORD COUNT TYPE ; IMMEDIATE
```

This works fine when used in a line like:

```
. ( HELLO)      5 .
```

but consider what happens if the user enters an empty string:

```
. ( )          5 .
```

¹¹⁰ I still have to understand why, since it's so useful in numerical input...

The definition of . (shown above would treat the) as a leading delimiter; skip it, and continue consuming characters until it located another) that followed a non-) character; or until the parse area was empty. In the example shown, the 5 . would be treated as part of the string to be printed".

In **tetra**, **WORD** does not feature the behaviour exposed in the Rationale, because the terminator always terminates the string reading, and is never skipped. This adheres to the indications in *Forth Dimensions* Vol. III n. 3, in the "Forth Standards Corner" by Robert L. Smith. This does not prevent the usage of **WORD** in normal Forth-79 procedures, and since in modern Linux the terminator is usually a printable char, or at least any character that belongs to the ASCII table (and so always greater than zero), this is anyway a minor and negligible difference. Anyway, for strict adherence to the Forth-79 Standard, if needed, set **STRICT-STATE** to a non-zero value, and **tetra** will behave as prescribed in the Standard for what about **WORD**'s behaviour.

5.2. Enhancements

5.2.1. Memory

The Standard requires memory to be organized in single bytes (i.e. 8 bits) for characters, in one *cell* (i.e. two bytes, or 16 bits) for single numbers and two *cells* (i.e. four bytes, or 32 bits) for double numbers.

tetra uses the same number structure, but uses 32-bit numbers (corresponding to the C `int` type) for the basic memory **unit** (the Forth-79 byte), replicating the Forth-79 memory model (i.e. one single *unit* for characters, two *units* for 16-bit numbers and four *units* for the double numbers), using for each unit only the lower 8 bits¹¹¹ (thus, a *unit* corresponds to a byte). The full 32-bit unit capacity is reserved for inner addresses and specific markers used in compilation.

The starting address of a word definition in the dictionary is not a 'cell' (two units) but a full 32-bit number. This is done for speed reasons. The interpreter (and mainly **EXECUTE** and all the built-in words) skips the bytes before the Compilation address (a fixed width) and knows what to do within each instruction. If you want to read this cell address, `@` and `2@` are of no use. **tetra**'s programmer can scan the memory with **DUMP**, and interact with the special 32-bit numbers by means of the elective words `32@ 32,` and `32!`, with double numbers arguments, for storing and reading this extra-bit values.

5.2.2. The (comment

The (comment has a feature that the Standard does not require: if the closing parenthesis `)` is not followed by a space, a TAB or by the end-of-line, it is not considered as a closing parenthesis, and the comment continues. See the note for the (entry in this manual.

5.2.3. CREATE

In **tetra**, **CREATE** does not simply create a dictionary entry. Rather, it reserves a coded sequence that is used by **FIND** and `'` (tick) to distinguish the CFA from the PFA of a **CREATED** word. Moreover, the first byte after the end of the coded sequence is reserved. The PFA starts always at the second byte.

These are not really differences, they're more like implementation features, but in any case, here they are.

5.2.4. DOES>

The **DOES>** instructions inserted in a defining word are inherited by the defined word, and this is the normal usage of **CREATE**. `..DOES>`; but **tetra** has a property that the Standard does not specify or clarify: if the defining word with a **DOES>** section is used as a defining word itself in another defining word, and this second defining word is used to define a word without **DOES>**, this last word inherits the **DOES>** section of the original defining word; an example is thousand words:

```
: ARRAY    CREATE    DOES>    2+ @ ;
```

¹¹¹ I know that this may result, at a first glance, in a great waste of memory, but in a 64-bit Operating System (and maybe more significantly in a 32-bit one) the execution of 32-bit numbers is faster than using the `short` or `char` C-types.

```
: NEWARRAY    ARRAY ;
NEWARRAY MYLIST
SEE MYLIST
: MYLIST
    4518 2+ @
;
ok
```

In this example, ARRAY is the original defining word with a DOES> section, NEWARRAY is a defining word without a DOES> part, but defined through ARRAY; MYLIST is defined by means of NEWARRAY, and inherits the ARRAY DOES> section, because NEWARRAY has none, as can be seen inspecting MYLIST. IT seems the Gforth (and thus ANSI) behaves exactly like this.

Another different feature is that DOES>, as prescribed by the Standard:

```
Define the run-time action of a word created by a high-level
defining word. Used in the form:
    : <name> ... CREATE ... DOES> ... ;
    and then <namex> <name>
Marks the termination of the defining part of the defining
word <name> and begins the defining of the run-time action for
words that will later be defined by <name>. On execution of
<namex> the sequence of words between DOES> and ; are
executed, with the address of <namex>'s parameter field on the
stack.
```

tetra does not leave the address of <namex>'s parameter field on the stack, but rather copies the DOES> part in the compilation address of the created word. By what is written at page https://gforth.org/manual/DOES_003e.html of the Gforth manual, *"In fig-Forth the code field points directly to the dodoes and the DOES>-code address is stored in the cell after the code address (i.e. at CFA cell+). It may seem that this solution is illegal in the Forth-79 and all later standards, because in fig-Forth this address lies in the body (which is illegal in these standards). However, by making the code field larger for all words this solution becomes legal again. We use this approach. Leaving a cell unused in most words is a bit wasteful, but on the machines we are targeting this is hardly a problem"*.

As results from the previous paragraph, the Gforth's approach is different: to avoid the illegal reference, its creators prefer adding an unused cell for each word. My approach, though naive, ensures that no Standard violation is committed, at the cost of not leaving the address of the parameter field of <name> on the stack, presumably for EXECUTE; in tetra this is not necessary at all.

5.2.5. EMIT

In tetra, EMIT uses the full {0..255} range for output, while the standard specified the {0..127} range, though most of the characters beyond 127 may result not printable.

5.2.6. TYPE

In tetra, when TYPE is used to print n characters from a specific address and if a zero occurs in the characters queue, it stops printing the rest, even if not all n characters have been output and the rest is not entirely null. In the standard, there's no mention about this feature, but since all strings are terminated by zero, I guess all Forth-79 interpreters did the same, because printing the rest of the string could ruin the output with random characters. Moreover, TYPE does not limit the output to the string length of 255 characters; this lets printing a whole block (1024 characters) or a complete textual line read from a file (whose length may be not predefined).

5.2.7. WORD

tetra dedicates to WORD a fixed and independent buffer, 1024-characters long. In the Standard it's not clear whether this should be independent and fixed, or variable in dimensions and positions (like PAD, for instance), even in *Forth Dimensions* Vol. III n. 5, Robert L. Smith discusses this topic asserting that placing the WORD string at HERE is non-standard (it was a fig-Forth feature). In any case, this feature lets the programmer use WORD safely because its memory space does not belong to dictionary memory.

This solves a number of problems. Consider what's written in the Apple Forth-79 manual:

"Now consider

```
" CAT" " DOG" S= . 1 OK
```

What happened? Since all string literals are created in the same temporary location, the string "DOG" replaced the string "CAT". The command S= subsequently compared "DOG" with "DOG" and found them equal!"

This cannot happen in **tetra**, unless you need to compare more than four objects at one time! But even in this case, the solution adopted by the Apple Forth-79 creators was to store objects in variables, to save their content. This is anyway a very good programming tip.

6. SOME DISCUSSIONS ABOUT Forth-79

In the following I present some discussions about some Forth-79 requirements and how **tetra** behaves, with all the considerations that lie behind the design. I thank Robert L. Smith for his illuminating articles about some Forth peculiarities, that were useful during the design phase of **tetra**.

6.1. A word about **FIND** and **'** (tick)

The two words **FIND** and **'** (tick) have in Forth-79 similar but subtly different scopes. While **FIND** retrieves, from the input stream, the Compilation Address of a word (in **tetra**'s jargon this means the Code Field Address), that can be conveniently used by **EXECUTE** in a vectored execution, the word **'** (tick) retrieves, from the program text, the Parameter Field Address, and thus can be conveniently used to inspect a variable's content, a word's compiled code or data of a vocabulary.¹¹²

In **tetra**'s dictionary, the CFA and PFA coincide only for colon words; for all variables, constants, vocabularies and **CREATED** entities, CFA and PFA addresses differ. To be sure to use the correct data, use always **'** for PFA and **FIND** for CFA, even if they might return the same value.

The words **CFA**, **LFA**, **PFA**, **NFA** (which can be found in the **tlibrary**), belonging to the fig-Forth tradition, can be **ENSURED** and used to convert one address to another, in particular:

CFA converts **PFA** to **CFA** (may result, as said, a no-op)
LFA converts **PFA** to **LFA**
NFA converts **PFA** to **NFA**
PFA converts **NFA** to **PFA**

The distance between **LFA** and **NFA** is fixed and equal to 4.

Read also the interesting article by Robert L. Smith, titled "*Compilation Addresses and Parameter Fields*" in "*Forth Dimensions*" Vol. IV, N. 6 March/April 1983.

6.2. **DO . . LOOP**/**+LOOP** problems

In his article "*Forth Standard Corner - DO, LOOP, and +LOOP*", on *Forth Dimensions* vol. III N. 6, Robert L. Smith describes some problems that arise in the Forth-79 Standard definition of the **DO . . LOOP** and **DO . . +LOOP** cycles.

The first problem deals with memory addressing; since **DO** cycles use signed values as indices, when these indices are used as memory slots, the memory addressing is not guaranteed, because in the Standard there's no requirement about this. The problem posed by Mr. Smith is a serious one, and in the following versions of Forth the **DO . . LOOP** cycles were defined to treat unsigned values.

The second problem posed by Mr. Smith is when the increment of **+LOOP** is zero, i.e. $n=0$; in this case, since there is neither an increase nor a decrease in n , the cycle is truly infinite.

In this case, what happens in Forth-79 is that in any direction the cycle goes, the second iteration is beyond the limit, and thus the cycle is performed only once. This is coherent with what established in the Forth-83 Standard, where it's written for **DO**: "*The loop is always executed at least once. For example: `w DUP DO . . . LOOP` executes 65,536 times.*". The same is not guaranteed in the Forth-79 Standard, where this feature is not contemplated.

One third problem, reading among the lines, is that Forth traditionally does not assure against counter-intuitive cycles, that have the increase or decrease value that goes against the direction established by the two initial values. For

¹¹² The Forth-83 and later changed these things: the Forth-79 **'** (tick) was renamed **[']** (bracket-tick) and constrained to compilation only, to retrieve the word name from the program text; the **'** (tick) was retained to retrieve the name from input stream (like Forth-79 **FIND**). **FIND** was changed to retrieve the word name from a counted string. In Forth-83 and later, **'**, **[']** and **FIND** all return an execution token (that is a Code Address). The word **>BODY** was added to convert an execution token to the start of the parameter field. Well, I think Forth-79 had a neater design. My opinion, though.

example:

```
: TEST1 11 0 DO ... -2 +LOOP ;  
: TEST2 0 11 DO ... +2 +LOOP ;
```

The word TEST1 has a normal cycle supposed to go from 0 to 10, but the index is negative, so that starting from 0, the index decreases continually and will not reach 11. Anyway, more sooner than later, the index will become -32768, and the next subtraction will lead to 32766, then 32764 and so on until it reaches 11 from the opposite side, and stops. For what about TEST2, it's supposed to run backward from 11 to 0, but with a positive index, so that the index will increase and never reach 0. Again, the index will reach 32765 and then stop, because the index plus +2 and plus +1, to test if beyond the cycle, produces 32768 (which is negative for signed 16 bits values, and is seen as -32768)¹¹³.

6.3. LEAVE a good memory of you

In his article "*Forth-79 Compatible LEAVE for FORTH-83 DO..LOOPS*", Klaxon Suralis writes that the LEAVE in Forth-79 is (I quote) the "*horror of horrors*" because LOOP and +LOOP work with signed values. It's indisputable that working with signed values arises some problems, but defining this an "*horror*" seems quite exaggerated. Anyway, his anger is directed to the then-to-come Forth-83 LEAVE version, which is defined in his own words as "*intolerable*".

Probably there is some humour in this article that I, as an Italian, cannot fully understand. In any case, my personal point of view is that the LEAVE word in Forth-79 may be bettered, yes, but far from being an horror. It's anyway true that LEAVE, as designed in the Forth-79 Standard, does not get rid of the loop parameters, but rather it changes them as to *deceive* the terminator LOOP and force it to consider the cycle ended. The Standard itself suggests that EXIT cannot be used within a DO..LOOP cycle, since there is no way to get rid of the loop parameters (usually stored in the Return Stack) with the available words.

6.4. The three musketeers (weren't they four?)

The control flow of a Forth-79 program is not limited to the classic words IF..ELSE..THEN, DO..LOOP, BEGIN..UNTIL and BEGIN..WHILE..REPEAT. They are joined by EXIT, QUIT, ABORT and INTERPRET. These four musketeers play a crucial role in the control flow of a word's execution.

But this is not evident at all. Let's see how.

6.4.1. The three musketeers

Consider the program below.

```
: AB1 ( n -- )  
  DUP 1000 > IF EXIT THEN  
  DUP 100 > IF QUIT EXIT THEN  
  DUP 10 > IF ABORT EXIT THEN  
;
```

- Suppose we invoke 1001 AB1; the first condition is true, so EXIT is invoked; in this case AB1 is immediately terminated, and the control steps back to the system. The stack contains the value that was DUPped (1001 in this case).
- Suppose now that we invoke 101 AB1; now the second condition is true, so QUIT is invoked, that returns control to the console immediately; the ok is not printed and the stack is preserved (QUIT resets only the Return Stack).

¹¹³ I admit that in quartus, the Alpha version of this program, the DO..+LOOP was treated differently, and was built as to avoid the infinite cycles, with a safe guard against the null increment. But in this Beta version I decided to give the programmer the maximum freedom in using cycles, finite or not, being in so more adherent to the Standard. I must here thank Ian Jones, that helped me to understand the nature of the DO..+LOOP cycle, so I could redesign the whole algorithm.

- Suppose now that we invoke `11 AB1`; now the third condition is true: so `ABORT` is invoked, that also immediately returns control to the console, terminating all invocations, but also resets the stacks and sets `>IN` to zero. Since `ABORT` calls `QUIT`, the `ok` is not printed; all stacks are reset, so that the value inspection is impossible.
- Suppose in the end that we invoke `1 AB1`; now no condition is true, so that `AB1` terminates regularly; the control is passed back to the system. All proceeds regularly, the `ok` is printed and all stacks are available.

Thus, `EXIT` stops one interrupt level, while `QUIT` and `ABORT` (with different intensity) stop all interrupt levels; while `EXIT` can terminate the current word, `QUIT` and `ABORT` terminate the program, i.e. the first invoked word (the caller not in the example), no matter how many levels are there. `QUIT` stops without further messages, resetting the Return Stack (and thus deleting `DO . . LOOP` data) awaiting for another input. `ABORT` does the same, but also resets all the stacks and sets `>IN` to zero. The next input will populate the terminal's input buffer and a new request will then be processed.

The action of `EXIT` does not clean the stacks. For this reason it's not advisable to use `EXIT` in `DO . . LOOP` cycles (as the Forth-79 standard states); the cumulative effect of using `EXIT` in loops would leave the Return Stack intact, with unpredictable results in case the interrupt falls back into another outer loop, which would be fooled by the values left behind in the Return Stack! The ANSI Forth solves the problem by adding another word: `UNLOOP` (encoded in the library for `tetra`, if you ever need it). The word `UNLOOP` removes the parameters left on the return stack from the current loop; therefore, after `UNLOOP`, only `EXIT` is allowed, since any other choice would risk returning to the loop, this time without the correct values on the Return Stack.

6.4.2. The musketeer that wasn't there

The word `INTERPRET` is the fourth musketeer. It has nothing to do with stops; but its usage is peculiar. Why? It interprets the Terminal Input Buffer, and thus any word that invokes `INTERPRET` would destroy its own buffer! Thus, to avoid problems, nothing can follow this command; so the following are valid inputs:

```
: TEST1 ..... IF INTERPRET ELSE EXIT THEN ;
: TEST2 .... INTERPRET ;
```

while the following is syntactically valid, but some parts will never execute:

```
: TEST3 ... INTERPRET ... FOO BAR ;
```

In case you have to interpret some text, you should (e.g. see for example the `test-tools-79.ft` file in the package):

- Copy the text to be evaluated somewhere, in the memory (see the `T{` instruction. at start):

```
...
test-loc wlen 0 fill           \ erase test part memory
ascii | word dup c@ 1+ test-loc swap cmove \ copy test command
...
```

- Predispose a word that loads the text into `TIB` and calls `INTERPRET` - and does only this - (see `PARSE-TEST` and `PARSE-RESULT`):

```
: PARSE-TEST
...
test-loc count tib swap cmove
0 tib test-loc c@ + c!
interpret
;
```

- Predispose a word that calls the evaluator of the previous point and proceeds evaluating the results of the interpretation (again `T{`, central part):

```
...  
PARSE-TEST  
...
```

- Then proceed to analyse the results (again T{, final part).

The word INTERPRET is not officially part of the Forth-79 Standard, but it's contained in the Reserved Word Set. This is weird, because it's the main workhorse for the Forth engine, the responsible of the evaluation of the input typed by the user or passed to TIB by BLOCK, INCLUDE or INCLUDED or passed to the main program from the command line¹¹⁴, so that anyone could think that it should be one of the first to be part of the Standard, if not the absolute first.

Anyway, many Forth-79 system (if not all) had this word compiled in the main dictionary; it's utility is evident, and there is no other word that perform text evaluation¹¹⁵. No Forth engine (either Standard) can exist without this word, in one shape or another. It could be missing only if you exclude the design of programs that analyse Forth input, but this is something that any Forth programmer must face, sooner or later, so that such a word is absolutely necessary.

6.5. Using BLOCK and BUFFER

The default Forth-79 way to store information (Forth was born as a file-driven environment) is the blocks set. A block is a contiguous space of 1024 bytes (characters) capable of storing anything in textual form (programs, ideas, your pasta recipe, your girlfriends' cellphone numbers, texts and even books).

The 1024 quantity was chosen for three reasons: the first is that it's a power of two, and this has some fascination; the second is that it's the power of two closer to 1000, which is another fascinating number; the third is that it can be split on a screen and shown (or edited) in 16 lines of 64 characters, which seems a very reasonable screen space for humans facing a monitor.

The words that govern the block system are BLOCK, BUFFER (Forth-79) and USE (specific of tetra); let's see them in detail.

6.5.1. BLOCK

The Rationale of BLOCK in the Forth-79 Standard is:

```
BLOCK          n -- addr          191  
Leave the address of the first byte in block n.  If the block  
is not already in memory, it is transferred from mass storage  
into whichever memory buffer has been least recently accessed.  
If the block occupying that buffer has been UPDATED (i.e.  
modified), it is rewritten onto mass storage before block n is  
read into the buffer.  n is an unsigned number.  If correct  
mass storage read or write is not possible, an error condition  
exists.  Only data within the latest block referenced by BLOCK  
is valid by byte address, due to sharing of the block buffers.
```

The Rationale if the ANSI BLOCK is:

¹¹⁴ To be true, in tetra, it's not that INTERPRET is the workhorse; the workhorse is a low-level procedure, coded in C, that is called by the Forth engine; the same low-level procedure is invoked by INTERPRET, so that it executes the same routine as the Forth engine. The fact that it was put in the Reserved Word Set means that this was contemplated by the Forth-79 Committee. These are subtleties, I know, you can continue to have the sleep of the righteous...

¹¹⁵ It's so important that the ANSI Forth introduced the EVALUATE procedure, whose Rationale is: "(i * x c-addr u -- j * x)
Save the current input source specification. Store minus-one (-1) in SOURCE-ID if it is present. Make the string described by c-addr and u
both the input source and input buffer, set >IN to zero, and interpret. When the parse area is empty, restore the prior input source specification.
Other stack effects are due to the words EVALUATED". This procedure is much more complete than INTERPRET, and the preservation of the input source and the index lets evaluate the string in the Terminal Input Buffer and restore the line afterwards, so that the previous concepts become redundant!

7.6.1.0800 BLOCK (u -- a-addr)

a-addr is the address of the first character of the block buffer assigned to mass-storage block u. An ambiguous condition exists if u is not an available block number.

If block u is already in a block buffer, a-addr is the address of that block buffer.

If block u is not already in memory and there is an unassigned block buffer, transfer block u from mass storage to an unassigned block buffer. a-addr is the address of that block buffer. If block u is not already in memory and there are no unassigned block buffers, unassign a block buffer. If the block in that buffer has been UPDATED, transfer the block to mass storage and transfer block u from mass storage into that buffer. a-addr is the address of that block buffer.

At the conclusion of the operation, the block buffer pointed to by a-addr is the current block buffer and is assigned to u.

The command in the ANSI Forth is quite identical to the one of the Forth-79 Standard. Here follows the *tetra* Rationale:

BLOCK needs a block identifier (an unsigned integer n) on top of the stack. If n is not an available block number (the only case in *tetra* is if $n = 0$) an error is generated. The returned value is addr, the address of the first character of the memory screen assigned to mass-storage block n.

If block n is already in a memory screen, addr is the address of that block screen.

If block n is not already in memory and there is an unassigned memory screen, BLOCK transfers block n from mass storage to an unassigned memory screen, and addr is the address of that memory screen.

If block n is not already in memory and there are no unassigned block screens, BLOCK assigns the least accessed memory screen. If the block in that screen has been UPDATED, it transfers the block to mass storage and transfers block n from mass storage into that screen, and addr is the address of that block screen.

After the operation, the block screen pointed to by addr is the current memory screen and is assigned to n.

6.5.2. BUFFER

The Rationale of BUFFER in the Forth-79 Standard is:

```
BUFFER          n -- addr          130
  Obtain the next block buffer, assigning it to block n. The
  block is not read from mass storage. If the previous
  contents of the buffer has been marked as UPDATED, it is
  written to mass storage. If correct writing to mass storage
  is not possible, an error condition exists. The address left
  is the first byte within the buffer for data storage. n is an
  unsigned number.
```

The Rationale of the ANSI BUFFER is:

7.6.1.0820 BUFFER (u -- a-addr)

a-addr is the address of the first character of the block buffer assigned to block u.

The contents of the block are unspecified. An ambiguous condition exists if u is not

an available block number. If block u is already in a block buffer, a-addr is the

address of that block buffer. If block u is not already in memory and there is an

unassigned buffer, a-addr is the address of that block buffer. If block u is not

already in memory and there are no unassigned block buffers, unassign a block buffer.

If the block in that buffer has been UPDATED, transfer the block to mass storage.

a-addr is the address of that block buffer.

At the conclusion of the operation, the block buffer pointed to by `a-addr` is the current block buffer and is assigned to `u`.

Again, the command in the ANSI Forth is quite identical to the one of the Forth-79 Standard. Here follows the *tetra* Rationale:

`BUFFER` needs a block identifier (an unsigned integer `n`) on top of the stack. If `n` is not an available block number (the only case in *tetra* is if `n = 0`) an error is generated. The returned value is `addr`, the address of the first character of the block screen assigned to mass-storage block `n`.

If block `n` is already in a block screen, `addr` is the address of that block screen.

If block `n` is not already in memory and there is an unassigned block screen, `BUFFER` sets `addr` as the address of that block screen.

If block `n` is not already in memory and there are no unassigned block screens, `BUFFER` assigns the least accessed block screen. If the block in that screen has been `UPDATED`, it transfers the block to mass storage, and `addr` is the address of that block screen.

After the operation, the block screen pointed to by `addr` is the current block screen and is assigned to `n`.

My personal belief is that the only practical difference between `BLOCK` and `BUFFER` in Forth-79 is that `BLOCK` loads the block, while `BUFFER` does not. Anyway, a statistic based on *Forth Dimensions* (made informally by me) shows that `BUFFER` is seldom used, if never. Nonetheless, it's a Standard word.

As a last consideration, read what GForth explains about `BUFFER`: "[...] *The Standard Forth definition of `buffer` is intended not to cause disk I/O; if the data associated with the particular block is already stored in a block buffer due to an earlier `block` command, `buffer` will return that block buffer and the existing contents of the block will be available. Otherwise, `buffer` will simply assign a new, empty block buffer for the block*".

6.5.3. Choosing specific blocks directories with `USE`

The system blocks are by default contained in `$HOME/.tetra/blocks/`, which is created by the first program execution and populated by the `--config` option (see the file `INSTALL`); if you want to use another directory, you can type

```
USE mydir
```

If the name is a relative one, the directory `mydir` will be created in the current directory (the one where you are) if not existing. You can specify also an absolute path for it:

```
USE $HOME/data/tetra/proj1
```

All new blocks created with `EDIT` or loaded with `LOAD` or searched through `SEARCH` will use the specified directory from now on. The other directories will maintain their contents so that you can create one directory for each project (or group of projects) and simply pass from one to the other by calling `USE` on each.

Calling `USE` alone, without arguments, simply resets the system directory `$HOME/.tetra/blocks/`:

```
USE
```

6.6. Compiling issues

There are some issues in compilation that any Forth programmer sooner or later must face during the learning of this wonderful language. I present here some of the most relevant questions that I detected, and show how *tetra* behaves with those.

A foreword: in the following paragraphs, all capitalized text is supposed to run on an ANSI engine (e.g. GForth), while the lowercase text is supposed to run on tetra. Any Forth system is case insensitive, so this differentiation is only to separate, in this manual, the Forth-79 code from the ANSI code.

6.6.1. Compiling tricks

During compilation, the words `[` and `]` disable/re-enable the compilation state, so that in some case the compiler can be deceived. Let's see some.

- 1) The first case is somehow amusing:

```
: TEST [ 42 ] ;
```

This is a famous case, because someone¹¹⁶ said "[This] is not a Standard program". The total effect expected by the previous line is to define `TEST` as an empty word, leaving 42 on Top of Stack. `tetra` and `SwiftForth` accept it, while `Gforth` does not. Why?

The answer is in the *colon-sys* technique, that most if not all Forth system adopt: when the colon `:` is executed, a marker is left somewhere (called *colon-sys*); when the semicolon `;` is executed, the marker is removed.

- A In case the *colon-sys* is saved to the Data Stack, the previous line leaves 42 on top of the *colon-sys*, so that the semicolon makes the whole line fail, because it does not find the *colon-sys*; this is the case of `Gforth`.
- B If instead the marker is saved elsewhere (in some other stacks or to an independent memory), the 42 does not interfere, and so the previous line succeeds. This is the case of `SwiftForth` and `tetra`.

For the sake of precision, `tetra` uses a single flag, that is activated by the colon and deactivated by the semicolon.

- 2) For what stated in the previous point, the colon-semicolon words cannot be nested. This could be performed as this (it seems a trick and it is indeed):

```
: FOO [ : BAR ; ] BAR ;
```

The intended result is to define `BAR` within the definition of `FOO`, just before the `BAR` invocation. This may cause any sort of damage to the system, because the two initialized functions may interfere with each other, leaving the dictionary memory in a dire state. The *colon-sys* mechanism described in the previous point should prevent the execution of this trick, and I'm proud to write here that `tetra` and `SwiftForth` reject this line, because the colon for `BAR` when executed, finds the *colon-sys* left by the colon for `FOO`, and makes the whole line fail; `Gforth`, apparently, accepts the compilation of this line, but then `FOO` cannot execute, and `BAR` does not exist at all...

6.6.2. Compilation and Immediacy

In their wonderful (ANSI-driven manual) "*Forth Programmer's Handbook*", Elizabeth Rather and Edward Conklin show a famous example that works for the ANSI Forth:

```
: ?IF POSTPONE ?DUP POSTPONE IF ; IMMEDIATE
```

The purpose of this word is to create an `IF` clone that checks for the value truth, and setting the `IF . . THEN` execution only in case of true value. For instance, we could create:

```
: TEST ?IF . " NOT NULL" THEN ;
```

This word accepts only zero and in case of not null value, prints a warning message (don't ask if this word has sense,

¹¹⁶ Anton Ertl on <https://forth-standard.org/standard/core/LITERAL>.

it hasn't obviously any, it's just an example).

The translation for Forth-79 (which has no POSTPONE actually) must take into account the following facts:

- 1 `COMPILE` compiles the PFA of the word that follows in the compiling definition; if this word is immediate or not has no importance;
- 2 `[COMPILE]` compiles the execution token of the word that follows in the compiling definition, and its purpose is to compile `IMMEDIATE` words, that are thus compiled, not executed. If used with non immediate words, nothing changes.
- 3 The overall effect of choosing to use `COMPILE` or `[COMPILE]` lies (obviously) in the immediacy of the word created

Apparently, they can be applied to all kind of words, but are we sure? If we think to use `[COMPILE]` for both `?DUP` and `IF` (putting a mere translation `POSTPONE = [COMPILE]`), well, it does not work at all! Here's what happens:

```
: ?if [compile] ?dup [compile] if ; immediate ok
: test ?if ." not null" then ;
? empty stack.
: test ?if ." not null" then ;
      ^
```

The compilation fails. The problem is that `?IF` is `IMMEDIATE`, so that all the words in it (diligently compiled by `[COMPILE]`) are called in interpret mode (with `STATE = 0`), and when `?DUP` executes, it finds no values on the stack and thus complains.

What must be kept in mind (and this was a hard lesson for me) is that in Forth-79 you have to manually do what `POSTPONE` automatically does, that is in `IMMEDIATE` words creation, use `[COMPILE]` for `IMMEDIATE` compiling words and `COMPILE` for non-`IMMEDIATE` compiling words, to prevent them from executing. Here's the correct translation in Forth-79:

```
: ?if compile ?dup [compile] if ; immediate ok
: test ?if ." not null" then ; ok
see test
: test
    ?dup (if)
    ." not null"
    (then)
;
ok
0 test ok
4 test not null ok
```

See how the two `?IF` are compiled in the two examples:

```
SEE ?IF
: ?IF
    ?DUP IF
; IMMEDIATE
ok
see ?if
: ?if
    (compile) 2532 if
; immediate
ok
```

In the first example, the word ?IF seems complete, but being IMMEDIATE, when compiled into TEST, it *executes* its body, rather than *compile* it. That's why ?DUP fails.

In the second example the ?DUP is not explicit, but it's represented by its LFA address so, even in an IMMEDIATE word, it does not really execute; anyway, in TEST, the ?DUP regularly appears. In practice, the COMPILER defers the compilation of the object word.

Another proof?

In the Complang site¹¹⁷ there are a couple of examples that extensively use LITERAL; the first is the (famous) [FOO] /FLIP /BAR case (output is from):

```
: [FOO]          ( COMPILATION: --; RUN-TIME: -- N )
  500 POSTPONE LITERAL
; IMMEDIATE
: FLIP
  [FOO]
;
FLIP . 500  ok
```

The previous lines work painlessly with tetra and the Forth-79, provided we substitute POSTPONE (as usual by now) with [COMPILE], because LITERAL is immediate:

```
: [foo]
  500 [compile] literal
; immediate ok
: flip
  [foo]
; ok
see flip
: flip
  500
;
ok
flip . 500 ok
see [foo]
: [foo]
  500 literal
; immediate
ok
```

The final test here discussed uses a similar scheme¹¹⁸ which is useful to understand the behaviour of POSTPONE (again, this is the output of):

```
: MY-+ ( COMPILATION: -- ; RUN-TIME OF COMPILED CODE: N1 N2 -- N3 )
  POSTPONE +
; IMMEDIATE ok
: FOO ( N1 N2 -- N )
  MY-+
; ok
1 2 FOO . 3 ok
```

¹¹⁷ <https://www.complang.tuwien.ac.at/forth//Docs-html/Literal-Tutorial.html#Literal-Tutorial>);

¹¹⁸ <https://www.complang.tuwien.ac.at/forth//Docs-html/POSTPONE-Tutorial.html>


```
: MY-- ( COMPILATION: -- ; RUN-TIME OF COMPILED CODE: N1 N2 -- N3 )
  POSTPONE NEGATE  POSTPONE +
; IMMEDIATE COMPILE-ONLY  ok
: BAR ( N1 N2 -- N )
  MY--
; ok
2 1 BAR . 1 ok
```

The natural conversion to Forth-79 leads to the following code:

```
: my+  compile + ; immediate ok
: foo  my+  ; ok
1 2 foo . 3 ok
see my+
: my+
  (compile) 1539
; immediate
ok
see foo
: foo
  +
;
ok
: my--  compile negate  compile + ; immediate compile-only ok
: bar my-- ; ok
2 1 bar . 1 ok
see my--
: my--
  (compile) 1592 (compile) 1539
; compile-only immediate
ok
see bar
: bar
  negate +
;
ok
```

The words + and NEGATE are not IMMEDIATE, so it's natural using COMPILE, in place of POSTPONE rather than [COMPILE].

The POSTPONE version in the tlibabry, not optimal of course, being structured by means of Forth-79 words, uses just this criterion to decide what to use for compilation, and setting for [COMPILE] in case of IMMEDIATE words and COMPILE for non-IMMEDIATE words (this may not be sufficient for all cases, of course).

6.6.3. Compiling is looking forward

The compilation of words that look ahead in the terminal buffer input is a tough argument when converting from ANSI Forth to Forth-79. An example is the compiling version of CHAR, which the standard calls [CHAR]. Neither are Forth-79 words, and they are contained in the library, if you want to use them.

Let's suppose we have CHAR installed and working¹¹⁹; if we look at the Standard version of [CHAR]¹²⁰, it looks like:

¹¹⁹ You can type ENSURE CHAR before going on.

¹²⁰ at <https://forth-standard.org/standard/core/BracketCHAR>

```
: [CHAR] ( "name"<spaces -- ) CHAR POSTPONE LITERAL ; IMMEDIATE
```

A definition like the one following (watch out! This is ANSI, not Forth-79!):

```
: TEST [CHAR] HELLO ;
```

is conceived to store the literal 72 (the letter 'H' ASCII code) into TEST by calling [CHAR]. The final result should be:

```
SEE TEST
: TEST
  72
;
ok
```

If we apply the same in Forth-79, and thinking to use [COMPILE] here (since again LITERAL is immediate), the following happens:

```
: [char] char [compile] literal ; immediate ok
: test [char] hello ; test isn't unique. ok
see test
: test
  72
;
ok
see [char]
: [char]
  char literal
; immediate
ok
```

This is another proof that, with immediate words, [COMPILE] is a better substitute to the ANSI POSTPONE.

6.6.4. Compiling is thinking

The words `:` (colon) and `;` (semicolon) are the main defining words in Forth, and they may be used in any context; the colon works only in interpret mode (the colon *compiles*, so it must be started in interpret mode) and the semicolon in compile mode (the semicolon *stops compilation*, so it must be found in compile mode). The Standard defines the colon without any particular property, the second (the semicolon) is instead marked as immediate. The general reason is: the first, the colon, doesn't need to be immediate, because its effect is anyway the starting of a definition, and immediacy here would be pleonastic. The second instead must execute to stop compilation, not compiled, so it must be immediate.

Now, one may be tempted to write new definitions for the colon and the semicolon, basing on the rules exposed in the previous paragraphs. We write:

```
: start [compile] : ;
: close [compile] ; ; immediate
```

Why the colon, which is not immediate, is compiled with [COMPILE]? Fact is that START is not to be put into a definition... it actually starts a definition, so it must effectively *be* the colon. We must remember that COMPILE is to be used for non immediate words into immediate words (tough things!). But START is not immediate (See point 2 below).

These two definitions are regularly accepted, because they are regular Forth words. If we try to *use* them, we find

that they also work regularly:

```
start cycle 11 0 do i . loop close ok
```

The compilation of CLOSE, in immediate mode, acts as the semicolon, and thus stops the compilation phase. The compiled word is a normal word:

```
see cycle
: cycle
  11 0 (do)
    i .
  (loop)
;
```

- 1) If we try to change this behaviour by marking CLOSE as not IMMEDIATE, we get:

```
: closes [compile] ; ;
```

What happens now, using CLOSES instead of CLOSE?

```
start cycle2 11 0 do i . loop closes (compiled)
;
```

The word CLOSES is compiled, rather than executed, so it's natural for the interpreter to wait for another compilation word. This must explicitly be accomplished by the semicolon (here CLOSES won't work, but CLOSE will). In case CYCLE2 is run, it performs the cycle and then it raises an error when CLOSES is invoked, because the semicolon is executed in interpret mode.

- 2) If we try to use COMPILE for START, we have:

```
: start compile : ; ok
: close [compile] ; ; immediate ok
start cycle2 11 0 do i . loop close cycle2 ?
```

The compilation is not started, so that the interpreter cannot understand CYCLE2. See how START looks like in this case:

```
see start
: start
  (compile) 2548
;
```

So, the colon, being an ordinary interpret-state word, can be redefined as in START, but has to be compiled with [COMPILE], while the semicolon being immediate must be compiled with [COMPILE] in IMMEDIATE mode. This leads to the conclusion that using COMPILE or [COMPILE] is not always a matter of immediacy... It depends on what we must compile: so when choosing between the two, one more reflection is necessary.

6.6.5. Let me be frank (oops, immediate!)

The word IMMEDIATE has one simple function: to mark as immediate the "*most recently made dictionary entry*"; it's not clear whether the dictionary entry on which it acts must be correctly completed or not yet, but since modern Forths seem to interpret IMMEDIATE as acting even before the completion of one definition, I think this is also valid for Forth-79, because IMMEDIATE (a pun) is not immediate by itself, so it must work as any other normal words, and thus be compilable.

In this regard, it's noteworthy noting that it's useless to compile IMMEDIATE into a general usage definition,

because it would always act on the most recently made dictionary entry, which is not really useful or even advisable. So that, if we define the following lines:

```
: test dup immediate . . ; ok
variable myop ok
see test
: test
    dup immediate . .
;
    ok
see myop
: myop
    (variable) 0
;
    ok
24 test 24 24 ok
see myop
: myop
    (variable) 0
; immediate
    ok
```

the intended behaviour is erratic, because at every invocation of TEST, the most recently made definition is marked as immediate!

A more intriguing behaviour is instead the following:

```
: test2 create , immediate does> @ 2 * ; ok
222 test2 gen ok
see gen
: gen
    4891 @ 2 *
; immediate
    ok
variable myop2 ok
gen . 444 ok
see myop2
: myop2
    (variable) 0
;
    ok
```

The word TEST2 creates a special word GEN which is immediate, but GEN has no IMMEDIATE in it, so it works *per se* without declaring. Moreover, every word created by TEST2 is immediate!

The word GEN, as can be easily understood, being immediate, cannot be compiled as-is, because it would execute, not compile; as it should now be clear from the previous chapters, it must be compiled with [COMPILE] GEN, as any other immediate words.

6.6.6. Compiling is...

Compiling is the core of Forth. Forth has no meaning if compilation wouldn't be possible. As a final rule-of-thumb, I can give the following basic rules:

- To compile an immediate word A in an immediate or not immediate word , use [COMPILE] A, to compile the word A in B, rather than executing it.

- To compile a word A into an immediate word B, and if A expects values on the stack, you have to compile A with `COMPILE`, in order to pass the address of A, rather than executing it. When in a second time B will be compiled into a third word C, the address of A will be retrieved and A regularly compiled into C.

You thought this was an easy matter, didn't you?

6.7. A couple of **VOCABULARY** problems

The word **VOCABULARY** has belonged to the Forth since its origin, but its management has been deeply changed from one version to the next.

Here I present some reflections on the concatenation of vocabularies and on the immediacy of a definition of a vocabulary, subjects not clearly exposed in the Forth-79 standard; these reflections guided my work in implementing **VOCABULARY** in tetra.

Here we go!

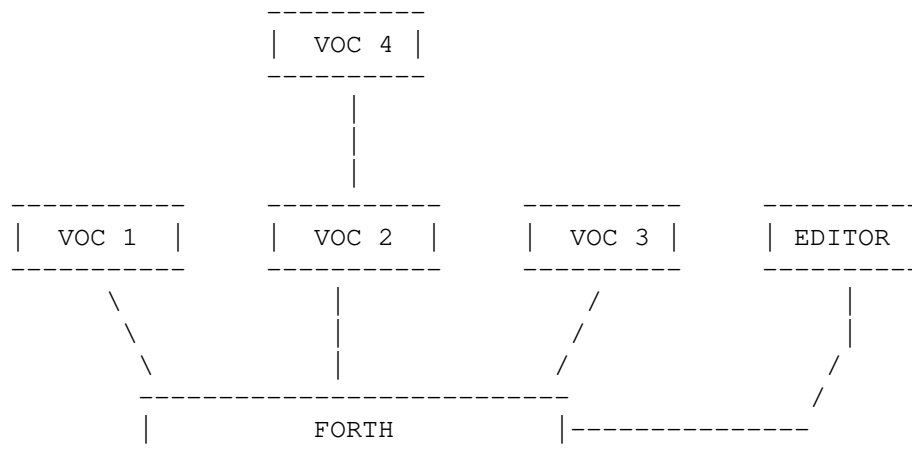
6.7.1. Chaining

The pure definition contained in the Standard: "*In lieu of any further specification, new vocabularies 'chain' to FORTH. That is, when a dictionary search through a vocabulary is exhausted, FORTH will be searched*" makes the **VOCABULARY** definition quite incomplete, someway a little weird, and leaves space to speculation.

This definition seems confirmed in the Forth-83 manual, where William F. Ragsdale writes about Forth-83: "*The method of selecting the order in which the dictionary is searched has grown from unchained vocabularies to the present [Forth-83] use of chained vocabularies*" (forgetting though that fig-Forth was the system that first used chained vocabularies).

The only conclusion here is that the Forth-79 Standard didn't require vocabularies to be explicitly chained, but they must be so in any implementation, as I will demonstrate.

Let's build a model of chained vocabularies. The design model example is the following:



VOC1, VOC2 and VOC3 are bound to FORTH and VOC4 is bound to VOC2 (that is created into it). The **EDITOR** vocabulary is always present. The dictionary is built with the following steps (this is an example, of course):

```
VOCABULARY VOC1 ok
VOC1 DEFINITIONS ok
: INVOC1 ; ok
```

```
FORTH DEFINITIONS    ok
VOCABULARY VOC2      ok
VOC2 DEFINITIONS     ok
: INVOC2 ;           ok
VOCABULARY VOC4      ok
VOC4 DEFINITIONS     ok
: INVOC4 ;           ok
FORTH DEFINITIONS    ok
VOCABULARY VOC3      ok
VOC3 DEFINITIONS     ok
: INVOC3 ;           ok
```

Following strictly the Forth-79 definition, every search is limited to the CONTEXT and FORTH dictionaries, so when the user is in VOC1, VOC2 or VOC3 the definition guarantees that the CURRENT and FORTH dictionaries are found; but the same cannot be said for VOC4: when the user is in VOC4, the words in VOC2 are not clearly guaranteed to be found (VOC2 is not CURRENT and is not FORTH). The definition is therefore not exhaustive; and, what does "*in lieu of any further specifications*" mean? Isn't this a Standard? Shouldn't it have all the needed specifications?

The only sage proposal is that vocabularies are chained according to their creation, and the engine traverses dictionaries in turn (the chain is not "CURRENT *and* FORTH", but "*from* CURRENT *to* FORTH").

In *tetra*, vocabularies are built so that this unclear rule is circumvented: when in VOC4, this is searched first, then VOC2, and finally FORTH, which remains always in the background. Words in VOC1 or VOC3 are not available, of course (while the words VOC1 and VOC3 remain available because they were built in FORTH). The example shows that when the user is in FORTH, invoking VOC2 and then VOC4, we get:

```
WORDS
INVOC4 VOC4 INVOC2 VOC3 VOC2 VOC1 EDIT INFO LISTEN ...
```

The words INVOC2 and INVOC4 are found, but INVOC1 and INVOC3 are not. The ORDER word returns actually:

```
ORDER    voc4 voc2 forth    forth
```

On the contrary, setting the vocabulary to, e.g., VOC1:

```
VOC1 WORDS
INVOC1 VOC3 VOC2 VOC1 EDIT INFO LISTEN ...
```

The word INVOC1 returns visible, but all the words in VOC3 and VOC2 (included VOC4 and all the words inside it) are hidden.

Technically speaking, every word (see the Appendix) has a hardwired vocabulary reference coded into it. This is the reference that makes a word available or not. If the vocabularies are chained as VOC4–VOC2–FORTH (as exemplified by the word ORDER), words in vocabularies not referenced in the chain are not seen (for instance, EDITOR is not available during the search in that chain).

The fact that the Standard definition is somewhat incomplete does not mean that Forth-79 implementers can ignore these facts. I didn't, anyway.

6.7.2. Immediacy

In *tetra*, vocabularies created with VOCABULARY are not IMMEDIATE by default. Actually, this option was not specified by the Standard, and different examples in the Forth literature use IMMEDIATE after each VOCABULARY, suggesting that this feature was not a requirement but it was anyway necessary¹²¹.

¹²¹ In fig-Forth, it was required that "All vocabularies have to be declared IMMEDIATE, so that context can be switched during compilation."

Anyway, in the Forth-83 Standard, chapter 5, it's written: "*Previously standardized words continue in their use: VOCABULARY, FORTH, DEFINITIONS, and FORGET. However, this proposal [the Forth-83] assumes that vocabulary names are not IMMEDIATE*". This statement implies that in (some) Forth-79 compilers, vocabularies were IMMEDIATE, though there is no evidence of this in the Forth-79 Standard document.

The solution, for me, is to leave the programmer the greatest freedom; if a VOCABULARY has to be immediate, it is declared as such (using IMMEDIATE), but if this is not required by its designer, IMMEDIATE is not necessary, and no error condition exists.

So in tetra VOCABULARY and the vocabularies generated by it are not immediate (but FORTH and EDITOR are, as indicated in the Standard).

6.8. The CONTEXT of the CURRENT affairs

The two system variables CONTEXT and CURRENT deal respectively with the searching into vocabularies and the compilation words process (or where new words are stored).

The two variables are not coordinated, and this may be a problem. Let's re-use the previous little program, extending it a bit:

```
VOCABULARY VOC1  ok
VOC1 DEFINITIONS  ok
: INVOC1 ; ok
FORTH DEFINITIONS  ok
VOCABULARY VOC2  ok
VOC2 DEFINITIONS  ok
: INVOC2 ; ok
VOCABULARY VOC4  ok
VOC4 DEFINITIONS  ok
: INVOC4 ; ok
FORTH DEFINITIONS  ok
VOCABULARY VOC3  ok
VOC3 DEFINITIONS  ok
: INVOC3 ; ok
VOC2 VOC4
```

In the last statement, this program sets VOC4 (found in VOC2) as the CONTEXT vocabulary (invoking a vocabulary name has the effect of changing the value of CONTEXT), and this means that all searches are performed into VOC4, then VOC2 and finally FORTH (to which all vocabularies are linked).

The last DEFINITIONS statement set VOC3 as the CURRENT compilation vocabulary; this situation is dangerous, since the programmer may 'forget' words that are in VOC4, but all the newly defined words are not findable (because VOC3 is not CONTEXT). Or the programmer may create new words in VOC3 (as he wishes), but if some of them have to be 'forgotten', there is a risk of FORGETting words in VOC4 with the same name instead.

That's why, in my opinion, fig-Forth required that CONTEXT and CURRENT had to be equal, in order to enable FORGET, otherwise an error was emitted.

The Forth-79 does not requires this: it only uses FORGET to "*Delete from the dictionary <name> (which is the CURRENT vocabulary) and all words added to the dictionary after <name>, regardless of their vocabulary.*"

It is up to the programmer to be aware of the situation: the word ORDER can be useful, in this case, because it writes the chain of CONTEXT and the vocabulary of CURRENT. My advice is to use it often, and in any case always before

But this does not imply that immediacy was implicit, and this is confirmed by the definition of the FORTH vocabulary in the fig-Forth as:

```
VOCABULARY FORTH IMMEDIATE
```

using FORGET! Here is the output that fits the example:

```
ORDER    voc4 voc2 forth    voc3 ok
```

It's easy to see that VOC3 is not in the CONTEXT chain. In this case, it's not recommended to use FORGET before thinking carefully about what we're doing. Before FORGETting check, check, check!

6.9. One word (or two) about LITERAL (and DLITERAL)

The specifics for LITERAL in the Standard didn't set it as a compile-only word,; they require that LITERAL *"If compiling, then compile the stack value n as a 16-bit literal, which when later executed, will leave n on the stack"*. This definition does not include the behaviour during execution: should n be maintained? Or is it consumed? Or should an error condition be issued? The solution I found was the one later applied in Forth-83, and probably left inadvertently behind in the Forth-79 Standard: setting LITERAL as compile-only. Simple. Elegant.

Of course, the same concept was applied to DLITERAL.

7. Forth in retrospective

Forth has passed many phases since its initial conception, and this is demonstrated by the several editions of the Standard, any new version taking a direction slightly different with respect to the previous.

In the following, I present some (unnecessary) discussions about Forth and the various versions, but remember that in all cases this is my personal point of view about the matter, and I'd be glad if you could suggest something.

7.1. Forth-77 (or the importance of having good ancestors)

Before the release of Forth-79, the various committee gurus agreed on a bunch of statements and a form of the language to discuss, called humbly simply Forth-77. The discussion certainly led to the final form of the language, that is Forth-79¹²².

It represents a first attempt to systematize and regularize the Forth language, and offers a first glance of the process of Standardization, even in the choice of the symbols and for the text organization.

It's peculiarity is that it is still a rough document, for us that made our way from fig-Forth onward. For instance, in the document, the letter C identifies words that are compile-only, while the immediacy is represented by the letter P (from *Precedence bit*); notable is also the letter E that represents words that are not normally compilable (something that the Forth-79 Standard tore off).

In any case, it represents the first schematization of a possible Standard, and the Forth-79 Standard owes much to it.

7.1.1. Ancestors? What ancestors?

The importance of Forth-77 lies in two facts:

- 1) the fact that the language in the Forth-77 was definite. The general plan was confirmed by the Forth-79 committee, and the language has already a definite form. If it had been adopted at this stage, not much would have changed.
- 2) the fact that many words that seemed important at the time would no longer be so in the definitive version, and in general many of such Forth-77 words were then reversed to the Reserved Word Set without giving them an official status.

7.1.2. The provisional does

In other cases the words received a final form only successively.

For instance, the `; :` was intended to be used in a general defining sense, for example:

```
: DEFINER CREATE cccc ;: nnnn ;
```

This was conceived for defining words, to perform the `cccc` word at the calling of the defining word and assign the `nnnn` part of code to the defined word:

```
DEFINER TEST \ <cccc> executes  
TEST        \ <nnnn> executes
```

This word received its final and definitive state as `DOES>` since the fig-Forth birth, so `; :` didn't survived:

```
: DEFINER CREATE cccc DOES> nnnn ;
```

(even if its relics are in the Reserved Words Set of the Forth-79).

¹²² You can find more news about the Forth-77 in the site www.complang.tuwien.ac.at/forth/forth-77.txt

7.1.3. The byte conception

Another singular issue in Forth-77 is that the 8-bit entity was called *byte*, not *character*, so that instructions to treat 8-bit objects were named B@, B! or BMOVE.

Passing to Forth-79, they would be promoted to treat characters, and renamed C@, C! and CMOVE. This is an important change of step, but it's necessary to recall here that fig-Forth was already aligned to the character-based names...

To me, the byte conception was probably a (failed) attempt to move the committee towards a new direction, rather than a historical point of view that was updated.

7.1.4. Influence?

The word ARRAY that was present in the Forth-77, represents a strange influence from other languages (that were proliferating in those magmatic times).

It was conceived to be used as such;

```
20 ARRAY GOOFY
```

to define a space of 21 (sic!) contiguous cells called GOOFY, where the sequence:

```
4 GOOFY
```

returned the 5th item in the array. In Forth-79 and in all modern Forth interpreters, this is done through CREATE and DOES>, without the need of a special word.

But this shows that in 1977 the Forth mental model was not complete, and some members of the Forth committee thought that this was necessary.

I observe incidentally here that neither Forth-79, nor Forth-83 and its successors (not even fig-Forth), and neither in the Reserved Word Set this word is present; it was only a tool of the Apple Forth. Once you get that ARRAY can be managed with CREATE, it's natural to think: "Who bothers, anyway?".

A rough example (without caring of checking the indices) is:

```
: ARRAY 1+ CREATE ALLOT DOES> + ; ok
20 ARRAY GOOFY ok
24 0 GOOFY ! ok
240 19 GOOFY ! ok
0 GOOFY @ u. 24 ok
19 GOOFY @ u. 240 ok
```

Indices run from 0 to 20, of course. A more sophisticated version could check the index value and if greater or equal to the limit or lower than zero, an error could be generated. In any case, this is Forth matter, no need to have a special and dedicated word...

Anyway, the Forth-77 defined it as such:

```
ARRAY      ( m ARRAY nnnn --- )
```

Define an array named nnnn and allocate m+1 uninitialized locations onto the dictionary. The sequence i nnnn will leave the address of the i-th cell on the stack. The index should be in the range 0 <= i <= m, but no check is made for values exceeding this range.

Stating this, my rough reconstruction is not that bad...

7.1.5. Curiosity killed the cat

Many words that were in the discussing list were curiously expunged from the final Forth-79 word set and placed in the Reserved Word List (that is to say: "*...they are not important, but if you want to implement them, you're welcome*"). For example, words like SET, SP@, READ-MAP, REMEMBER, REWIND are in the Forth-79 Reserved word list.

I noticed also other curious words: the word TYPE was accompanied by the equivalent word WRITE, that worked in the same vein, but directed "[...] *to the selected output device*". In the Forth-77 there is no command to select a particular device, so I suppose that TYPE was directed to the monitor (or printer) and WRITE to the tape (this needs confirmation, anyway).

The last curiosity is the word $\hat{\ } (\ n \ p \ \ -- \ q \)$ that was used (who would've ever thought?) to elevate n to the power of p to produced a signed q . To my knowledge, this word didn't survived in this form in any further Forth versions, included fig-Forth, and today is known as ****** (that is also in the Reserved Words Set of the Forth-79). A real and authentic relic from the past!

7.2. Apple Forth and 64Forth (or the importance of having good cousins)

Undoubtedly, the Apple Forth 79 and the Commodore 64Forth were among of the most important environments in the Forth history, not only because of the wide diffusion in the whole Western world of the Apple II computer and the CBM Commodore 64, which were gems for those times, but also because they followed strictly the Forth-79 Standard, with some eye-blinks to the growing market in the BASIC side (the ones developed by Apple and CBM themselves, or the ZX Spectrum), with the explosion of the micro-computer era of the early Eighties, dominated by the BASIC language.

Let's present them.

7.2.1. The Apple Forth-79

The Apple Micromotion Forth 79 (dating 1980) is robust in the interpretation of the Standard, but also adopts the fig-Forth terminology, and has a wide vocabulary, which adopts many non-standard words, and a bunch of definitions of the Reserved Words Set is also added, so if someone of you begins to think that **tetra** is following the very same path, he'd be right...

The manual (Volume I) of the Apple Forth specifies that the authors of this program are: Anita Anderson and Martin Tracy, with the help of Philip Wasson. Its particularity can be summarized as follows:

- The complete adherence to the Forth-79 Standard (even the PAD is limited to 64 characters);
- A complete assembler dedicated to the 6502 chip (the same of the Commodore 64), a chip with advanced features for the times (graphics, files devices - cassettes, printers, 5&1/4 and 3%1/2 floppy disks - sprites, etc);
- A complete set of words for the string management, mostly derived from BASIC (at least for what about the terminology): LEN, RIGHT\$, LEFT\$, MID\$, for instance (you can find my rendition of some of them in the library);
- An advanced set of words in bit-management: TOGGLE, HILO, SQUISH (you can find my rendition of them in the library);
- An advanced set of words for file-treatment: LOAD-BUFFERS, \$DRS, INDEX, DOC, R/W, SAVE;
- An advanced control-flow system, adding the CASE . . OF . . ENDOF . . ENDCASE structure, the GODO structure (inspired the by the ON A GOTO . . . of the BASIC), the ONERR and TURNKEY words, to help in using the Forth loading and error management.
- The SWAPDROP word, progenitor of the modern NIP

7.2.2. The 64Forth

The Apple Forth was a very complete environment, but unfortunately, the Apple II computer was too expensive, where I lived, so I reverted to a Commodore 64, for my youth plays, founding that it was not bad at all.

The 64Forth of the Commodore 64 (dating 1983) is based upon fig-Forth, and so it's someway *older* than the Apple Forth, though published later, but it's a robust and working version, with all the extended words to treat sprites, sound and all those little amenities of the Commodore 64 (a wonderful machine, after all).

The manual specifies that the 64Forth was the result of the "*combined efforts of many people. Here is a list of some of these people and their specific contribution.*"

<i>Bob Reigling</i>	<i>Entered and debugged the original 6502 source.</i>
<i>Lyle Thompson</i>	<i>Wrote all of the relative file disk interface primitives.</i>
<i>Ken Madell</i>	<i>Wrote the sprite control words, and the sprite editor.</i>
<i>Bill Ragsdale</i>	<i>Provided the Assembler and assembler documentation.</i>
<i>Forth Inc. & Leo Brodie</i>	<i>Created STARTING FORTH and the line editor which is the primary basis of the editor included in this package.</i>
<i>Forth Interest Group</i>	<i>Provided support and assistance to the Forth community.</i>

Many of these name will recur in the Standard teams or in the review "*Forth Dimensions*", and this makes the 64Forth a peculiar version. Its particularity can be summarized as follows:

- The complete adherence to the fig-Forth version (with PAD extended to 80 characters, and two additional pads called PADF for the find buffer and PADI for the insert buffer); THEN is always misplaced as ENDIF, which is a fig-Forth mark.
- The main vocabulary is not FORTH but SYSTEM;
- A complete assembler dedicated to the 6502 chip (the same of the Apple II), a chip with advanced features for the times (graphics, files devices - cassettes, printers, 5&1/4 and 3%1/2 floppy disks - sprites, etc); sprite words are all classified beginning with X or SP, and this is a Commodore mark.
- A complete set of words for the sound management: VOICE<x>, VOLUME@, VOLUME!, VRESET, TRIANGLE, SQUARE, SAWTOOTH, TEST<x>, SYNC<x>, SUSTAIN!, SUSTAIN@, RESONANCE!, RESONANCE@, RELEASE!, RELEASE@, OSC<x>!, OSC<x>@;
- An advanced set of words for port and RS-232 serial port management: UPORT, CHKIN, CHKOUT, UNTALK, UNLSN, STYPE;
- An advanced set of words for file-treatment: ^FILE, FCLOSE, READ, , FLOAD, SAVE;

Being a fig-Forth with Commodore graphics and sound insertions, I cannot emulate the related non-fig commands inside *tetra*, which is a purely textual and non-graphical environment, but in any case the 64Forth is a typical example of a Forth System inspired by the Standards of the Seventies, which did not take into account the release of the Forth-83 Standard, which remained in the corner until the new revision of 1994 set a new path for the modern Forth.

7.3. Conversion of fig-Forth programs

I'm sure you all have a lot more experience than me in finding differences between Forth-79 and fig-Forth and knowing how to convert a program from one environment to another.

So, I present here only what I found in my experience while writing programs for **tetra**, having to decide whether the faults in the program were due to errors in my implementation or to a precise difference in behavior with respect to the original Forth environment from which I was translating the software.

fig-Forth is, for Forth-79, the closest cousin, quite a twin actually, for many reasons. They were conceived in the same period (late Seventies), and the Forth-79 Standard was built using the fig-Forth experience.

But not all was plainly passed to the Standard. Some important features changed, making the two environments not completely compatible.

Here is what I found.

7.3.1. Can I say a **WORD** ?

The word **WORD** (you'll excuse the forced pun) has passed a robust change in passing from fig-Forth to Forth-79; here are the two definitions cores:

fig-Forth:

WORD c ---

"[...stores...] the packed character string beginning at the dictionary buffer HERE [...]"

Forth-79:

WORD char -- addr

"The characters are stored as a packed string with the character count in the first character position [...]. The address of the beginning of this packed string is left on the stack."

The difference is subtle but important: Forth-79 does not involve **HERE** anymore. The address may be any (a reserved dictionary space, a dedicated buffer, or **HERE** again), and it's left on the stack (incidentally, this explains why in fig-Forth it's frequent seeing the token **WORD HERE**). In **tetra**, by the way, **WORD** has a dedicated buffer.

To convert this word from fig-Forth, the word **HERE** must be omitted (because the address of the counted string is already on top on stack after the execution of **WORD**).

The evolution of Forth is not favorable to **WORD**: it will receive more critics after the release of the ANSI 1994, because of the bad features here described, in favour of the new ANSI definitions **PARSE** and **PARSE-NAME** (see for instance **GForth**). **WORD** will remain in the Standard (up to now, at least) only for back-compatibility issues. A bad destiny for such a historical word...

7.3.2. **+LOOP** limits

The two definitions in the two manuals are quite the same:

fig-Forth:

*"..The branch back to **DO** occurs until the new index is equal to or greater than the limit ($n1 > 0$), or until the new index is equal to or less than the limit ($n1 < 0$)¹²³."*

Forth-79:

*"...Return execution to the corresponding **DO** until the new index is equal to or greater than the limit ($n > 0$), or until the new index is less than the limit ($n < 0$)."*

The real difference is when **+LOOP** is used with negative increments; while Forth-79 includes the limit, fig-Forth explicitly avoids it. So if something like

```
0 10 DO ... -1 +LOOP
```

¹²³ Well, the text suggests that `@n1≤0#` is a more exact limit, but the fig-Forth manual really writes `@n1<0#`.

is written in fig-Forth, the Forth-79 equivalent instruction is

```
1 10 DO ... -1 +LOOP
```

7.3.3. Using **VARIABLE** properly

The fig-Forth **VARIABLE** needs an initialization parameter on the stack:

```
10 VARIABLE ICONS
```

Of course, the Forth-79 version of **VARIABLE** is slightly different, and the previous line must be translated as:

```
VARIABLE ICONS 10 ICONS !
```

Sometimes, you may find a declaration like

```
0 VARIABLE NAME 28 ALLOT
```

This is writable in Forth-79 as a plain translation:

```
VARIABLE NAME 28 ALLOT
```

with the same effect on program, but the proper way to do it in Forth-79 is

```
CREATE NAME 30 ALLOT
```

which offers a visual declaration of the dedicated space. The same must be said for **2VARIABLE** (considering that **2VARIABLE** allocates four bytes at start)¹²⁴.

7.3.4. The **SIGN** of destiny

Curiously (and unexpectedly) the word **SIGN** was changed passing from fig-Forth to Forth-79; the change is small and, since then, it has been maintained in Forth83, ANSI 1994 Forth and probably will also be in next Standard. The fig-Forth stack picture is

```
SIGN      n d -- d
```

This curiously means that **d** is not touched at all. The Forth-79 stack picture is

```
SIGN      n --
```

Since it's always used after **#S**, which leaves a double number on the stack (a double zero), the global effect of **SIGN** in fig-Forth is to perform an internal **ROT**, to surface the signed value **n** to evaluate; the fig-Forth usual figure is:

```
<# ..... #S SIGN #>
```

See for instance the Alan Winfield book *The Complete Forth*, and precisely the example of **. \$** at par. 8.4, where he

¹²⁴ The difference in the two operations for what about Forth-79 is:

- The phrase **VARIABLE NAME** reserves two bytes, whose start address is returned by **NAME**; the addition **28 ALLOT** simply extends the memory, starting from the address following **NAME**, adding so 14 new slots, in total 15. Now, **NAME** returns the address of the first elements of the array.
- The phrase **CREATE NAME** reserves no bytes at all, but simply builds **NAME** to return when invoked the address of the memory after the initialization token: the addition **30 ALLOT** reserves 15 slots for 15 variables starting from the position after the initialization token, so that now, when invoked, **NAME** will return the address of the first element of the array.

The difference is only in the memory tokens, but the user does not perceive it: one way or the other is perfectly equivalent in terms of Forth.

uses SIGN directly after #S (he claims to use Forth-79 but his R-Forth compiler has some strange relations with fig-Forth...)

The Forth-79 SIGN, instead, looks for the signed value on the top of stack (more naturally, I must say), so the ROT must be explicitly performed by the programmer; here is the Forth-79 usual figure:

```
<# ..... #S ROT SIGN #>
```

See for instance the *Forth Programmer's Handbook* page 91, where the . (dot) and NNN words are designed, both using ROT SIGN:

```
: NNN ( d -- )   SWAP OVER DABS  <# # # #S
  ROT SIGN  #>   TYPE SPACE ;
```

7.3.5. How to FIND it?

The word FIND of Forth-79 passed a robust change from fig-Forth to Forth-79. Here are the two definitions:

fig-Forth:

```
"-FIND --- pfa b tf (found) --- ff (not found)
```

Accepts the next text word (delimited by blanks) in the input stream to HERE, and searches the CONTEXT and then CURRENT vocabularies for a matching entry. If found, the dictionary entry's parameter field address, its length byte, and a boolean true is left. Otherwise, only a boolean false is left".

Forth-79:

```
"FIND          -- addr
```

Leave the compilation address of the next word name, which is accepted from the input stream. If that word cannot be found in the dictionary after a search of CONTEXT and FORTH leave zero".

Changes are many:

- * the name was changed from -FIND to FIND
- * the return value was changed from the parameter field address to the compilation address
- * the length byte and the true flag were removed
- * the search field changes from "CONTEXT and CURRENT" to "CONTEXT and FORTH"

To use a version almost compatible with -FIND, you can ENSURE it from the library (the word simply finds the compilation address or zero, and if not zero converts it to PFA and then adds the length byte and the true flag). The search field is not changed (and remains the one of the Forth-79) but this -FIND use is suitable for most of the common scopes.

7.3.6. A VOCABULARY lookup

The VOCABULARY word, in my opinion, was better defined in fig-Forth than in the Forth-79 Standard; let's look at the definitions:

fig-Forth:

"A defining word used in the form:

```
VOCABULARY cccc
```

to create a vocabulary definition cccc. Subsequent use of cccc will make it the CONTEXT vocabulary which is searched first by INTERPRET. The sequence "cccc DEFINITIONS" will also make cccc the CURRENT vocabulary into which new definitions are placed.

In fig-FORTH, cccc will be so chained as to include all definitions of the vocabulary in which cccc is itself defined. All vocabularies ultimately chain to Forth. By convention, vocabulary names are to be declared IMMEDIATE. See VOC-LINK".

Forth-79:

"A defining word executed in the form:

VOCABULARY <name>

to create (in the *CURRENT* vocabulary) a dictionary entry for <name>, which specifies a new ordered list of word definitions. Subsequent execution of <name> will make it the *CONTEXT* vocabulary. When <name> becomes the *CURRENT* vocabulary (see *DEFINITIONS*), new definitions will be created in that list.

In lieu of any further specification, new vocabularies 'chain' to *FORTH*. That is, when a dictionary search through a vocabulary is exhausted, *FORTH* will be searched".

The difference is not subtle at all. fig-Forth **does** specify that all vocabularies are linked according to their creation, and ultimately link to *FORTH*, while Forth-79 simply attests that vocabularies are simply chained to *FORTH*, leaving some speculation behind.

But any possible reasoning about it (see the chapter "VOCABULARY problems", for instance) lead to one conclusion only: no Forth-79 compiler must differ from the fig-Forth behaviour. My humble opinion is that the Standard fails in clearly defining the matter.

7.3.7. Where is it?

The introduction of the 79 Standard, after many years in which fig-Forth was the *de facto* Standard, deliberately changed the name of some words that was current cash in fig-Forth, without really changing the runtime behaviour, but causing some mental displacements in old fig-Forth users. I suppose the conversion of fig-Forth programs to Forth-79 revealed quite painful, in those first years.

Anyway, here is a list of commands that work quite exactly like the Standard, but their name is not the Standard one, basing on my experience:

fig-Forth	Forth-79
-DUP	?DUP
<BUILDS	CREATE
DMINUS	DNEGATE
IN	>IN
MINUS	NEGATE
R	R@
U/	U/MOD

7.4. Forth-83 and beyond (or how evolution has won)

The release of Forth-79 established a firm point in the Forth development. But Forth scientists, in the following years, posed a number of questions that the Forth community couldn't ultimately ignore any more:

For instance, John Arkley, in "*Forth Dimensions*" Vol. IV, N. 3 September/October 1982, writes: "*79-Standard FORTH fails to address the problems of directories, directory management, file creation, device definition, device assignment, special driver linkages, keyboard mapping, function keys, and the like. The philosophical reason is that no one operating system will ever do what is desired, and it usually hinders rather than supports the programming language system*". Robert L. Smith writes in Vol. IV, N. 2 July/August 1982: "*The area of greatest concern for the next Standard is that of vocabularies. Forth-79 has a very weak vocabulary structure. It was chosen as the minimum subset of most FORTH implementations*".

All these instances led to the release of Forth-83 in early 1984. In the following, many concepts of Forth-83 are discussed; keep them in mind if you want to translate Forth-83 code for *tetra* (which is a 79-er). Keep also in mind that the ANSI 1994 is a direct evolution of Forth-83, and that many concepts apply also to it.

I imagine this change was a very painful act for the Forth-79 programmers... many revisions of existing code... and the need to change considerable years of proven habits... However, at least as far as the Forth-83 is concerned, I must recognize here that the Forth-79 can simulate the behavior of its successor quite well, although in some marginal

cases problems may arise (e.g., the family of operators >MARK, >RESOLVE are too implementation-dependent to be really easy to write in high-level Forth-79).

Anyway, here it is¹²⁵.

7.4.1. The Compilation Address

In fig-Forth, four addresses are used to name different fields in a word definition in the dictionary: the Name Field Address (NFA), the Link Field Address (LFA), the Code Field Address (CFA), and the Parameter Field Address (PFA).

The Forth-79 uses only two of these addresses, the Compilation Address (that in Forth-79 identifies the start of the definition) and the Code Field Address.

To enhance code portability, Forth-83 uses only one address, the Compilation Address (roughly equivalent to the Code Field Address in Forth-79). The Compilation Address is the one now returned by ' and FIND, compiled to colon definitions, and used by EXECUTE to run the word definition. One extra word is provided in the Forth-83 Standard to access information stored in the Parameter Field: >BODY.

7.4.2. To be or NOT to be

In Forth-83 the operator NOT was built according to this definition:

```
NOT          16b1 -- 16b2          83
16b2 is the one's complement of 16b1.
```

This differed from the definition in Forth-79 which was:

```
NOT          flag1 -- flag2        165
Reverse the boolean value of flag1. This is identical to 0=.
```

The difference is that, while the one's complement is performed by inverting all the bits in a number (and this justified the turning of the True value to -1, because by inverting all bits of zero, one gets -1), the inversion of the boolean value means turning zero to a not-zero value or turning any not-zero value to zero, without taking into account the bits state.¹²⁶

7.4.3. FIND and ' (tick)

In Forth-83 FIND operates differently than in Forth-79, and expects the string to be searched as a counted string address, rather than searching it in the Input Buffer. The ' (tick) instead returns the Compilation address, rather than the Parameter Field Address.

7.4.4. Unsigned index arguments for memory commands

The words BLANK, FILL, MOVE and MOVE> in Forth-83 use unsigned index values.

7.4.5. Removal of STATE- dependent words

Many Forth-79 words work both in the interpret mode or compile mode, with different behaviours, like LITERAL, ' (tick), ." (dot-quotes), for instance. In Forth-83, the STATE-dependent words are either eliminated or separated in interpretation-only or compilation-only words, making the system less ambiguous. For instance, ' is split into two words: ' (interpreted version) and ['] (compiled version), and ." is split into . (interpreted version) and ." (compiled version).

¹²⁵ More can be read on the interesting web page: <http://lars.nocrew.org/dpans/dpansd.htm> where all these differences are exposed.

¹²⁶ Read, for instance, the article "Representation for Logical True", by Robert L. Smith, in "Forth Dimensions", Vol. IV, N. 4 November/December 1982.

7.4.6. Improved DO . . LOOP

The major change in DO . . LOOP was that in Forth-83 LEAVE is made to terminate the loop immediately upon its execution rather than setting the index of the DO . . LOOP to the terminating value.

7.4.7. Division type

Division in Forth-83 is floored towards negative infinity, instead of rounded towards zero. The quotient and modulus have now a smooth change between positive integer and negative integer domain when either divisor and/or dividend is negative. This lead to the change of the definitions for /MOD and */MOD, which now return a remainder equal in sign to the divisor, rather than to the dividend. This behaviour will be maintained until now, but ANSI will introduce SM/REM to explicitly return the same behaviour of the Forth-79 /MOD, and thus asserting that the symmetric version is not too bad!

7.4.8. Zero-based PICK and ROLL

In Forth-79, ROLL and PICK were 1-based, that is the stack count numbered the top item as 1. In Forth-83 the top of the data stack is treated as a memory area: addressing into this area is zero-based like addressing regular memory areas.

7.4.9. Improved WORD

WORD was improved to return a blank space after the string, while Forth-79 explicitly states that the current delimiter found is stored at the end of the definition and excluded from counting. Nothing changes in ignoring leading delimiters so, in case of empty strings, Forth-83 behaves exactly like Forth-79. This will change with Forth-94.

7.4.10. Renaming of some words

The words listed below have been renamed but retain substantially identical properties:

Forth-79	Forth-83
U/MOD	UM/MOD
U*	UM*
<CMOVE (Reference Word Set)	MOVE>
BLANKS (Reference Word Set)	BLANK
79-STANDARD	FORTH-83.

Curious thing is that U/MOD passes its second transformation from Forth-79 to Forth-83, after the change from fig-Forth to Forth-79! A real wanderer!

7.5. The ANSI Forth (or multiply like rabbits)

The modern ANSI Forth has extended to so much several official words that I think it's impossible for one human to store everything in his head, and fact is that many of the additions made to Forth are compilable with Forth-79, apart from some notable aspects:

- The CATCH/THROW error detection mechanism, which is impossible to implement exactly equal in Forth-79...
- The floating point stack and the dozens of words added to Forth; moreover, integers now are in full 32 or 64 bits, which covers an innatural high dominion for single numbers...¹²⁷
- The locals and xchar sections, which was not a problem, when Forth-79 was born...
- The files section, which tetra has tried to replicate for a small number of words, but that can be conveniently substituted with the old dear blocks system.
- The string section (the paradigm has changed: now most words that use strings expect address on the stack)
- The memory section (the paradigm has added one necessity: to use aligned addresses)
- The tools section (some were already in Forth-79, like WORDS, STATE, SEE, FORGET, EDITOR, .S, DUMP, ASSEMBLER)

¹²⁷ I always prefer simpler working fields, and 0.65535 is the right measure, to me...

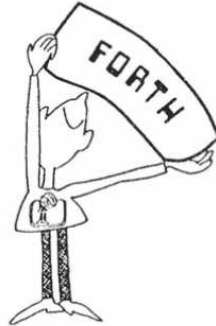
- The facility section, with many commands to treat keyboard keys, structures, and also AT-XY (that previous Forths know as GOTOXY)

The ANSI Forth today is a perfect machine, but also a huge mass of words that, yes, can do anything, but you must consider that, since any Forth system has more words than those in the Standard (because anyone of us has his own ideas), it happens that, excluding the ASSEMBLER dictionary, the modern SwiftForth has more than 1200 words and, hear this, GForth has more than 1800 words in the dictionary!¹²⁸, something that crushes the user like a boulder!

¹²⁸ For what it's worth, **tetra**, in its naked form, has about 340 words, which is even too much, to me!

8. MY FAVOURITE FD ARTICLES

I list and comment here on some of the Forth Dimensions articles I have found most interesting, especially those related to Forth enhancement, algorithms, games, math and its derivatives. I know that you may know them better than I do, but I hope that you, like me, see in these articles the flares of intelligence woven with the veils of time.



(Note: reproduced without permission. If its authors should ask for removal, this image will disappear in the next version.)

*The numbers in square brackets refer to the blocks (included in the package) where the software described in the article (translated to **tetra** and anyway tested) is written, using, as accurately as I could, the original text output; use **LOAD** to load the screens with the first block number on top of stack. If a name is reported in square brackets, it refers to the file sample (also included in the package), to be loaded from console as an argument of **tetra** (in this case, read the source to know how to use the program); use **INCLUDE** to load the file from within the Forth session.*

Let's begin...

Extensibility with Forth, by Kim Harris

Vol. I, N. 2 August/September 1978

A quasi-philosophical lesson about the power of Forth in solving any kind of problems.

Threaded Code, by John James

Vol. I, N. 2 August/September 1978

A brief essay about Forth's memory. After reading this article I thought: "Why not writing a Forth compiler by myself?"¹²⁹

Forth-85 "CASE" Statement, by Richard B. Main

Vol. I, N. 5 January/February 1980

Interesting study, the first appeared in this review with this topic, aimed to implement CASE (in a different way than the one now standard).

Forth, The Last Ten Years and The Next Two Weeks..., by Charles Moore

Vol. I, N. 6 March/April 1980

Ladies and Gentlemen: Charles Moore! 16 pages of history of Forth, experiences, a bit of projects, by the Forth's creator... This is the audiotape transcripts of the Forth Convention held in San Francisco in October 1979, only months before the Forth-79 Standard issue. The fig-Forth member Jim Berkey recorded the technical sessions and the banquet speech by Charles Moore, along with the questions posed to Moore and the following answers.

Forth in Literature, by W.F. Ragsdale

Vol. II, N. 1 May/June 1980 [20108-20110]

An automated speech generator: The Theory that Jack Built.

¹²⁹ Now I can tell you that this experience was like entering Doom!

Recursion - The Eight Queens Problem, by Jerry LeVan

Vol. II, N. 1 May/June 1980 [20057-20059]

A fast algorithm to give all solutions to the Eight Queens problem.

Temporal Aspects of the Forth language, by John M. Derick

Vol. II, N. 2 July/August 1980

A very instructive article about Forth, directed to programmers with different experiences not belonging to the Forth world.

Towers of Hanoi, by Peter Midnight

Vol. II, N. 2 July/August 1980 [20012-20017]

The algorithm for the Towers of Hanoi, solved with a direct translation from a Pascal version. I had to introduce some more delay (namely increasing the DELAY parameters and adding a call to DELAY in TRAVERSE) because computers nowadays are too fast!¹³⁰ To stop the execution, press any key.¹³¹

Random number generator, by Peter H. Helmers

Vol. II, N. 2 July/August 1980 [20044]

A simple but useful random number generator algorithm, which appears on the last page of the issue.

CASE contest (monography)

Vol. II, N. 3 September/October 1980

The whole number is devoted to the CASE structure proposals, submitted by many, who show their own point of view. The first (titled "*Just in CASE*", by Charles E. Eaker) is the only one exactly equal to the current Standard version. So I assume (I don't know it for certain) that Mr. Eaker is the CASE winner!

The execution - Variable and array, by Michael A. McCourt

Vol. II, N. 4 November/December 1980 [20502] [xeq.ft]

This article shows the power of Forth in dealing with deferred execution. Unfortunately, the code for () XEQ (deferred execution in arrays) is quite unreadable; so I limited the transcription to the XEQ, INSTALL and IN words only. If you can do better than me, let me know.

The CASE, SEL and COND structures, by Peter H. Helmers

Vol. II, N. 4 November/December 1980

This article shows some interesting structures for multiple selection (all predating the CASE structure of Forth-83); these structures, adopted from UORTH, the Forth system at the Rochester University, didn't turn to be widely used, but it's nonetheless an interesting point of view, in times when the language was still magmatic.

Programming Aids & Utilities, by Kim Harris

Vol. III, N. 1 January/February 1981

A collection of utilities screens in Forth. Many of them are so common that I put them in the tlibrary.

Product Review: Timin-Forth", by C.H. Ting

*Vol. III, N. 1 January/February 1981 [*bench.ft]*

In this small but interesting article, Mr. Ting presents the Timin-Forth for CP/M, by Mitchel¹³² E. Timin. He then compares this CP/M Forth with the Z80 Forth by Ray Duncan using six benchmark programs.

I was so intrigued by this test that I built a bench test for tetra using four out of the six words (the remaining are less interesting), and translated the code for GForth and swiftforth. I know it's quite unfair, but here are the comparison results of the original behaviours with the timing of the same program ran through my current computer (a laptop

¹³⁰ The original algorithm showed no progresses and I thought it was a failure in tetral, or maybe the algorithm was bad conceived, or maybe there was an error of trascription; figure out how many time I spent trying to understand that there were no failures but only that the program was too fast!

¹³¹ The program is executed in fig-Forth mode, and it is not reset to default at the stop. So be sure to set FIG-STATE to zero if you want to restore the Forth-79 mode afterwards.

¹³² Sic.

with 4 processors Intel Core i3-5005U @ 2.00 GHz, arch. x86_64) adapted for tetra, GForth and Swiftforth:

Absolute values (seconds):

	Z-80	Timin	tetra	GForth	Swiftforth
test 1 (loop)	2.9	2.3	0.00375	0.000143	0.000010
test 2 (-)	7.4	5.9	0.01057	0.000233	0.000010
test 3 (*)	54.9	44.0	0.01105	0.000239	0.000010
test 4 (/)	88.6	74.3	0.01049	0.000231	0.000010

Speed ratios (base=Z-80, the slowest):

	Z-80	Timin	tetra	GForth	Swiftforth
test 1 (loop)	1.00	1.26	773.33	20,279.7	290,000.0
test 2 (-)	1.00	1.25	700.09	31,759.6	740,000.0
test 3 (*)	1.00	1.25	4,968.33	229,707.1	5,490,000.0
test 4 (/)	1.00	1.19	8,446.14	383,549.8	8,860,000.0

tetra is (44 years after!) at least 8500 times faster than the Z80 Forth in 1981 (for test 4, division). My results are good, but... what about GForth? 380000 times faster (for the same test 4)! And what about the astonishing result of Swiftforth, which is (for the very same test 4) 8.8 million times faster? Or, to say it differently, tetra is 45 times slower than Gforth and 1000 times slower than Swiftforth, that is to say I still have a long way to go...

You can find the benchmarks files in the files/ directory of the installation folder: qbench.ft (for tetra), gbench.ft (for GForth) and sbench.ft (for Swiftforth). The qartus version was faster, I know, but less gifted...

HEX strip, by Baroudi

Vol. III, N. 1 January/February 1981

A humorous strip, understandable only by computer scientists like you...



(Note: reproduced without permission. If its authors should ask for removal, this strip will disappear in the next version.)

Compiler Security, by George W. Shaw III

Vol. III, N. 1 January/February 1981

Some notes (in a long article) about the security issue on fig-Forth. The article is too much fig-Forth-oriented to be fully compatible with a Forth-79 compiler, but it explains in detail how fig-Forth error-dealing words can help in validating Forth code in many fields: structural errors, parameters errors, machine state errors; The conclusion is: "Should we have all the protection all the time, or just some of it and a programmer protection package? Or maybe there is a better alternative."

Userstack, by Peter H. Helmers

Vol. III, N. 1 January/February 1981

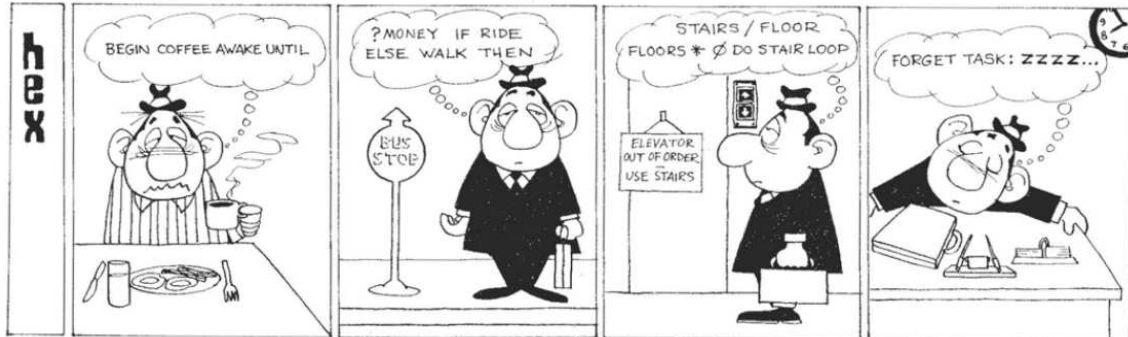
A proposal for a User Stack to avoid the problems involved with the Return Stack. Unfortunately, this interesting

article cannot be converted to **tetra** because it uses Machine Language targeted for the 8080 CPU. Holy patience...

HEX strip, by Baroudi

Vol. III, N. 2 July/August 1981

Another humorous strip, this time understandable only by Forth programmers like you...



(Note: reproduced without permission. If its authors should ask for removal, this strip will disappear in the next version.)

Forth Standard Corner, by Robert L. Smith

Vol. III, N. 3 September/October 1981

This article, which is about the **WORD** implications, had such a strong impact on me that I modeled the **WORD** definition in **tetra** following some of Mr. Smith suggestions.

The Forth-79 Standard - A Tool Box?, by George W. Shaw

Vol. III, N. 3 September/October 1981

A very interesting but small article about the definition of transportability of Forth-79 environments (which translates to me in "how far can you go in adding new not-compatible statements to your compiler?"). Here is the conclusion: "When considering the Forth-79 Standard, treat it as a basic tool box. Additional tools are applications from the point of view of the standard. Extend it as necessary. Can you add what you need by defining it only in terms of standard words? If not, what is the minimum necessary to allow you to do that: more definitions or more explanations? Experience is all that will tell. Send in your results."

Book Review: Starting Forth, review by George W. Shaw

Vol. III, N. 3 September/October 1981

This is not a book review. This is **THE** book review. About the book of the books about Forth. About the author of the authors about Forth. The Master. The Teacher.

Recursion and the Ackermann Function, by Joel V. Petersen

Vol. III, N. 3 September/October 1981 [ack.ft]

A recursion study which is totally compatible with **tetra**; the computation limits are the limits for the data and return stacks (1024 cells); I don't know which were the stacks limits for the NIC-Forth used in the test, but the study results for the NIC-Forth and **tetra** are the same (for both, the test cannot go beyond the 4,0 couple).¹³³

¹³³ To use the program effectively, use option `--stack-size=32767` and `--return-stack-size=16384` (that is the maximum extension for stacks) when invoking **tetra** on the **ack.ft** file.

HEX strip, by Baroudi

Vol. III, N. 3 September/October 1981

Another humorous strip: epitome of my nerd life...



(Note: reproduced without permission. If its authors should ask for removal, this strip will disappear in the next version.)

Forth in Laser Fusion, by Lawrence P. Forsley

Vol. III, N. 4 November/December 1981

This volume shows a number of Forth implementations for some industrial and medical fields. The first article, for instance, is a very interesting perspective about an industrial Forth implementation; written with precision, it exposes some interesting results using Forth as the language for the implementation of a laser beam technology. Think this volume dates back to 1978, and think how many years passed from then, and think how early those men did conceived all this...

Implementing Forth based microcomputers at the University of Rochester Medical Center, by Peter H. Helmers

Vol. III, N. 4 November/December 1981

Another Forth specificity. Microcomputers (and also dedicated units) are common today, but in 1981 rather a novelty. The application fields of this medical research were: Ultrasound Diffraction Apparatus (UDA), Vein Mechanics, Pulmonary Microcomputers, X-Ray Scanning System; in all these fields a Forth engine was built, to process input and output. Apparently with success. It's with surprise that the author ends with the Hamletic doubt: "*Anachronism or Portent?*". But further reading shows that "*admittedly, Forth is somewhat limited without such things as a file system or procedural name scoping of variables. Perhaps there should also be less explicit knowledge of addresses, and more system security. Perhaps. But if so, then these things will be evolved as Forth matures.*". Does Forth meet these standards today? I'm not the one that can answer to this.

Data Structures in a telecommunications from end, by John A. Lefor

Vol. III, N. 4 November/December 1981

An excursion within the URTN system, the Forth implementation at the Rochester University Computing Center, since 1977. The study exposes the hardware and software decisions that were made in order to build a telecommunication front end.

Complex Analysis in Forth, by Alfred Clark, Jr

Vol. III, N. 4 November/December 1981

This very interesting article (unfortunately without code) introduces to complex analysis in a very natural way (*natural* from the Forth point of view): ZDROP, ZDUP, ZOVER, ZROT, ZSWAP, REZ, IMZ, various math operations like ZEXP, ZSIN, ZCOS, ZTAN, even a function MAP to map curves and functions, and so forth. Built for the Apple II Forth environment.

HEX strip, by Baroudi

Vol. III, N. 4 November/December 1981

Another humorous strip: holidays are always too short, too distant, too expensive...



(Note: reproduced without permission. If its authors should ask for removal, this strip will disappear in the next version.)

(from letter to the Editor) Julian Date converter, by Peter B. Dunckel

Vol. III, N. 5 January/February 1982 [julian.ft]

A converter from a date expressed in MONTH DAY YEAR (as single numbers) on the stack (with YEAR on top), and returning the Julian Day double number. As stated in the article, the algorithm comes from the U.S. Naval Observatory, via an article appeared in the *Astrophysical Journal Supplement Series*, Vol. 41 No. 3, Nov. 1979, pp 391-2. The comment by the Editor expressed my exact feelings: "Really slick! But the algorithm would be hard to explain to most people."

(from letter to the Editor) Convert to Decimal, by Gregg Williams

Vol. III, N. 5 January/February 1982

A simple and useful converter, called CVD, to use decimal numbers when you are in a BASE different from 10. For instance, you need to list block 130 and you are in HEX; thus 130 LIST would list block 304! But if you, as suggested by Gregg, define LIST as CVD LIST, the converter will turn 130 in hexadecimal to 130 in decimal and load the correct block. It works in any BASE that accepts the input. You can find CVD in the library and you can test it using REQUIRE or ENSURE.

A technical tutorial: Table lookup examples, by Henry Laxen

Vol. III, N. 5 January/February 1982

An interesting tutorial about methods of creating tables and looking into them. Simple, elegant, useful.

The Game of Reverse, by M. Burton

Vol. III, N. 5 January/February 1982 [20228-20233]

The Game of Reverse, written for fig-Forth version 1.15, translated to tetra. Amusement for the brain. The algorithm is simple, yet powerful. And it works perfectly with the same results.

The 31 Game, by Tony Lewis

Vol. III, N. 5 January/February 1982 [20151-20164]

The "31 Game". I had to move screen numbers from 51-64 of the original article to 20151-20164, because other sources were in some of the original numbers. Moreover, the program was written using the *micromotion* (c) Forth-79 Version 1.2 for the Apple II, with non-standard screens dimensions (apparently more than 16 lines longer less than 64 characters), so that the transcription was altered both in the source¹³⁴ (adding non-standard commands) and in the formatting (to respect the 16*64 format of tetra).

¹³⁴ The block 20001 (loaded automatically by the first block 20151) contains special additions for Apple II Forth compatibility.

HEX strip, by Baroudi

Vol. III, N. 5 January/February 1982

Another humorous strip: Forth's way to redefine concepts. And life things.



(Note: reproduced without permission. If its authors should ask for removal, this strip will disappear in the next version.)

A video version of Master Mind, by David Butler

Vol. III, N. 5 January/February 1982 [20118-20132]

A wonderful Master Mind game implementation. I had to move screen numbers from 18-32 of the original article to 20118-20132, because other sources were in some of the original numbers. Written in fig-Forth, the conversion was fairly easy. It's an astounding work, because, even if it uses characters, it's nonetheless a graphic program! And I like characters graphic a lot!

Forth Standard Corner, by Robert L. Smith

Vol. III, N. 6 March/April 1982

This article about the Forth-79 Standard `DO...LOOP/+LOOP` works, analyses some problems in the definition of these procedures in the Standard. This article was a great lesson to me. I followed Mr. Smith indication to better `tetra` `DO`, `LOOP` and `+LOOP`. Read the `DO`, `LOOP` and `+LOOP` entries in this manual.

A techniques tutorial: execution vectors, by Henry Laxen

Vol. III, N. 6 March/April 1982 [execvec.ft, die.ft]

This article explains in detail how vectored execution word, by presenting a new enhanced version of `EMIT` (see the didactic file). The example is quite fully compatible with `tetra` (apart some conversions). Mr. Laxen will publish more articles about this, and in next volumes he will appear again. He was (and maybe still is?) one of the more influential experts in Forth, using the most advanced techniques, presented in a very clear language (clear even for me). Note: the `ASSIGN` case is not compatible with `tetra` and was not converted.

Charles Moore's BASIC Compiler Revisited, by Michael Perry

Vol. III, N. 6 March/April 1982

This interesting article shows how to emulate a simple BASIC interpreter in Forth. The compiler used for this program is an advanced fig-Forth version with many unknown non-Standard words, and it uses branching based on the Return Stack technique, and as such not compatible with `tetra`, but it's a wonderful lecture about Forth programming.

A Roundtable on Recursion, by various authors

Vol. III, N. 6 March/April 1982 [s-ack.ft, recurs.ft]

A one-page exposition of two recursive techniques. After the foreword by Leo Brody, the Editor, who explains the recursion concept and the working of the `SMUDGE` fig-Forth word, two studies follow: the first by Christoph P. Kukulies (in file `s-ack.ft`), where the Ackermann recursive function is exposed using the `SMUDGE` word, and the second by Arthur J. Smith (in file `recurs.ft`), with his own definition of the `RECURS` word (predating the `RECURSE` Standard word). The `MYSELF` word of the Editor's section differ qualitatively from the `tetra` version contained in the tlibrary; finally, the two Brodie words `:R` and `R;` here defined are not implementable in `tetra`.

Sieve of Eratosthenes [SIC] in Forth, by Mitchell E. Timin

Vol. III, N. 6 March/April 1982 [20235-20236]

This program computes the sieve of Eratosthenes for the first 1899 prime numbers in first 8190 integers. I had to change the block numbers, because of other blocks there. Here are Mr. Timin's words: "*The enclosed version of Eratosthenes Sieve was written for an implementation of Timin Forth release 3. I was pleased that it executed in 75.9 seconds, as compared to the 85 seconds of fig-Forth. Mine was run on a 4 MHz 2-80 machine, as were the others in the BYTE magazine article. The speed improvement is primarily due to the array handling capability of Timin Forth release 3. FLAGS is created with the defining word STRING; n FLAGS leaves the address of the nth element of FLAGS. This calculation occurs in machine code.*". The STRING word is not standard, of course. I used CREATE and changed the source to adapt to it. Incidentally, on my laptop, the PRIME-TEST is run in 385 milliseconds (0.385 seconds), measured through NTIMER (contained in the tlibrary). Amazing.

Turning the stack into local variables, by Marc Perkel

Vol. III, N. 6 March/April 1982 [argvar.ft]

A perfect utility that shows the power of Forth. I offer you Mr. Perkel's words: "*Occasionally in writing a definition, I find that I need to do unwieldy stack juggling. For example, suppose you come into a word with the length, width, and height of a box and want to return the volume, surface area, and length of edges. Try it! For this kind of situation I developed my ARGUMENTS-RESULTS words...*".

I only add that I had to change a bit the code you find in the included file, because the original code didn't work under tetra. So, I judged unfair to put that code in a block which couldn't host the original thought. In any case, Mr. Perkel, you had a very interesting idea!

CASES continued, by various authors

Vol. III, N. 6 March/April 1982

Some pages back, I wrote that in the CASE contest, the solution proposed by Mr. Eaker was the nearest match to the current word, or better it **is** the current word. This is plainly shown by a couple of versions (one for the 8080/Z80 computer, by, John J. Cassady, the second for fig-Forth by Alfred J. Monroe, an augmented version) that show how well the Eaker solution was received by the Forth community. The presented cases in this article continue with other proposed versions to solve the CASE affair.

Math (monography), by various authors

Vol. IV, N. 1 May/June 1982

My favourite issue! A monography dedicated to math in Fixed-Point. A delightful lecture!

Proposals for Relationals and Multiplication Operators, by Robert L. Smith

Vol. IV, N. 1 May/June 1982

The stainless Mr. Smith offers a tasty appetizer, some fast code and some considerations about operators and signed/unsigned multiplication in Forth.¹³⁵ But the generalized concepts remain valid, and this is a good start for a monography!

Defining words (part. 1/3), by Henry Laxen

Vol. IV, N. 1 May/June 1982 [verbs.ft]

This is the first of a three-parts article about advanced techniques in using CREATE and \fCDOES>. These articles were a mine of information, and leader texts for the developing of tetra. The word EMITTER, being a useful tool, was placed in the tlibrary. Thanks, Mr. Laxen, for these articles.

Fixed-Point Trig by Table-lookup, by John S. James

Vol. IV, N. 1 May/June 1982 [20732-20735]

This is a simple but powerful way of dealing with trig in fixed-point math on Forth-79. It gives the functions SIN and COS that calculate their value basing on tables (screen 20732), in the range 0°10000 (fixed point numbers with four digits). The application in screen 20735 prints a sine wave in the range 0÷360° (degrees). Load the screens and type WAVE.

¹³⁵ The < code in tetra is built in a strong math-view, so that -20000 20000 < returns true; remember also that M* and S->D are found in the tlibrary.

Fixed-Point Trig by Derivation, by J. Bumgarner

Vol. IV, N. 1 May/June 1982 [20036-20037]

Another technique that uses the Taylor-Maclaurin series to derive an algorithm for the sine, cosine and tangent calculation. The functions SIN and COS return a number in the range 0÷10000; the result is thus a fixed point number (the result multiplied by 10000) with four digits after the dot. TAN instead, is a fixed point number (the result multiplied by 100) in the range 0÷30000; the output is thus not compatible with SIN and COS.

Fixed Point Square Roots, by Klaxon Suralis

Vol. IV, N. 1 May/June 1982 [20222-20224]

Calculating square roots of integer numbers (up to 4095) and getting the printing of the square root with three digits of precision? Here is the solution that Mr. Suralis proposes. Being a Forth-79 program, no change was done. I let the redefinition of three words (2DUP, 2DROP and DU<) that Mr. Suralis adds to the program but that **tetra** has as built-in, but this is harmless. Ah, and I added the word --> to screens 20222 and 20223, to let the compiler LOAD them all in one turn.

Fractional Arithmetic, by Leo Brodie

Vol. IV, N. 1 May/June 1982 [20097]

Here's an interesting point of view: fractions using not decimal values, but scaling the integer with 16384. This has a counterpart: numbers are confined to -2..+2 in range, but you can accomplish calculations with fractions! Here is the example in the article (example that works painlessly in **tetra**):

```
7 34 /. 23 99 /. + .F <ret> 0.4381
```

(the real value with modern calculators is 0.4382, but you get the idea!) The algorithm is due (in Brodie's words) to Greg Bailey, who conceived the basic design.

The Cordic Algorithm for Fixed-Point Polar Geometry, by Alan T. Furman

Vol. IV, N. 1 May/June 1982 [20028-20030]

This article, based on the author's presentation at the March 1982 meeting of FIGGRAPH, shows an interesting approach to polar/rectangular conversions, based on the Cordic Algorithm, in a very few steps (2 blocks). This program runs smoothly on **tetra** because it's pure Forth-79. Screen 20030 is added, containing the words of the article that show the power of the algorithm. The algorithm is based on the circular behaviour of the integer value, in the range -32768..32767, which is mapped to the circle angles range -180..180, to give a complete 360° elements (this mapping is called 16bitANGLE). This is one of the most interesting articles in *Forth Dimensions*, because it unveils what is the trigonometric calculation done with the Cordic algorithm, that is the basis of all the functions in modern programming languages and in all electronic calculators.

Quadruple Word Simple Arithmetic, by David A. Beers

Vol. IV, N. 1 May/June 1982 [20400-20412]

The author with my favourite surname! An interesting article about the development of words for treating quadruple numbers (in the range -2⁶³..2⁶³), that is 8 bytes, or 64bit numbers. The article is clear. At page 22, after an ads, a one-page summary (very useful) is given.

Floating Point FORTH?, by Michael Jesch

Vol. IV, N. 1 May/June 1982 [float.ft]

A complete article about the implementation of some words dealing with floating point math using the normal data stack. A very pleasant article written by an expert in Forth. I found an error, though: the operator F/ for the float division is incorrect. So in float.ft I correct it. I hope that Mr. Jesch appreciates.

Let's go to the saloon!, by McLellan & Brodie

Vol. IV, N. 1 May/June 1982

A humorous cartoon by two Forth programmers, which explains in Forth jargon how this saloon works. In Italy, the sign over the door should recite `18 < ABORT`", but U.S.A. are (were?) more rigorous!



(Note: reproduced without permission. If its authors should ask for removal, this cartoon will disappear in the next version.)

Towards a New Standard, by Robert L. Smith

Vol. IV, N. 2 July/August 1982

In this article, Robert L. Smith introduces the affirmation that Forth-79 vocabularies system is weak, and this problem must be faced. His conclusion: *"There are other designs for vocabulary mechanisms. Almost any of them would be an improvement over FORTH-79. In my opinion it is important that the next Standard have a significant improvement in vocabulary structures."* Forth-83 will be the answer to this need. Unfortunately for Forth-79...

Defining words II (part. 2/3), by Henry Laxen

Vol. IV, N. 2 July/August 1982 [**class.ft**]

This is the second of a three-parts article about advanced techniques in using `CREATE` and `DOES>`. In this article the `CLASS` word and its correlated words are presented. The program `class.ft` is a transcript of all the listings, suitable for `tetra`; my advice is to run the program *after* reading the source, and to check the source line by line after run. You'll see the power of this application!

Source Screen Documentation, by Kim Harris

Vol. IV, N. 2 July/August 1982

In this short article, Kim Harris (of the Laxen & Harris, Inc.) presents a small utility that may reveal useful in treating blocks. `OUTLINE` and `!OUTLINE` may be used to inspect blocks contents if a syntax is respected (the only rule is to start the definitions and comments from the start of the line and the body of definitions from second character on. Simple, isn't it?). The two programs are in the `tlibrary`, and can be added to the dictionary with `ENSURE OUTLINE`.

Q T F Quick Text Formatter - Part I, by Leo Brodie

Vol. IV, N. 3 September/October 1982 [**textf.ft**, **20073-20075**]

This is the first of two articles by the great Leo, who wrote this program (that you can find it in `textf.ft` with all the corrections for `tetra`) to print formatted text. The format seems those of the old word assemblers (like `Word-Star` and the like), and reminds me a bit like `groff` or `Latex`. It is a very simple 'language', and it's limited to spacing and indentation, but nonetheless it is another brick to the Forth building.

Note: to execute the program, call `textf.ft` from console, and then execute :

20073 load

to see the formatted text appear on the monitor. Block 20073 contains some aid-definitions by Leo, while block 20074 is a text block created by me.

Forth-79 Compatible LEAVE for FORTH-83 DO..LOOPS, by Klaxon Suralis

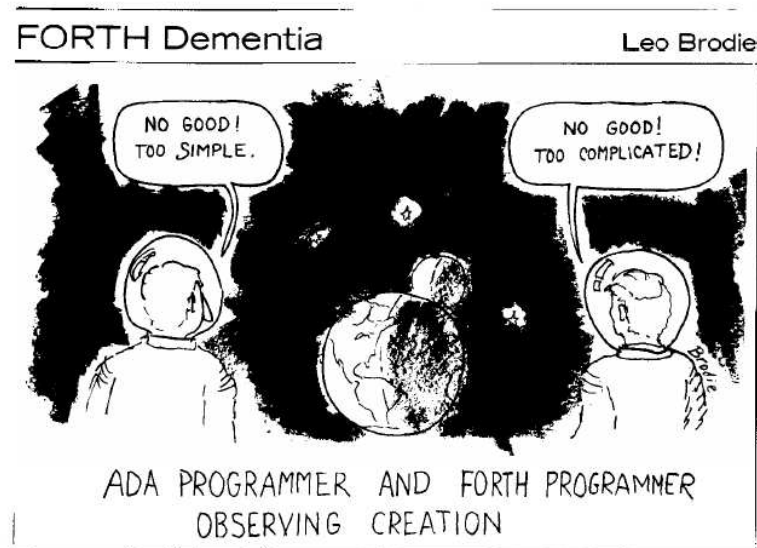
Vol. IV, N. 3 September/October 1982

In this article, Mr. Suralis explains his doubts about the forth-coming Forth-83 implementation of the LEAVE in the DO..LOOP cycles. His conclusion is "*The existing LEAVE usage can work with the new DO..LOOP. We don't have to adopt a direct-jumping LEAVE in order to reap the benefits of boundary-crossing loop termination. If the immediate LEAVE is to become standard, let it do so strictly on the basis of its own merits. I sez [sic] it LEAVES much to be desired*"

Forth Dementia!, by Leo Brodie

Vol. IV, N. 3 September/October 1982

A humorous cartoon by the great Leo.



(Note: reproduced without permission. If Leo Brodie or the heirs should ask for removal, this cartoon will disappear in the next version.)

Defining words III¹³⁶ (part. 3/3), by Henry Laxen

Vol. IV, N. 3 September/October 1982 [poets.ft]

This is the third and last of a three-parts article about advanced techniques in using CREATE and DOES>. In this article the class concept is extended, and it is used to create object classes and sub-classes, that is classes derived by other classes (in this case Mr. Laxen proposes English poets as classes, and their masterpieces are used as a base of data). This technique is not understodd plainly by tetra, so I had to do a bit of workaround; look carefully at the original code (reported in comment) and the code developed for tetra: if you can conceive a better and portable solution, you're welcome...

Q T F Quick Text Formatter - Part II, by Leo Brodie

Vol. IV, N. 4 November/December 1982 [qtf.ft, 21017-21037]

This is Leo Brodie's editor. The program is long, and I have introduced some changes, in order to make it work under tetra. The first is that CTRL-C cannot be used as a 'character', because it's a system code. So I turned it to 'c as a command (letter 'c'). The second is that I wrote the codes for the terminal in block 21017 (Leo didn't provide any), and they should work for any modern terminal. The third is that I changed the code for the DELETE key with the code for BACKSPACE (changing from 8 to 127): this seems more adherent to modern requirements. The version

¹³⁶ Curiously not listed in the first page summary...

on blocks 21018-21037 is complete (you can customize some parts, for instance enable FLUSH by default or enable the snapshot debugger), while the version on file `qtf.ft` is the Continuous Mode with some transformations (for instance, block 21017, which contains the terminal features, was brought to the end). To start the editor from the base block, type:

```
21018 load
```

from `tetra`'s command line. If you start from block 21017, you'll get an error.

I report here the updated version of the Glossary, that appears at page 25, corrected with the changes I introduced and the unsaid in Brodie's work.

Quick Text Formatter Editor Glossary

Command Mode

In this mode, the following keys perform these functions:

i	moves cursor up
j	moves cursor left
k	moves cursor right
m	moves cursor down
J	moves cursor left four characters
K	moves cursor right four characters
a	sets cursor to first character in block
z	sets cursor to last non-blank character in block
c	cuts-off everything to the right of the cursor
n	displays next block
b	displays one block back
e	enters "Enter Mode", allowing you to enter or insert text
r	enters "Replace Mode", allowing you to overwrite text
x	deletes one character, and enters "Delete Mode"
X	the first hit deletes one character and enters "Delete Mode"; from the second hit on deletes four characters at a time
t	takes the text that was most recently deleted with "x", "X" or "c", and inserts it at the current cursor position
RETURN	if in Command Mode, returns to FORTH; otherwise, in any other mode, returns to Command Mode
BACKSPACE	in Enter Mode, deletes the characters at the left; in Delete Mode, cancels the deletion at the left; in any other mode, simply moves the cursor to the left. It does NOT store characters available to 't'.

Keys 'x' and 'X' works cumulatively, that is they store the whole body of removed characters, making them available to 't'.
In any mode, a bell sounds when you type an illegal key.

Warning: Leo's point of view is not the same of common current editors; he wants his rude editor to be a wide 78 characters space, with a variable number of lines, where you type continuously, and the editor takes care of inserting breaks. A counter appears in the low-right corner of the screen to show how many characters were introduced. It's the user's responsibility to avoid exceeding this limit. I warn you, it's stuff for tough programmers!

Representation for Logical True, by Robert L. Smith

Vol. IV, N. 4 November/December 1982

In this small article. Mr. Smith exposes the proposal to turn the logical true value as a bit representation (all bits sets, that is -1) rather than as a truth value (one bit set, that is 1), as in Forth-79. The proposal will be accepted, and the next Forth-83 will have -1 as the true value.

Choosing names, by Henry Laxen

Vol. IV, N. 4 November/December 1982

In this article. Mr. Laxen shows his rules for choosing names; I find that his theoris are appreciable still today. The article is full of advices and is an enjoyable lecture, due to his personal wit.

Parameterized CREATE DOES>, by Henry Laxen

Vol. IV, N. 5 January/February 1983 [**param.ft**, **inform.ft**]

Another interesting article by Henry Laxen, the guru of CREATE; in this series, the CREATE/DOES> mechanism is pushed towards new boundaries. This article in particular treats the execution of the DOES> part choosing the procedure according to a stack structure that holds the execution tokens of the desired procedures, which are effectively pieces of compiled code. The first example (see the file `param.ft`) creates an execution table of the roman numbers printing and, using the index on top of stack, print the corresponding roman: typing for instance 6 ROMAN, will print VI. The second example builds two tables, to print the arabic/roman and arabic/english conversion basing on the two words ENGLISH and ROMAN.

The code was updated to be used plainly in *tetra* (for example, 2* was converted to 2 * because it is missing in the default database, though present in the library) and, since Mr. Laxen compiler was a "Starting Forth" version Forth (a Forth with some enhancements, as he gently admits in the article), I had to change it a bit, but the version in the included files `param.ft` and `inform.ft` should run smoothly on any Forth-79 compliant compiler.

Personal note: these months between Vol. IV, N. 4 and Vol. V, N. 1 (i.e. between November 1982 and June 1983) include the moment in which I became aware of Forth. Just to let you all know.

FORTH as a Teaching Language, by Albert S. Woodhull

Vol. IV, N. 6 March/April 1983

This article describe an interesting experience by professor Woodhull, Ph.D at the Hampshire College in Amherst, Massachusetts (USA). In three weeks of a January Term, he introduced his students to Forth, which was a complete unknown for all of them. Professor Woodhull explains: "*FORTH has some [...] educational advantages [...]. Because it has a direct interpretation capability program modules can be debugged much more easily than in a "compile-only" language like Pascal. Yet FORTH is also a compiler and in a course for advanced students such as I have described FORTH can be used to gain an insight into the compilation process itself...*".

Compilation addresses and Parameter Fields, by Robert L. Smith

Vol. IV, N. 6 March/April 1983

This article throws light into the obscure path that led to the Forth-83 Standard; it actually describes one meeting in which the words FIND and ' (tick) were dissected, and the committee in the end formulated the proposal to adopt FIND and ' for the sole Compilation Addresses in runtime, to introduce ['] as the compilation side of ' and to introduce >BODY (at this stage with the temporary name of BODY). Illuminating.

Algebraic Expression Evaluation in FORTH, by Michael Stolowitz

Vol. IV, N. 6 March/April 1983

This article introduces one interesting program to evaluate algebraic expressions involving the basic operators * (times), / (divided), + (sum), - (subtraction), < and > (lesser than and greater than), = (equality operator), plus AND, OR and NOT as logical operators. Unfortunately, the code is already in Forth-83 (or in a close enough version), and I couldn't find a translation suitable for *tetra*. So this remains an exercise for the ones who want to apply.

U/ Bug Fumigated [Technotes, pag. 35], by K.G. Lander

Vol. V, N. 1 May/June 1983

In these few lines (BTW, this is the last issue with Leo Brodie as editor), Mr. Lander critics one Jack Haller for his CODE solution (called U/MOD) for a bug he claimed to have found in U/ (the fig-Forth word for U/MOD, the unsigned division between an unsigned double and an unsigned single number, leaving the unsigned dividend and the unsigned remainder). Mr Lander shows that Haller's solution fails in dealing with the following case:

```
1048576. 65535 U/MOD U. U.
```

which returned 31 and 31 rather than 16 and 16. I'm proud to say that **tetra** behaves exactly as Mr. Lander said U/MOD should do! I believe that, since Mr. Lander found this bug, the whole line must be put in the test suite, and so I did.

Forth in the Arts

Vol. V, N. 1 May/June 1983

This article covers some topics about digital imaging made with a 6502 FORTH (AED 512 high resolution color system) by Howard Pearlmuter, Santa Cruz, California, and electronic music made on a 6502 FORTH (Ohio Scientific system with synthesizers with digital logic) by Allen Strange, San Jose State University. The central argument is that both results are gained through Forth; in particular, the second about music was a system with "*...a language called MASC-Metalanguage for Analog Synthesizer Control. It's a series of about 25 words that lets us do anything we want to do. For instance, we can cause the synthesizer to play a major scale, play a minor scale, play it backwards. We can randomize the notes in a major scale according to some format. We can create timing structures in musical terms, such as commands called WHOLE-NOTE and HALF-NOTE*". Very very interesting.

3-D Animation, by Paul Lutus and Phil Thompson

Vol. V, N. 1 May/June 1983

This article is about graphic animation, made with an Apple with GraForth. Thinking that such topic dates back to 1983, I believe Forth programmers did magic! The core of the article explains that with a small set of low-level words, the high level words for manipulation of images can be built in a very natural way through the colon mechanism, common in Forth.

Double-Precision Math Words, by L. H. Bieman

Vol. V, N. 1 May/June 1983

Complete with screens! In case your Forth-79 system has no Double-precision words... Three agile screens. I didn't copy it, because *tetra* has all these words (or they are in the library).

Interview with Charles Moore, by Martin Ouverson (the new editor)

Vol. V, N. 2 July/August 1983

A long, pleasant and cheerful interview with the founder of Forth. Themes like productivity, Moore's experiences, idolatry, education in Forth, how he spent his free time, prophetic thoughts about the influence of computers in our lives... His conclusions: "*You have to get inside the world of software, the world of imagination; and you create your own world. You create your own problems and so it is almost frightening the amount of power you've got. I've always had the sort of dream - I've always favored software over hardware. The hardware is the nuisance you have to put up with in order to have software. If you want to talk about a religious aspect of FORTH, it is to say, "Do you need hardware? Can you conceive of a way of representing these ideas, of making these castles in the air without any particular underpinning?" I'm sure that [micro]chips will get down to molecular size, but there is still going to be that matter at the core of things. Can you take energy fields, somehow, and weave them to make software? Maybe that's what the layers and layers of operating systems and languages are doing, taking you so far away from the hardware that you forget it is even there*". I completely agree still nowadays.

Recursive Sort on the Stack, by Richard H. Turpin

Vol. V, N. 2 July/August 1983 [20045]

This article shows how to sort any numbers on the stack and print them in increasing order (the stack is empty afterwards). The program works perfectly in **tetra**, in grace of a REQUIRE MYSELF. This program should work

painlessly both in any fig-Forth or Forth-79 interpreter (and maybe in all further versions). The version contained in the block 20045 is the one using FLAG.

3-D Animation

Vol. V, N. 1 May/June 1983

<TODO>

The R65F11 FORTH Chip, by Randy Dumse

Vol. V, N. 2 July/August 1983

This article deals with the description of a microchip for a computer architecture that has Forth hardwired in its circuit. In fact, *"The kernel¹³⁷ of FORTH in ROM consists of 133 words available for user definitions. These words are headerless, kept in the format of code field address followed by the parameter fields. About half of the definitions are machine coded. The remainder are high-level definitions. A typical FORTH definition executes in under 100 microseconds (BASIC is ten to a hundred times slower). Although a micro monitor is included in the internal ROM as part of the operating system, programming the R65F11 usually requires the use of the R65FR1 development ROM. This port contains the rest of FORTH, the non-runtime words and the heads of the words from the kernel. Many utilities specifically designed for easy development of stand-alone programs are also added..."*. And more: *"...Programmers of dedicated applications will find the selection of words rich in content. All the math and stack functions of FIG-FORTH and several double-number extensions from FORTH-79 are included. The runtime constructs of DO, LOOP, BEGIN, WHILE, UNTIL programming structures and essentially all I/O words are in internal ROM. Even the low-level disk handlers below R/W are in the single-chip computer. This allows about 4 mega-bytes of mass storage by adding a 1793-type floppy disk controller and associated support circuitry"*.

Why did this chip suffer oblivion? Probably because this article (August 1983) was published on the eve of the advent of Forth-83 that, by changing the PICK, ROLL commands, altering the DO . . LOOP constructs, the division type, the true flag, the ' (tick) and FIND words, and in general the construction of a single word, made this chip instantly antique. It bears some interest because Leo Brodie, in his book "Thinking Forth", cite it as a perspective, but it actually has rarely been implemented, IMHO. It's anyway a very generous and dedicated effort, to make a speedy environment that could be used directly as a forth operating system.

The articles in the volumes that followed (very interesting as well) deal most with Forth-83 and beyond, and so they are not remembered here. But an exception must be made... Read on.

China's National Forth Examination, by C.H. Ting (transl)

Vol. XIV, N. 2 June/August 1992

Here is the foreword: *"Last year, the People's Republic of China administered its first - and, as far we know, - the world's first national test of Forth knowledge and expertise. In a huge country where positions in industry and academia are earnestly sought by many candidates, and where the results of formal examination can be decisive factors shaping one's future, the test assumes great significance in the lives of Forth programmers there. We invite you to measure your own Forth expertise by taking the same examination. Those who would like to have their answers checked can send them to Dr. Ting at 156 - 14th Avenue, San Mateo, California 94402"*.

Needless to say, the test is tough. And it deserve at least a mention here, even if I confess that it's too tough for me¹³⁸.

In the files/ directory (located in the installation folder) you will find other Forth programs, by various authors (me included), that I added to this package and that don't directly come from *Forth Dimensions* transcripts. They are, among all:

¹³⁷ Sic.

¹³⁸ No particular sympathy for China and its people, but I admire their schools system. And this test shows that I'm right: the one that passed the whole four-parts test (including questions about the English language) probably now is a senior programmer in some industry... it remains to know in which country, though...

- * `create.ft`: this program shows how `CREATED` words may or may not inherit the `DOES>` section according to the used creator-word (written by me).
- * `electron.ft`: the SED editor for Forth-79, edited and converted for **tetra** by Ian Jones. Documentation is in the file `electron.txt`.
- * `fix.ft`: how to implement fixed point math (written by Alan Winfield).
- * `iftrue.ft`: the library for the three `EXEC-ONLY` words `IFTRUE`, `OTHERWISE` and `IFEND`, described in the Reference Words Section of the Forth-79 Standard (written by me). These words work only if they are all in the same input line.
- * `open.ft`: how to use `OPEN-FILE`, `CLOSE-FILE`, `READ-LINE` and `WRITE-LINE` (file dictionary words addenda)
- * `stepdown.ft`: simulation of a downward `DO. .LOOP`, using the readdressing of the program counter (originally written by Alan Winfield for fig-Forth, adapted for **tetra** by me). This program does not use `HERE` but uses `PC`.
- * `sudokiller.ft`: gigantic solution of sudoku grids (written by David Stevenson, and based on Daniele Mazzocchio's `sudokiller.asm`, adapted for **tetra** by me).
- * Many others...

These programs should be studied before and after execution; you are free to change them and try other solutions suitable for **tetra**. If they are interesting, send them to me: I'll add them to the next release, if notable.

If you have written other programs in Forth-79 that **tetra** can crunch, and you want them to be part of the next distribution, send them to me. I'll put them in the package the ones that I will find interesting.

9. CONCLUSIONS

The great effort I reversed into **tetra**, spread among many years, should not be misunderstood. I didn't want to build neither a SwiftForth competitor, nor a GForth 79-sided clone, nor a Polyforth successor, nor an R-Forth cousin or the like. My intents are historic. Forth-79 was a milestone in computer programming and I wanted to honor this. **tetra** is not to be considered a professional environment, though it can be used profitably by anyone, for learning purposes, for playing, for anything. I hope you like it.

This document and the man page were started in 2011 on a Mac OS X 10.4 Tiger emacs and completed in 2025 on a Linux PC with openSuse Tumbleweed.

Two things are still to be said.

The first is: I hope you like **tetra**.

The second is: it's GPL, enjoy! ♥

APPENDIX A: Internals

This appendix, largely intended for enthusiasts only, covers some technical data describing the compilation processes of *tetra*. Each dictionary entry is compiled following the scheme of Fig. 2 found at the beginning of this manual, and here replicated:

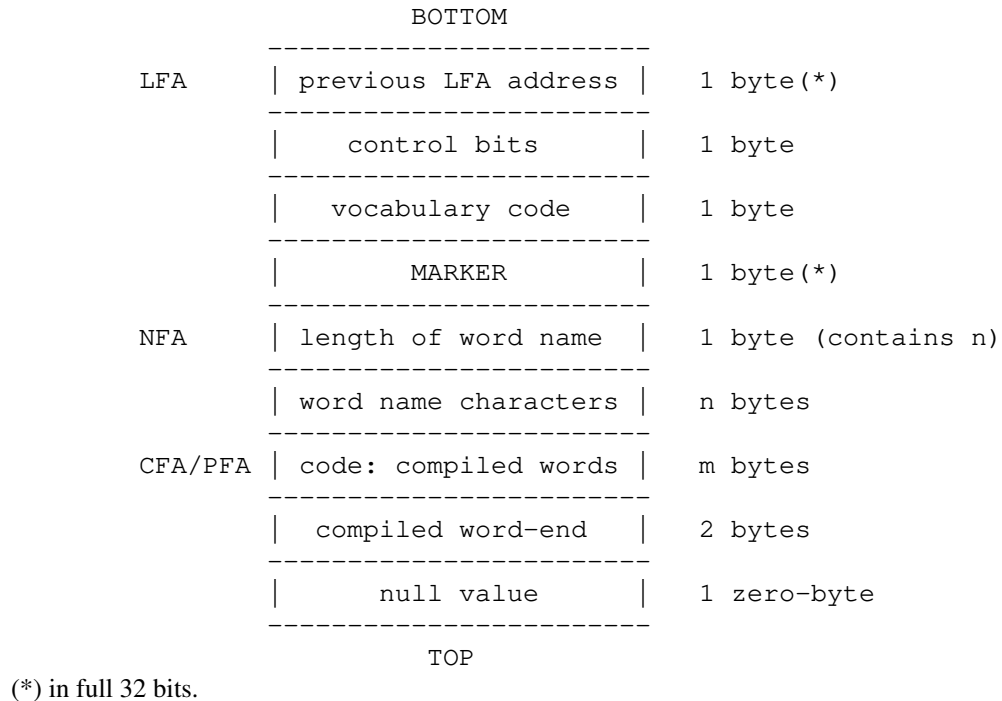


Figure #2

This scheme is personal, and maybe does not reflect the classic Forth-79 scheme. My personal design is characterized by a strong division between the word data (the three data before the MARKER), the word name (the count and the characters following the MARKER, in a classic packed string) and the execution section. This design allows for a simple mechanism for running a word, which can be compiled via its link address from which everything else is derived. Here follow some explanations about all the sections of the compilation scheme:

- * The 'previous LFA address' is a link to the word defined immediately earlier in the dictionary. It's a 32-bit address, used only internally.
- * The 'control bits' over a byte define the standard behavior of a word; the byte is structured as follows:

Definition	Bit	Value	Note
no bits set	-	0	no properties
precedence bit	0	1	word is immediate
smudge bit	1	2	word is under compilation
buffered bit	2	4	word is buffered ¹³⁹
compile bit	3	8	word is compile-only
defining bit	4	16	word is a defining word
addcomp bit	5	32	word needs special compilation ¹⁴⁰
execution bit	6	64	word cannot be compiled
unused bits	7+31		future expansion
- * The 'vocabulary code' contains an index that uniquely defines the vocabulary to which the word belongs to. The vocabulary characteristics are contained in the vocabulary definition word itself.

¹³⁹ A buffered word expects characters in the Input Buffer. At present, this characteristic is signaled but not used.

¹⁴⁰ User words have this bit unset. Built-in words have it generally set.

- * The name is stored in a packed array, with the length in the first byte. All letters are turned to lowercase, before storing. This assures uniformity in data, but this is no problem in using words, because Forth is case-insensitive (Word is like word and like WORD). Use the declarations you like best and call then the way you like best.
- * The compiled code follows, and each unit is composed by a two bytes scheme: in the first byte there's a control code, followed by a number of bytes that contain the data.

Control codes are listed in fig. #3.

Hex	Decimal	Character ¹⁴¹	Identifies a	Followed by
49	73	I	16-bit number	2 x 8-bit
44	68	D	32-bit number	4 x 8-bit
64	100	d	16-bit sys. var.	2 x 8-bit
76	118	v	16-bit variable	2 x 8-bit
56	86	V	32-bit variable	4 x 8-bit
6B	107	k	16-bit sys const.	2 x 8-bit
63	99	c	16-bit constant	2 x 8-bit
43	67	C	32-bit constant	4 x 8-bit
65	101	e	compiled sys. var.	1 x 32-bit
DC	220	Ü	address execution	1 x 32-bit
E6	230	æ	code execution	1 x 32-bit

Figure #3

Some observations:

- * System variables (all 16-bit) in the dictionary are coded with 'd' (100 decimal), followed by two bytes (that form the value cell); when compiled, for speed purposes, they are coded with 'e' (101 decimal), followed by their link address in 32-bit.
- * Constants are used with their name, and their value is returned. When compiled, constant do not compile their word-code or link address, but directly their value. This is common to all compilers, and is faster than retrieving the constant value on runtime at each invocation.
- * The words are compiled in the following modes:
 - A builtin word is coded with id E6 followed by its special and unique word-code; this assures a faster search during runtime.
 - A user word is coded with id DC followed by its LFA; this mechanism assures to find it immediately, calculate its CFA and start its execution, all in few fast steps involving no multiplications.
 - A vocabulary compiled with [COMPILE] (immediate or not) is coded with id DC followed by its LFA.
 - The compilation of words through COMPILER follows the above rules (user words are compiled with DC and builtin words are compiled with E6).
- * Some words are compiled along with the data they treat. For instance, the ." word is stored with id E6 followed by 36 (the word-code of ."), followed by the packed string which is the object of compilation. The same applies to SLITERAL.
- * About the word-end coding:
 - The word-end for built-in words is marked by an empty byte.
 - The word-end for user words is marked by E6 FF (decimal 230 255) plus an empty byte.

As an example, take a look at figs. #4, #5, #6 and #7, where the built-in word DUP, the system variable SCR, the

¹⁴¹ I introduced these characters values identifiers to make them immediately detectable in a memory DUMP.

system constant FIRST and a user variable YAP24 are compiled. In the first, since there is no parameter, the PFA has no meaning; in the second through fourth (supposing that SCR holds the value of 100 and YAP24 2100), since there is no code, the CFA has no meaning. All the following schemes are produced by the program DISASS whose source can be found in the files/ directory in the tetra package.

	MEM.	CODE	DESCRIPTION

LFA	0A8F	00000A83	link to previous word (32-bit)
	0A90	00000020	control bits
	0A91	00000000	vocabulary code (0 = FORTH)
	0A92	0AACAADA	marker (32-bit)
NFA	0A93	00000003	number of characters in name
	0A94	00000064	letter d
	0A95	00000075	letter u
	0A96	00000070	letter p
CFA	0A97	000000E6	id for builtin words
	0A98	0000003B	code of DUP
	0A99	00000000	null value

Figure #4: DUP (dictionary word)

	MEM.	CODE	DESCRIPTION

LFA	004D	00000041	link to previous word (32-bit)
	004E	00000000	control bits
	004F	00000000	vocabulary code (0 = FORTH)
	0050	0AACAADA	marker (32-bit)
NFA	0051	00000003	number of characters in name
	0052	00000073	letter s
	0053	00000063	letter c
	0054	00000072	letter r
CFA	0055	00000064	id for 16-bit system variable
PFA	0056	00000064	16-bit low part
	0057	00000000	16-bit high part
	0058	00000000	null value

Figure #5: SCR (system variable)

	MEM.	CODE	DESCRIPTION

LFA	0E88	00000E77	link to previous word (32-bit)
	0E89	00000000	control bits
	0E8A	00000000	vocabulary code (0 = FORTH)
	0E8B	0AACAADA	marker (32-bit)
NFA	0E8C	00000005	number of characters in name
	0E8D	00000066	letter f
	0E8E	00000069	letter i
	0E8F	00000072	letter r
	0E90	00000073	letter s
	0E91	00000074	letter t
CFA	0E92	0000006B	id for 16-bit system constant
PFA	0E93	00000000	16-bit low part
	0E94	000000E4	16-bit high part
	0E95	00000000	null value

Figure #6: FIRST (system constant)

	MEM.	CODE	DESCRIPTION
LFA	226F	0000224D	link to previous word (32-bit)
	2270	00000000	control bits
	2271	00000000	vocabulary code (0 = FORTH)
	2272	0AACAADA	marker (32-bit)
NFA	2273	00000005	number of characters in name
	2274	00000079	letter y
	2275	00000061	letter a
	2276	00000070	letter p
	2277	00000032	literal 2
	2278	00000034	literal 4
CFA	2279	00000076	id for 16-bit variable
PFA	227A	00000034	16-bit low part
	227B	00000008	16-bit high part
	227C	00000000	null value

Figure #7: YAP24 (user variable)

Now, let's consider the following small program:

```
: MYDUP  DUP DUP ;
: DOUBLE MYDUP + ;
```

Previous words are compiled this way:

	MEM.	CODE	DESCRIPTION
LFA	227D	0000226F	link to previous word (32-bit)
	227E	00000000	control bits
	227F	00000000	vocabulary code (0 = FORTH)
	2280	0AACAADA	marker (32-bit)
NFA	2281	00000005	number of characters in name
	2282	0000006D	letter m
	2283	00000079	letter y
	2284	00000064	letter d
	2285	00000075	letter u
	2286	00000070	letter p
CFA	2287	000000E6	id for builtin words
	2288	0000003B	code of DUP
	2289	000000E6	id for builtin words
	228A	0000003B	code of DUP
	228B	000000E6	id for builtin words
	228C	000000FF	END-OF-WORD code
	228D	00000000	null value

Figure #8: MYDUP (user program)

	MEM.	CODE	DESCRIPTION
LFA	228E	0000227D	link to previous word (32-bit)
	228F	00000000	control bits
	2290	00000000	vocabulary code (0 = FORTH)
	2291	0AACAADA	marker (32-bit)
NFA	2292	00000006	number of characters in name
	2293	00000064	letter d
	2294	0000006F	letter o
	2295	00000075	letter u
	2296	00000062	letter b
	2297	0000006C	letter l
	2298	00000065	letter e
CFA	2299	000000DC	id for calling address
	229A	0000227D	MYDUP execution address
	229B	000000E6	id for builtin words
	229C	00000097	code of +
	229D	000000E6	id for builtin words
	229E	000000FF	END-OF-WORD code
	229F	00000000	null value

Figure #9: DOUBLE (user program)

The next figure is an example of a CREATE word; the following example is performed by a console command as such:

```
CREATE ARRAY
```

This is a simple declaration, followed by no memory allocation. It's disassembling is:

	MEM.	CODE	DESCRIPTION
LFA	22A0	0000228E	link to previous word (32-bit)
	22A1	00000000	control bits
	22A2	00000000	vocabulary code (0 = FORTH)
	22A3	0AACAADA	marker (32-bit)
NFA	22A4	00000005	number of characters in name
	22A5	00000061	letter a
	22A6	00000072	letter r
	22A7	00000072	letter r
	22A8	00000061	letter a
	22A9	00000079	letter y
CFA	22AA	000000E6	id for builtin words
	22AB	000000EB	NOOP code
	22AC	00000049	id for 16-bit number
	22AD	000000B2	16-bit low part
	22AE	00000022	16-bit high part
	22AF	000000E6	id for builtin words
	22B0	000000FF	END-OF-WORD code
	22B1	00000000	Parameter field start (reserved): param #0
PFA	22B2	00000000	End of parameter field (Next word's start)

Figure #10: ARRAY (user structure)

The address 22B2 (the next after reserved 22B1) is not part of ARRAY, unless some memory allocation is done, using ALLOT or the comma words that store particular values reserving memory space. The 22B3 address is returned

by ARRAY word in virtue of the 16-bit integer code hardwired in it (as can be easily seen in the disassembled version). So, in a line like:

```
CREATE ARRAY
ARRAY .
```

the value of HERE is output; any subsequent compilation would start from the parameter field returned by ARRAY; that's why compiling words like ALLOT, , (comma) and C, (C-comma) are used, because they advance HERE and *protect* data.

If CREATE is used in conjunction with DOES>, the compilation scheme is structured as follows; suppose we have a defining word like this:

```
: HELPER
  CREATE ." Helper created"
  DOES> ." How can I help?" ;

HELPER STEWARD
```

When HELPER STEWARD is invoked, the string "Helper created" is printed. When STEWARD is invoked, the string "How can I help?" is printed. The two words HELPER and STEWARD are compiled as follows:

	MEM.	CODE	DESCRIPTION
	LFA 22B2	000022A0	link to previous word (32-bit)
	22B3	00000010	control bits
	22B4	00000000	vocabulary code (0 = FORTH)
	22B5	0AACAADA	marker (32-bit)
	NFA 22B6	00000006	number of characters in name
	22B7	00000068	letter h
	22B8	00000065	letter e
	22B9	0000006C	letter l
	22BA	00000070	letter p
	22BB	00000065	letter e
	22BC	00000072	letter r
	CFA 22BD	000000E6	id for builtin words
	22BE	00000027	CREATE code
	22BF	0DAAFAAC	does-mark
	22C0	000022D4	DOES> address
	22C1	000000E6	id for builtin words
	22C2	00000036	." code
	22C3	0000000E	string length
	22C3-22D1		"Helper created"
	22D2	000000E6	id for builtin words
	22D3	000000ED	(DOES>) code
	22D4	000000E6	id for builtin words
	22D5	00000036	." code
	22D6	0000000F	string length
	22D6-22E5		"How can I help?"
	22E6	000000E6	id for builtin words
	22E7	000000FF	END-OF-WORD code
	22E8	00000000	null value

Figure 11: HELPER (defining word)

MEM.	CODE	DESCRIPTION
LFA 22E9	000022B2	link to previous word (32-bit)
22EA	00000000	control bits
22EB	00000000	vocabulary code (0 = FORTH)
22EC	0AACAADA	marker (32-bit)
NFA 22ED	00000007	number of characters in name
22EE	00000073	letter s
22EF	00000074	letter t
22F0	00000065	letter e
22F1	00000077	letter w
22F2	00000061	letter a
22F3	00000072	letter r
22F4	00000064	letter d
CFA 22F5	000000E6	id for builtin words
22F6	000000EB	NOOP code
22F7	00000049	id for 16-bit number
22F8	0000000F	16-bit low part
22F9	00000023	16-bit high part
22FA	000000E6	id for builtin words
22FB	00000036	." code
22FC	0000000F	string length
22FC-230B		"How can I help?"
230C	000000E6	id for builtin words
230D	000000FF	END-OF-WORD code
230E	00000000	null value
230F	00000000	Parameter field start (reserved): param #0
PFA 2310	00000000	End of parameter field (Next word's start)

Figure 12: ROOM (created word - through HELPER)

The does-mark in HELPER serves as a hook for the address of DOES>, from which the words that follow are copied to the destination word. Again, when STEWARD is invoked, the address 230F is pushed to the stack, for saving address. This can be useful or not, but as in the previous case, if some values are stored here, they can be accessed through the pushed value, in a very natural way.

Next, as an example of a 32-bit item, here's the decompilation of a 2CONSTANT element, obtained via

```
3.14159265 2CONSTANT PI
```

MEM.	CODE	DESCRIPTION
LFA 230F	000022E9	link to previous word (32-bit)
2310	00000000	control bits
2311	00000000	vocabulary code (0 = FORTH)
2312	0AACAADA	marker (32-bit)
NFA 2313	00000002	number of characters in name
2314	00000070	letter p
2315	00000069	letter i
CFA 2316	00000043	id for 32-bit constant
PFA 2317	000000B9	32-bit high part: 16-bit low part
2318	00000012	32-bit high part: 16-bit high part
2319	0000FFA1	32-bit low part: 16-bit low part
231A	0000FFB1	32-bit low part: 16-bit high part
231B	00000000	null value

Figure 13: PI (user 32-bit constant)

Notice the two negative values in the 32-bit low part expressed as high values in HEX (the two components of the double constant are 4793 and -20319).

See now the decompilation of a peculiar word, the RECURSE word contained in the tlibrary¹⁴².

¹⁴² The RECURSE word contained in the tlibrary is not portable, because it uses the peculiar word 32, to store in a unit a 32-bit number (operation permitted by tetra given its memory structure). The disassembling here reported should so be taken as a mere example.

MEM.	CODE	DESCRIPTION
LFA 232E	0000231C	link to previous word (32-bit)
232F	00000001	control bits
2330	00000000	vocabulary code (0 = FORTH)
2331	0AACAADA	marker (32-bit)
NFA 2332	00000007	number of characters in name
2333	00000072	letter r
2334	00000065	letter e
2335	00000063	letter c
2336	00000075	letter u
2337	00000072	letter r
2338	00000073	letter s
2339	00000065	letter e
CFA 233A	000000E6	id for builtin words
233B	0000002D	code of DECIMAL
233C	00000044	id for 32-bit number
233D	00000000	32-bit high part: 16-bit low part
233E	00000000	32-bit high part: 16-bit high part
233F	000000DC	32-bit low part: 16-bit low part
2340	00000000	32-bit low part: 16-bit high part
2341	000000E6	id for builtin words
2342	00000001	code of 32,
2343	00000065	id for compiled 16-bit system variable
2344	000000C1	LAST link address
2345	000000E6	id for builtin words
2346	00000065	code of @
2347	000000DC	id for calling address
2348	0000231C	S->D execution address
2349	000000E6	id for builtin words
234A	00000001	code of 32,
234B	000000E6	id for builtin words
234C	000000FF	END-OF-WORD code
234D	00000000	null value

Figure #14: RECURSE (user program)

Here it is the decompilation of an user vocabulary TEST, created through the invocation of:

VOCABULARY TEST

	MEM.	CODE	DESCRIPTION
	LFA 234E	0000232E	link to previous word (32-bit)
	234F	00000000	control bits
	2350	00000000	vocabulary code (0 = FORTH)
	2351	0AACAADA	marker (32-bit)
NFA	2352	00000004	number of characters in name
	2353	00000074	letter t
	2354	00000065	letter e
	2355	00000073	letter s
	2356	00000074	letter t
CFA	2357	000000E6	id for builtin words
	2358	000000E9	(VOCABULARY) code
	2359	000000E6	code for builtin words
	235A	000000FF	END-OF-WORD code
PFA	235B	00000002	Vocabulary id low part
	235C	00000000	Vocabulary id high part
	235D	000000F8	Vocabulary link address low part
	235E	00000001	Vocabulary link address high part
	235F	00000000	null value

Figure #15: TEST (user vocabulary)

A word defined through VOCABULARY is not a common word, in *tetra*; it is a special word which contains the code of a runtime procedure, called (VOCABULARY) that, when executed, will set the compiled number as CONTEXT, the compiled number of the vocabulary and the link to the father vocabulary. Thus, its PFA part is detached by the section from CFA.

The next figure is an example of the control flow words IF . . ELSE . . THEN, because they contain even the jump address built into. For instance, suppose to disassemble this word:

```
: EVALTEMP    ( n -- f )
  [COMPILE] FORTH
  100 > IF ." TOO HOT!" ELSE ." GOOD!" THEN CR ;
```

In the compilation phase, the words IF, ELSE and THEN work together to define their proper position in the compile code to ease the jumping process, and compile into the object word their run-time parts (which have the same name enclosed in parentheses); actually:

MEM.	CODE	DESCRIPTION
LFA 2360	0000234E	link to previous word (32-bit)
2361	00000000	control bits
2362	00000000	vocabulary code (0 = FORTH)
2363	0AACAADA	marker (32-bit)
NFA 2364	00000008	number of characters in name
2365	00000065	letter e
2366	00000076	letter v
2367	00000061	letter a
2368	0000006C	letter l
2369	00000074	letter t
236A	00000065	letter e
236B	0000006D	letter m
236C	00000070	letter p
CFA 236D	000000DC	id for calling address
236E	000001EA	FORTH execution address
236F	00000049	id for 16-bit number
2370	00000064	16-bit low part
2371	00000000	16-bit high part
2372	000000E6	id for builtin words
2373	0000006A	code of >
2374	000000E6	id for builtin words
2375	000000F6	(IF) code
2376	00002382	?branch address
2377	000000E6	id for builtin words
2378	00000036	." code
2379	00000008	string length
2379-2381		"TOO HOT!"
2382	000000E6	id for builtin words
2383	000000F3	(ELSE) code
2384	0000238D	branch address
2385	000000E6	id for builtin words
2386	00000036	." code
2387	00000005	string length
2387-238C		"GOOD!"
238D	000000E6	id for builtin words
238E	000000FB	(THEN) code
238F	000000E6	id for builtin words
2390	00000026	code of CR
2391	000000E6	id for builtin words
2392	000000FF	END-OF-WORD code
2393	00000000	null value

Figure #16: IF-ELSE-THEN (control flow words)

Since the run-time (IF) knows the address of (ELSE), in case of false flag it passes control to (ELSE) (along with a signal on the control flow stack); in case of true flag, (ELSE) on its turn passes control to (THEN), because it knows that (IF) has already processed the words in between. (THEN) has no special address, because it's only a terminator point for the control flow process, that has only the duty of getting rid of the signal left by (IF) or (ELSE) on the control flow stack. The sequence [COMPILE] FORTH, necessary because FORTH is immediate, and thus it's compiled by address, and not by reference, as specified in the premises.

The classic decompiled version of this program is:

```

: evaltemp
  forth 100 > (if)
    ." TOO HOT!"
  (else)
    ." GOOD!"
  (then) cr
;
ok

```

This confirms what DISASS has shown, that the run-time version of the commands dedicated to the control flow are compiled, not their compile-time versions. If you wonder what (IF), for instance, looks like, here is its disassembling:

	MEM.	CODE	DESCRIPTION
	LFA 0B43	00000B39	link to previous word (32-bit)
	0B44	00000028	control bits
	0B45	00000000	vocabulary code (0 = FORTH)
	0B46	0AACAADA	marker (32-bit)
	NFA 0B47	00000004	number of characters in name
	0B48	00000028	character (
	0B49	00000069	letter i
	0B4A	00000066	letter f
	0B4B	00000029	character)
	CFA 0B4C	000000E6	id for builtin words
	0B4D	000000F6	(IF) code
	0B4E	00000000	null value

Figure #17: (IF) word (run-time control flow)

Starting from the Beta version, the *state* of any execution is a two-level variable:

- 1 the first level, linked to the variable STATE, is set by : (colon) to compile state and reset by ; (semicolon) to interpret state. The two words [and] respectively restore the interpret state and the compile state. Any error will also reset the interpret state.
- 2 The execution of immediate words is coordinated by the second level of *state*, which is set to the interpret state in case an immediate word is invoked; this second level of *state* has meaning only for **tetra** and the user can forget about it. Each need is satisfied by the STATE variable, and the execution of immediate words can be driven by the STATE control mechanism, which is common in ANSI:

```

STATE @ IF
  <compile section>
ELSE
  <interpret section>
THEN

```

As you have certainly noted, the DOES> section of a word is copied, not referenced to the created word. This assures that the created word is a complete and independent entry. This feature is thus a design feature, that differ from what exposed in the Standard. Moreover, as you have certainly noted, in **tetra** the word's name is stored in full.

I think the previous design features have some advantages and some disadvantages:

- Advantages are an easier search, an easier compilation, and an easier understanding of programs, names positions and addresses when dumping memory; something that often turns out to be essential during debugging.
- . Disadvantages consist in a small delay in execution, while searching for words names in the dictionary, and (probably) a small increment of memory occupation. Something I can live with.

APPENDIX B: The Test Suite

Starting from the Beta Version, *tetra* is distributed with the Forth-79 Standard Test Suite. This suite is composed by the three files:

- 1 `test-tools-79.ft`, which contains the basic utilities and the main variables definitions;
- 2 `testsuite.ft`, which contains several tests to assert the correctness of the Forth-79 compiler that executes the test; this test may be run in strict-mode or not, it does not matter;
- 3 `testsuite-add.ft`, which contains several more tests that assert the correctness of the builtin non-Forth-79 words or words built in Forth-79 that emulate the ANSI words; this test should not be run in strict-mode.

The tests have been verified against the results obtained with GForth (using the symmetric division words of ANSI), and after a thorough research in the Forth-79 Standard book.

The test suite is modeled on the John Hayes test suite¹⁴³ for what about the core words, and on the Steve Palmer block test suite¹⁴⁴, for the block tests, to which I added tests for `VOCABULARY`, `DEFINITIONS`, `CONTEXT` and `CURRENT`.

It seems needless to say, but the tests were very useful in discovering bugs in the Alpha version and also in understanding some of the behaviors of Forth, whether they belonged to Forth-79 or ANSI Forth. In addition, many words in the library were modified to satisfy the tests.

I created these tests not for the casual user, but for the enthusiast programmer who loves Forth-79 and I expect her/him to report any invalid test to me and indicate any missing tests along with the necessary output that satisfies the Forth-79 protocol, to make the tests stronger and more precise.

¹⁴³ available here:

<https://github.com/gerryjackson/forth2012-test-suite/blob/master/src/core.fr>

¹⁴⁴ available here:

<https://github.com/gerryjackson/forth2012-test-suite/blob/master/src/blocktest.fth>

APPENDIX C: The floating point system

Starting from the Beta Version, **tetra** is distributed with the float system written entirely in Forth-79.

You can find the files in the `/floats` subfolder of the installation directory, after unpackaging. They are:

- the file `float-ext.ft` which contains the main files.
- the file `float_test.ft` which contains some tests that can show the precision of this environment
- the documentation in pdf, the file `float-ext_manual.pdf`, along with the source `float-ext.man.roff`.

The float system mimics the ANSI floats, but adds a lot of new commands, both in the tools section and in the mathematical library. The manual offers many perspectives about the usefulness of the mathematical functions implemented. All the words that are also in the ANSI version operate exactly as in ANSI. <TODO trig in degrees?>

I must thank OpenAI and ChatGPT, which helped me in finding new strategies for the corpus of float words, and in finding useful information about the various algorithms.

I hope you will like my effort to add more value to **tetra**!

APPENDIX D: Starting setup

To end up, here is the default output (from a fresh start) of the 1 INFO execution word (<user> stands for the user's name) using a :

```
1 info
```

```
tetra running mode: Forth-79
```

```
Memory in use:      4344 bytes
```

```
tetra memory settings (in bytes)
```

Memory sectors	Width [bytes]	Start [addr]
Start of memory	1	0 (reserved)
Program memory	55807	1 (available 51463)
Terminal input buffer	1024	55808
WORD buffer	1024	56832
Scratch area	512	57856
Screen buffer 1	1024	58368
Screen buffer 2	1024	59392
Screen buffer 3	1024	60416
Data stack	2048	61440
Return stack	2048	63488
End of memory		65535

```
tetra limits and features:
```

```
-----
Size of PAD: 84 (200 bytes after HERE)
Type of division: symmetric
Maximum dimension of a character: 8 bits
  Default ASCII range is 0..127 (7 bits)
  Range 128-255 is O.S. dependent
Screen/Block default size: 1024 bytes
Maximum strings size: 255
No limits to the number of vocabularies allowed
Largest unsigned number: 65535
Largest positive signed number: 32767
Largest negative signed number: -32768
Largest unsigned double number: 4294967295
Largest positive signed double number: 2147483647
Largest negative signed double number: -2147483648
```

```
tetra status:
```

```
-----
last compiled word:   edit
last accessed block:  none
last accessed buffer: none
words compiled so far: 00342
```

```
system dictionary status:
```

```
-----
list of immediate words: 36
cold warm bye --> endcase endof of case +loop loop do ?do until while
repeat begin then else if ascii ' [ forget : ; [compile] dliteral
sliteral literal does> ) (s ( editor forth

list of compile-only words: 51
(endcase) endcase (endof) endof (of) of (case) case (+loop) +loop (loop)
loop (do) do (?do) ?do (until) until (while) while (repeat) repeat (begin)
begin leave j i (then) then (else) else (if) if (vocabulary) (forget)
; [compile] (compile) compile exit dliteral (sliteral) sliteral literal
>r r> r@ (does>) does> (abort") (word)

list of defining words: 6
vocabulary 2constant 2variable constant variable create

list of executable-only words: 13
```

edit load-system save-system lsee search ensure require searchlib debug
trace exec-only compile-only immediate

user dictionary status:

list of user words: 0

list of user vocabularies: 0

system directories and files:

system directory: /home/<user>/.tetra/
blocks directory: /home/<user>/.tetra/blocks/
current linked library: /home/<user>/.tetra/tlibrary.ft
resource file: /home/<user>/.tetra.rc (enabled)

Table of Contents

1.	General notes	1
1.1.	Introducing tetra	1
1.2.	The documents	4
1.3.	Definition of terms	8
1.4.	Fulfilled requirements	18
2.	Inside the compiler	20
2.1.	The memory model	20
2.2.	The runtime engine	23
2.2.1.	The execution model	23
2.2.2.	The Program Counter	24
2.2.3.	The 'readline' facility	24
2.2.4.	Program interruption	24
2.2.5.	Crash course	25
2.3.	Strict behaviour	25
2.4.	System constants	26
2.5.	System variables	27
2.6.	Environmental queries	28
2.7.	Error management	29
2.7.1.	Default Error Management	30
2.7.2.	User Error management	30
2.8.	Extensions for Block-treatment	31
2.9.	Extensions for File-treatment	31
2.10.	Using computer ports	32
2.11.	Making sounds with tetra	33
2.11.1.	Using BELL	33
2.11.2.	Using SOUND	34
2.12.	Pipes with tetra	34
2.12.1.	Reading a pipe	34
2.12.2.	Writing to a pipe	34
2.13.	Super-immediate words	35
3.	Words Reference	36
3.1.	Notation in the Reference Manual	36
3.2.	Required Word Set	37
3.3.	The Dictionary	38
3.4.	The Line Editor in detail	84
3.4.1.	Some Line Editor techniques	88
3.5.	The Screen Editor in detail	89
3.5.1.	Status lines	90
3.5.2.	Screen Editor commands	90
3.6.	fig-Forth Compatibility	92
3.7.	Service words	92
3.8.	The integration file tlibrary.ft	94
4.	The built-in debugger	96
4.1.	Using DEBUG	96
4.2.	Using TRACE	97
4.3.	Debug Commands	97
5.	Differences between tetra and the Forth-79 Standard	99
5.1.	Changes	99
5.1.1.	Word sets	99
5.1.2.	-->	99
5.1.3.	BASE	99

5.1.4.	BLOCK and BUFFER	99
5.1.5.	CONTEXT	99
5.1.6.	CURRENT	100
5.1.7.	DPL	100
5.1.8.	FORGET	100
5.1.9.	LITERAL and DLITERAL	100
5.1.10.	PAGE	100
5.1.11.	WORD	100
5.2.	Enhancements	101
5.2.1.	Memory	101
5.2.2.	The (comment	101
5.2.3.	CREATE	101
5.2.4.	DOES>	101
5.2.5.	EMIT	102
5.2.6.	TYPE	102
5.2.7.	WORD	103
6.	Some discussions about Forth-79	104
6.1.	A word about FIND and ' (tick)	104
6.2.	DO . . LOOP / +LOOP problems	104
6.3.	LEAVE a good memory of you	105
6.4.	The three musketeers (weren't they four?)	105
6.4.1.	The three musketeers	105
6.4.2.	The musketeer that wasn't there	106
6.5.	Using BLOCK and BUFFER	107
6.5.1.	BLOCK	107
6.5.2.	BUFFER	108
6.5.3.	Choosing specific blocks directories with USE	109
6.6.	Compiling issues	109
6.6.1.	Compiling tricks	110
6.6.2.	Compilation and Immediacy	110
6.6.3.	Compiling is looking forward	113
6.6.4.	Compiling is thinking	114
6.6.5.	Let me be frank (oops, immediate!)	115
6.6.6.	Compiling is...	116
6.7.	A couple of VOCABULARY problems	117
6.7.1.	Chaining	117
6.7.2.	Immediacy	118
6.8.	The CONTEXT of the CURRENT affairs	119
6.9.	One word (or two) about LITERAL (and DLITERAL)	120
7.	Forth in retrospective	121
7.1.	Forth-77 (or the importance of having good ancestors)	121
7.1.1.	Ancestors? What ancestors?	121
7.1.2.	The provisional does	121
7.1.3.	The byte conception	122
7.1.4.	Influence?	122
7.1.5.	Curiosity killed the cat	123
7.2.	Apple Forth and 64Forth (or the importance of having good cousins)	123
7.2.1.	The Apple Forth-79	123
7.2.2.	The 64Forth	124
7.3.	Conversion of fig-Forth programs	124
7.3.1.	Can I say a WORD?	125
7.3.2.	+LOOP limits	125
7.3.3.	Using VARIABLE properly	126
7.3.4.	The SIGN of destiny	126

7.3.5.	How to FIND it?	127
7.3.6.	A VOCABULARY lookup	127
7.3.7.	Where is it?	128
7.4.	Forth-83 and beyond (or how evolution has won)	128
7.4.1.	The Compilation Address	129
7.4.2.	To be or NOT to be	129
7.4.3.	FIND and ' (tick)	129
7.4.4.	Unsigned index arguments for memory commands	129
7.4.5.	Removal of STATE-dependent words	129
7.4.6.	Improved DO . . LOOP	130
7.4.7.	Division type	130
7.4.8.	Zero-based PICK and ROLL	130
7.4.9.	Improved WORD	130
7.4.10.	Renaming of some words	130
7.5.	The ANSI Forth (or multiply like rabbits)	130
8.	My favourite FD Articles	132
9.	Conclusions	148
	<i>APPENDIX A: Internals</i>	149
	<i>APPENDIX B: The Test Suite</i>	162
	<i>APPENDIX C: The floating point system</i>	163
	<i>APPENDIX D: Starting setup</i>	164